

```

In [ ]: # =====
# Multimodal DINOv3 with Concatenation Fusion (Baseline for Ablation Study)
# - Uses 7-Fold CV  $\approx$  70% Train / 15% Val / 15% Test Overall
# - Serves as a direct comparison against more complex fusion methods like DSAF.
# - MODIFIED: Integrated a comprehensive visualization and metrics pipeline for res
#   matching the reporting structure of the DSAF model for fair comparison.
# =====

import os, time, json, warnings
from copy import deepcopy
from collections import defaultdict
from itertools import cycle
import numpy as np
import pandas as pd

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms
from PIL import Image

from sklearn.model_selection import StratifiedKFold, StratifiedShuffleSplit
from sklearn.metrics import (
    classification_report, accuracy_score, cohen_kappa_score,
    matthews_corrcoef, roc_auc_score, confusion_matrix, roc_curve, auc
)
from sklearn.preprocessing import label_binarize

# Conditional import for transformers
try:
    from transformers import AutoModel
    HAVE_TRANSFORMERS = True
except ImportError:
    HAVE_TRANSFORMERS = False

from torch.amp import GradScaler, autocast

warnings.filterwarnings('ignore')
torch.backends.cudnn.benchmark = True

# Optional FLOPs (best effort)
try:
    from thop import profile
    HAVE_THOP = True
except Exception:
    HAVE_THOP = False

# Optional plotting libs
try:
    import matplotlib.pyplot as plt
    import seaborn as sns
    HAVE_PLOTTING = True
except ImportError:

```

```

HAVE_PLOTTING = False

# -----
# 1) CONFIG - UPDATE THESE TWO PATHS
# -----
class Config:
    DATA_DIR = r"D:\AIUB\Neural\Synthetic_Skin_Disease_Dataset" # <-- update to
    ORIGINAL_METADATA_CSV = r"D:\AIUB\Neural\complete_metadata_m.csv" # <-- update

    RESULTS_DIR = "metadata_Multimodal_DINOv3_Concat_KFold" # <-- Directory for thi
    MODEL_NAME = "facebook/dinov3-vitb16-pretrain-lvd1689m"
    IMG_SIZE = 224
    BATCH_SIZE = 128
    LEARNING_RATE = 1e-4
    WEIGHT_DECAY = 2e-5
    EPOCHS = 200
    NUM_CLASSES = 5
    DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
    USE_AMP = True
    NUM_WORKERS = 0
    PATIENCE = 20
    WARMUP_EPOCHS = 10
    LABEL_SMOOTHING = 0.1
    PARTIALLY_UNFREEZE = False # set True to unfreeze last block of DINOv3

    # K-Fold settings for ~70/15/15 overall
    N_SPLITS = 7
    VAL_WITHIN_REMAINDER = 0.15 / (1.0 - 1.0/7.0) # ≈ 0.175

    # Metadata settings
    META_DROPOUT = 0.3
    META_HIDDEN_DIM = 128
    META_OUT_DIM = 256

    # Fairness: minimum subgroup size for stable metrics
    MIN_SUPPORT = 15

config = Config()
os.makedirs(config.RESULTS_DIR, exist_ok=True)
print("Device:", config.DEVICE)
print(f"Model: DINOv3 with Concatenation Fusion (Baseline)")
print(f"K-Fold: N_SPLITS={config.N_SPLITS} (~Test {100/config.N_SPLITS:.1f}%), Val_
if not HAVE_PLOTTING:
    print("Warning: matplotlib and/or seaborn not found. Plots will be skipped.")

# -----
# 2) CSV STANDARDIZATION
# -----
def standardize_metadata_columns(df: pd.DataFrame) -> pd.DataFrame:
    """Standardizes metadata column names and creates necessary fields."""
    df = df.copy()
    if 'filepath' in df.columns:
        df['filepath'] = df['filepath'].astype(str).str.replace('\\', '/', regex=False)
    if 'filename' not in df.columns:

```

```

    if 'filepath' in df.columns:
        df['filename'] = df['filepath'].apply(lambda p: os.path.basename(str(p))
    else:
        raise ValueError("CSV must include 'filepath' or 'filename'.")
    if 'label' not in df.columns and 'filepath' in df.columns:
        df['label'] = df['filepath'].str.split('/').str[1]
    df['label'] = df['label'].astype(str)
    if 'age_group' not in df.columns:
        if 'age' in df.columns:
            age_num = pd.to_numeric(df['age'], errors='coerce')
            bins = [-1, 17, 29, 44, 64, 200]
            names = ['child', 'young_adult', 'adult', 'middle_aged', 'senior']
            df['age_group'] = pd.cut(age_num, bins=bins, labels=names).astype(str).
        else:
            df['age_group'] = 'unknown'
    if 'gender' not in df.columns:
        if 'dominant_gender' in df.columns:
            g = df['dominant_gender'].astype(str).str.strip().str.lower()
            df['gender'] = g.map({'man': 'male', 'male': 'male', 'woman': 'female', 'fema
        else:
            df['gender'] = 'unknown'
    if 'skin_tone' not in df.columns:
        if 'dominant_race' in df.columns:
            df['skin_tone'] = df['dominant_race'].astype(str).str.strip().str.lower
        else:
            df['skin_tone'] = 'unknown'
    required = ['filename', 'label', 'age_group', 'gender', 'skin_tone']
    if any(c not in df.columns for c in required):
        raise ValueError(f"Missing required columns after standardization.")
    return df

# -----
# 3) DATASET & VOCAB
# -----
class GPUGAugOnlyNorm(nn.Module):
    """Module for GPU-side normalization."""
    def __init__(self):
        super().__init__()
        self.normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.22
    @torch.no_grad()
    def forward(self, x): return self.normalize(x)

class MetaVocab:
    """Creates and manages vocabularies for metadata fields."""
    def __init__(self, df_train, cols=('age_group', 'gender', 'skin_tone')):
        self.cols = cols
        self.unk_token = "<UNK>"
        self.vocabs = {}
        for c in cols:
            uniq = sorted([self.unk_token] + sorted(df_train[c].dropna().astype(str)
            self.vocabs[c] = {v:i for i,v in enumerate(uniq)}
        self.sizes = {c: len(self.vocabs[c]) for c in cols}
    def encode_row(self, row):
        out = {}
        for c in self.cols:

```

```

        v = str(row[c]) if pd.notnull(row[c]) else self.unk_token
        out[c] = self.vocabs[c].get(v, self.vocabs[c][self.unk_token])
    return out

class MultimodalDermDataset(Dataset):
    """Dataset class for images and metadata, with optional preloading."""
    def __init__(self, config, df, class_to_idx, meta_vocab, transform=None):
        self.config = config
        self.df = df.reset_index(drop=True)
        self.class_to_idx = class_to_idx
        self.transform = transform
        self.meta_vocab = meta_vocab
        self.image_paths, self.labels, self.meta = [], [], []
        use_filepath = 'filepath' in self.df.columns
        for _, row in self.df.iterrows():
            if use_filepath and pd.notnull(row.get('filepath', None)):
                raw = str(row['filepath'])
                p = raw if os.path.isabs(raw) else os.path.join(config.DATA_DIR, raw)
            else:
                p = os.path.join(config.DATA_DIR, str(row['split']), str(row['label']))
            self.image_paths.append(p)
            self.labels.append(self.class_to_idx[str(row['label'])])
            self.meta.append(self.meta_vocab.encode_row(row))
        self.images = []
        for p in self.image_paths:
            with Image.open(p).convert("RGB") as im:
                self.images.append(im.copy())

    def __len__(self): return len(self.labels)
    def __getitem__(self, idx):
        image = self.images[idx]
        if self.transform: image = self.transform(image)
        return image, self.labels[idx], self.meta[idx]

# -----
# 4) MODEL (DINOv3 + Metadata Concatenation)
# -----
class MetadataEncoder(nn.Module):
    """Encodes categorical metadata into a dense feature vector."""
    def __init__(self, meta_vocab, emb_dims=None, hidden=128, out_dim=256, dropout=0.5):
        super().__init__()
        sizes = meta_vocab.sizes
        if emb_dims is None:
            emb_dims = {
                'age_group': min(16, max(4, sizes['age_group'])),
                'gender': min(8, max(3, sizes['gender'])),
                'skin_tone': min(16, max(4, sizes['skin_tone'])),
            }
        self.emb_age = nn.Embedding(sizes['age_group'], emb_dims['age_group'])
        self.emb_gen = nn.Embedding(sizes['gender'], emb_dims['gender'])
        self.emb_tone = nn.Embedding(sizes['skin_tone'], emb_dims['skin_tone'])
        concat_dim = sum(emb_dims.values())
        self.mlp = nn.Sequential(
            nn.Linear(concat_dim, hidden), nn.GELU(), nn.Dropout(dropout),
            nn.Linear(hidden, out_dim), nn.GELU()
        )

```

```

    )
    def forward(self, meta_idx):
        a = self.emb_age(meta_idx['age_group'])
        g = self.emb_gen(meta_idx['gender'])
        s = self.emb_tone(meta_idx['skin_tone'])
        return self.mlp(torch.cat([a,g,s], dim=-1))

class DINOv3MultimodalConcat(nn.Module):
    """
    A multimodal model using a pretrained DINOv3 backbone.
    Features from the image backbone's [CLS] token are concatenated
    with features from the metadata encoder and passed to a final classifier.
    This serves as a strong but simple multimodal baseline.
    """
    def __init__(self, model_name, num_classes, meta_vocab, meta_hidden_dim, meta_out_dim, meta_dropout):
        super().__init__()
        if not HAVE_TRANSFORMERS:
            raise ImportError("Please install transformers: pip install transformers")

        # Image backbone (DINOv3)
        self.backbone = AutoModel.from_pretrained(model_name)

        # Freeze all backbone parameters initially
        for param in self.backbone.parameters():
            param.requires_grad = False

        # Unfreeze the final transformer block if configured
        if config.PARTIALLY_UNFREEZE and hasattr(self.backbone, "vit") and hasattr(self.backbone.vit.encoder, "layer"):
            for param in self.backbone.vit.encoder.layer[-1].parameters():
                param.requires_grad = True

        image_feature_dim = self.backbone.config.hidden_size # e.g., 768 for ViT-B

        # Metadata encoder
        self.meta_encoder = MetadataEncoder(meta_vocab,
                                            hidden_dim=meta_hidden_dim,
                                            out_dim=meta_out_dim,
                                            dropout=meta_dropout)

        # Fusion and Classifier Head
        self.classifier = nn.Sequential(
            nn.Linear(image_feature_dim + meta_out_dim, 512),
            nn.BatchNorm1d(512),
            nn.GELU(),
            nn.Dropout(0.5),
            nn.Linear(512, num_classes)
        )

    def forward(self, pixel_values, meta_idx):
        self.backbone.eval() if not config.PARTIALLY_UNFREEZE else None
        with torch.no_grad() if not config.PARTIALLY_UNFREEZE else torch.enable_grad():
            outputs = self.backbone(pixel_values=pixel_values)

        img_features = outputs.last_hidden_state[:, 0]
        meta_features = self.meta_encoder(meta_idx)
        combined_features = torch.cat((img_features, meta_features), dim=1)

```

```

        output = self.classifier(combined_features)
        return output

# -----
# 5) TRAIN / EVAL HELPERS
# -----
def _to_device_meta(batch_meta, device):
    if isinstance(batch_meta, dict):
        ag = torch.as_tensor(batch_meta['age_group'], dtype=torch.long, device=device)
        ge = torch.as_tensor(batch_meta['gender'], dtype=torch.long, device=device)
        st = torch.as_tensor(batch_meta['skin_tone'], dtype=torch.long, device=device)
        return {'age_group': ag, 'gender': ge, 'skin_tone': st}
    else:
        ag = torch.tensor([m['age_group'] for m in batch_meta], dtype=torch.long, device=device)
        ge = torch.tensor([m['gender'] for m in batch_meta], dtype=torch.long, device=device)
        st = torch.tensor([m['skin_tone'] for m in batch_meta], dtype=torch.long, device=device)
        return {'age_group': ag, 'gender': ge, 'skin_tone': st}

def make_data loaders_from_df(config, df_std):
    """Creates dataloaders for Train, Val, and Test splits from a DataFrame."""
    classes = sorted(df_std[df_std['split']=='Train']['label'].astype(str).unique())
    class_to_idx = {c:i for i,c in enumerate(classes)}
    config.NUM_CLASSES = len(classes)
    meta_vocab = MetaVocab(df_std[df_std['split']=='Train'])
    train_tfm = transforms.Compose([
        transforms.Resize((config.IMG_SIZE, config.IMG_SIZE)),
        transforms.RandomHorizontalFlip(),
        transforms.RandomApply([transforms.ColorJitter(0.4,0.4,0.2,0.1)], p=0.8),
        transforms.RandomRotation(15),
        transforms.ToTensor(),
    ])
    eval_tfm = transforms.Compose([
        transforms.Resize((config.IMG_SIZE, config.IMG_SIZE)),
        transforms.ToTensor(),
    ])
    dsets = {s: MultimodalDermDataset(config, df_std[df_std['split']==s], class_to_idx[s],
                                     for s, t in [('Train', train_tfm), ('Val', eval_tfm), ('Test', eval_tfm)])
    dls = {s: DataLoader(dsets[s], batch_size=config.BATCH_SIZE, shuffle=(s=='Train'),
                       num_workers=config.NUM_WORKERS, pin_memory=True) for s in dsets}
    return dls, {k: len(v) for k,v in dsets.items()}, classes, meta_vocab

def plot_and_save_history(hist, fold_id, results_dir):
    """Plots training and validation loss and accuracy and saves them to a file."""
    if not HAVE_PLOTTING: return

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))
    epochs = range(1, len(hist['train_loss']) + 1)
    ax1.plot(epochs, hist['train_loss'], 'bo-', label='Training loss')
    ax1.plot(epochs, hist['val_loss'], 'ro-', label='Validation loss')
    ax1.set_title(f'Training and Validation Loss (Fold {fold_id})'); ax1.set_xlabel('Epoch')
    ax1.legend(); ax1.grid(True)
    ax2.plot(epochs, hist['train_acc'], 'bo-', label='Training accuracy')
    ax2.plot(epochs, hist['val_acc'], 'ro-', label='Validation accuracy')
    ax2.set_title(f'Training and Validation Accuracy (Fold {fold_id})'); ax2.set_xlabel('Epoch')
    ax2.legend(); ax2.grid(True)

```

```

plt.tight_layout()
history_path = os.path.join(results_dir, f'training_history_fold_{fold_id}.png')
plt.savefig(history_path, dpi=300); plt.show(); plt.close()
print(f"Saved training history plot to: {history_path}")

def train_model(model, dls, sizes, num_epochs, fold_id):
    start = time.time()
    best = deepcopy(model.state_dict())
    best_val_loss = float('inf')
    patience = 0
    history = defaultdict(list)
    scaler = GradScaler(enabled=config.USE_AMP)
    gpu_norm = GPUAugOnlyNorm().to(config.DEVICE)
    optimizer = optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()),
                                lr=config.LEARNING_RATE, weight_decay=config.WEIGHT_DECAY)

    def lr_lambda(ep):
        if ep < config.WARMUP_EPOCHS: return float(ep+1)/config.WARMUP_EPOCHS
        return max(0.0, 0.5 * (1.0 + np.cos(np.pi * (ep - config.WARMUP_EPOCHS) / (
            config.EPOCHS - config.WARMUP_EPOCHS))))

    scheduler = optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)
    criterion = nn.CrossEntropyLoss(label_smoothing=config.LABEL_SMOOTHING)
    best_model_path = os.path.join(config.RESULTS_DIR, f'best_model_fold_{fold_id}.pth')

    print(f"\nSTART TRAINING (DINOv3 + Concat) for {num_epochs} epochs...")
    for epoch in range(num_epochs):
        for phase in ['Train', 'Val']:
            model.train(phase == 'Train')
            running_loss, running_corrects = 0.0, 0
            for images, labels, metas in dls[phase]:
                images = gpu_norm(images.to(config.DEVICE))
                labels = labels.to(config.DEVICE, dtype=torch.long)
                meta_idx = _to_device_meta(metas, config.DEVICE)
                optimizer.zero_grad(set_to_none=True)
                with autocast(device_type=config.DEVICE.split(':')[0], enabled=config.USE_AMP):
                    logits = model(pixel_values=images, meta_idx=meta_idx)
                    loss = criterion(logits, labels)
                if phase == 'Train':
                    scaler.scale(loss).backward(); scaler.step(optimizer); scaler.update()
            running_loss += loss.item() * images.size(0)
            running_corrects += (logits.argmax(1) == labels).sum().item()

        epoch_loss = running_loss / sizes[phase]
        epoch_acc = running_corrects / sizes[phase]
        history[f'{phase.lower()}_loss'].append(epoch_loss)
        history[f'{phase.lower()}_acc'].append(epoch_acc)

        if phase == 'Val':
            if epoch_loss < best_val_loss:
                best_val_loss = epoch_loss; best = deepcopy(model.state_dict())
                torch.save(model.state_dict(), best_model_path)
            else:
                patience += 1
            scheduler.step()
        print(f"Epoch {epoch+1}/{num_epochs} | Train Loss: {history['train_loss'][-1]:.4f} | Val Loss: {history['val_loss'][-1]:.4f} Acc: {history['val_acc'][-1]:.4f}")
        if patience >= config.PATIENCE:
            print("Early stopping triggered."); break

```

```

print(f"Training complete in {(time.time()-start)/60.0:.1f} minutes. Best Val L
print(f"Best model for fold saved to: {best_model_path}")
model.load_state_dict(best)
plot_and_save_history(history, fold_id, config.RESULTS_DIR)
return model, history

# -----
# 6) EVALUATION & FAIRNESS METRICS
# -----

def _print_section(title):
    print("\n" + "="*80); print(title.upper()); print("="*80 + "\n")

def _group_fairness_metrics(y_true, y_pred, groups, num_classes, min_support=None):
    uniq_groups = sorted(pd.Series(groups).astype(str).unique())
    per_group_acc, per_group_tprs, counts = {}, {}, {}
    y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
    for g in uniq_groups:
        mask = (np.asarray(groups) == g); counts[g] = int(mask.sum())
        yt, yp = y_true[mask], y_pred[mask]
        per_group_acc[g] = (yt == yp).mean() if len(yt) > 0 else np.nan
        tprs = [( (yt == c) & (yp == c)).sum() / ((yt == c).sum() or 1) ) for c in
        per_group_tprs[g] = np.array(tprs, dtype=float)
    filt_keys = [g for g in uniq_groups if counts[g] >= min_support] if min_support
    wga = float(np.nanmin([per_group_acc.get(g, np.nan) for g in filt_keys])) if fi
    max_gap, worst_pair = 0.0, (None, None)
    for i in range(len(filt_keys)):
        for j in range(i + 1, len(filt_keys)):
            gi, gj = filt_keys[i], filt_keys[j]
            a, b = per_group_tprs[gi], per_group_tprs[gj]
            mask = ~(np.isnan(a) | np.isnan(b))
            if mask.sum() > 0:
                gap = float(np.nanmean(np.abs(a[mask] - b[mask])))
                if gap > max_gap: max_gap, worst_pair = gap, (gi, gj)
    return {'wga': wga, 'eod': max_gap, 'worst_pair': worst_pair, 'keys_used': filt

def plot_and_save_roc_curves(y_true_bin, y_prob, class_names, fold_id, results_dir)
    if not HAVE_PLOTTING: return
    n_classes = len(class_names)
    fpr, tpr, roc_auc = dict(), dict(), dict()
    for i in range(n_classes):
        fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_prob[:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])
    all_fpr = np.unique(np.concatenate([fpr[i] for i in range(n_classes)]))
    mean_tpr = np.zeros_like(all_fpr)
    for i in range(n_classes): mean_tpr += np.interp(all_fpr, fpr[i], tpr[i])
    mean_tpr /= n_classes
    fpr["macro"], tpr["macro"], roc_auc["macro"] = all_fpr, mean_tpr, auc(all_fpr,
    plt.figure(figsize=(12, 10))
    colors = cycle(['aqua', 'darkorange', 'cornflowerblue', 'green', 'red', 'purple')
    for i, color in zip(range(n_classes), colors):
        plt.plot(fpr[i], tpr[i], color=color, lw=2, label=f'ROC of {class_names[i]}')
    plt.plot(fpr["macro"], tpr["macro"], label=f'Macro-average ROC (AUC = {roc_auc[
    plt.plot([0, 1], [0, 1], 'k--', lw=2); plt.xlim([0.0, 1.0]); plt.ylim([0.0, 1.0
    plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate'); plt.title(
    plt.legend(loc="lower right"); plt.grid(True)

```

```

roc_path = os.path.join(results_dir, f'roc_curves_fold_{fold_id}.png')
plt.savefig(roc_path, dpi=300); plt.show(); plt.close()
print(f"Saved ROC curves plot to: {roc_path}")

def plot_and_save_scalar_metrics(metrics_dict, fold_id, results_dir):
    if not HAVE_PLOTTING: return
    names, values = list(metrics_dict.keys()), list(metrics_dict.values())
    plt.figure(figsize=(10, 6))
    bars = sns.barplot(x=names, y=values, palette="viridis")
    plt.title(f'Overall Scalar Metrics Summary - Fold {fold_id}'); plt.ylabel('Score')
    for bar in bars.patches:
        plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01, f'{bar.get_height():.2f}')
    scalar_path = os.path.join(results_dir, f'scalar_metrics_fold_{fold_id}.png')
    plt.savefig(scalar_path, dpi=300); plt.show(); plt.close()
    print(f"Saved scalar metrics plot to: {scalar_path}")

def evaluate_and_visualize(model, dl_test, class_names, df_fold, fold_id):
    model.eval()
    gpu_norm = GPUAugOnlyNorm().to(config.DEVICE)
    y_true, y_pred, y_prob = [], [], []
    with torch.no_grad():
        for images, labels, metas in dl_test:
            images = gpu_norm(images.to(config.DEVICE))
            meta_idx = _to_device_meta(metas, config.DEVICE)
            logits = model(pixel_values=images, meta_idx=meta_idx)
            y_true.extend(labels.numpy())
            y_pred.extend(logits.argmax(1).cpu().numpy())
            y_prob.extend(torch.softmax(logits, dim=1).cpu().numpy())
    y_true, y_pred, y_prob = np.array(y_true), np.array(y_pred), np.array(y_prob)

    _print_section(f"OVERALL PERFORMANCE (FOLD {fold_id})")
    print(classification_report(y_true, y_pred, target_names=class_names, zero_divi

acc = accuracy_score(y_true, y_pred)
kappa = cohen_kappa_score(y_true, y_pred)
mcc = matthews_corrcoef(y_true, y_pred)
try:
    y_true_bin = label_binarize(y_true, classes=np.arange(len(class_names)))
    roc_auc_macro = roc_auc_score(y_true_bin, y_prob, average='macro', multi_cl
except Exception: y_true_bin, roc_auc_macro = None, float('nan')
print(f"\n--- Overall Scalar Metrics ---\nAccuracy: {acc:.4f}\nMacro ROC AUC: {

if HAVE_PLOTTING:
    cm = confusion_matrix(y_true, y_pred, labels=np.arange(len(class_names)))
    cm_norm = np.nan_to_num(cm.astype('float') / cm.sum(axis=1)[:, np.newaxis])
    plt.figure(figsize=(12, 10))
    sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues', xticklabels=class
    plt.title(f'Normalized Confusion Matrix (Fold {fold_id})'); plt.ylabel('True
    cm_path = os.path.join(config.RESULTS_DIR, f'confusion_matrix_fold_{fold_id}
    plt.savefig(cm_path, dpi=300); plt.show(); plt.close()
    print(f"Saved normalized confusion matrix to: {cm_path}")
    if y_true_bin is not None: plot_and_save_roc_curves(y_true_bin, y_prob, cla
    plot_and_save_scalar_metrics({'Accuracy': acc, 'Macro AUC': roc_auc_macro,

df_test = df_fold[df_fold['split'] == 'Test'].reset_index(drop=True)
df_test = df_test.iloc[:len(y_true)] # Align rows just in case

```

```

age_disp = df_test['age_group'].map({'child': '0-17', 'young_adult': '18-59', '
# *** FIX IS APPLIED HERE ***
# Define the full set of labels to expect in any report.
full_labels = np.arange(len(class_names))

for group_name, series in [('GENDER', df_test['gender']), ('AGE', age_disp), ('
    _print_section(f"PERFORMANCE BY {group_name}")
    for val in sorted(series.unique()):
        if val == 'unknown': continue
        mask = (series == val).values
        if mask.sum() > 0:
            print(f"--- Subgroup: {val} ---\n")
            # Add the 'labels' parameter to the classification_report call
            print(classification_report(y_true[mask], y_pred[mask],
                                      target_names=class_names,
                                      labels=full_labels, # This ensures all
                                      zero_division=0))

groups_composite = (age_disp.astype(str) + "|" + df_test['gender'].astype(str)
fair_all = _group_fairness_metrics(y_true, y_pred, groups_composite, len(class_
fair_filt = _group_fairness_metrics(y_true, y_pred, groups_composite, len(class
print("\n" + "="*80 + f"\nFAIRNESS SUMMARY (ALL GROUPS) | WGA: {fair_all['wga']}
print("\n" + "="*80 + f"\nFAIRNESS SUMMARY (FILTERED, n>={config.MIN_SUPPORT})

def acc_counts(s, yt, yp): return {str(g): ((yt[s==g] == yp[s==g]).sum(), (s==g
pooled_info = {'gender': acc_counts(df_test['gender'], y_true, y_pred), 'age':

return {'accuracy': acc, 'roc_auc_macro': roc_auc_macro, 'kappa': kappa, 'mcc':

# -----
# 7) K-FOLD CROSS-VALIDATION ORCHESTRATION
# -----
def build_kfold_splits(df_std, n_splits, val_within_remainder, seed=42):
    df, y = df_std.copy().reset_index(drop=True), df_std['label'].astype(str).value
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
    folds = []
    for i, (trainval_idx, test_idx) in enumerate(skf.split(np.arange(len(df)), y),
        sss2 = StratifiedShuffleSplit(n_splits=1, test_size=val_within_remainder, r
        tr_rel, val_rel = next(sss2.split(trainval_idx, y[trainval_idx]))
        dfi = df.copy(); dfi['split'] = 'Ignore'
        dfi.loc[trainval_idx[tr_rel], 'split'], dfi.loc[trainval_idx[val_rel], 'spl
        folds.append(dfi[dfi['split'] != 'Ignore'])
    return folds

def run_one_fold(fold_df, fold_id, config):
    print("\n" + "="*88 + f"\n### FOLD {fold_id} ###\n" + "="*88)
    dls, sizes, class_names, meta_vocab = make_data loaders_from_df(config, fold_df)
    model = DINOv3MultimodalConcat(model_name=config.MODEL_NAME, num_classes=len(cl
    print(f"Trainable params: {sum(p.numel() for p in model.parameters() if p.requi
    model, _ = train_model(model, dls, sizes, num_epochs=config.EPOCHS, fold_id=fol
    return evaluate_and_visualize(model, dls['Test'], class_names, fold_df, fold_id

def kfold_cross_val(df_std, config):
    folds = build_kfold_splits(df_std, config.N_SPLITS, config.VAL_WITHIN_REMAINDER
    per_fold = [run_one_fold(fold, i + 1, config) for i, fold in enumerate(folds)]

```

```

perf = pd.DataFrame(per_fold)

def mean_std(col): return (perf[col].mean(), perf[col].std())
summary = {k: mean_std(k) for k in ['accuracy', 'roc_auc_macro', 'kappa', 'mcc']}

print("\n" + "="*88 + f"\nCross-validated over {config.N_SPLITS} folds\n" + "=")
for k, (m, s) in summary.items(): print(f"{k.replace('_', ' ').title():<20}: {m} {s}")

def merge_pooled(key):
    agg = defaultdict(lambda: [0, 0])
    for m in per_fold:
        for g, (c, t) in m['pooled_groups'][key].items(): agg[g][0] += c; agg[g][1] += t
    return sorted([(g, c/t, t) for g, (c, t) in agg.items() if t>0], key=lambda x: x[1])

def print_table(title, rows):
    print(f"\n--- {title} ---"); print(f"{'Group':>35s} {'Acc':>8s} {'Count':>8s}")
    for g, acc, cnt in rows:
        if cnt >= config.MIN_SUPPORT: print(f"{g:>35s} {acc:>8.3f} {cnt:>8d}")

for key, name in [('gender', 'GENDER'), ('age', 'AGE'), ('skin_tone', 'SKIN TONE')]:
    print_table(f"POOLED OVERALL ACCURACY BY {name} (n>={config.MIN_SUPPORT})",
                merge_pooled(key))

perf['fold'] = range(1, len(perf) + 1); perf.set_index('fold', inplace=True)
perf.to_csv(os.path.join(config.RESULTS_DIR, "kfold_performance.csv"))
pd.DataFrame({k: {'mean': v[0], 'std': v[1]} for k, v in summary.items()}).T.to_csv(
    os.path.join(config.RESULTS_DIR, "summary.csv"))

print(f"\nResults, models, and plots saved to '{config.RESULTS_DIR}'")

# -----
# 8) MAIN EXECUTION
# -----
if __name__ == '__main__':
    try:
        df_orig = pd.read_csv(config.ORIGINAL_METADATA_CSV)
        df_std = standardize_metadata_columns(df_orig)
        print(f"Loaded and standardized metadata: {len(df_std)} rows.")
        kfold_cross_val(df_std, config)
    except FileNotFoundError:
        print(f"ERROR: Metadata file not found at '{config.ORIGINAL_METADATA_CSV}'")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
        import traceback
        traceback.print_exc()

```

Device: cuda
Model: DINOv3 with Concatenation Fusion (Baseline)
K-Fold: N_SPLITS=7 (~Test 14.3%), Val_in_remainder=0.175
Loaded and standardized metadata: 565 rows.

```
#####  
####  
### FOLD 1 ###  
#####  
####  
Trainable params: 0.56M
```

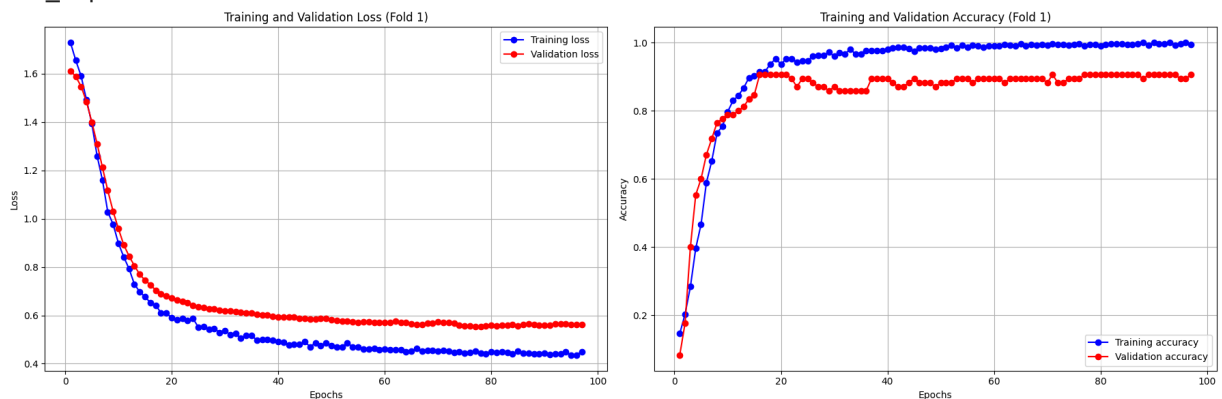
START TRAINING (DINOv3 + Concat) for 200 epochs...

Epoch 1/200		Train Loss: 1.7283	Acc: 0.1454		Val Loss: 1.6117	Acc: 0.0824		LR: 2.00e-05
Epoch 2/200		Train Loss: 1.6544	Acc: 0.2030		Val Loss: 1.5880	Acc: 0.1765		LR: 3.00e-05
Epoch 3/200		Train Loss: 1.5897	Acc: 0.2832		Val Loss: 1.5470	Acc: 0.4000		LR: 4.00e-05
Epoch 4/200		Train Loss: 1.4919	Acc: 0.3960		Val Loss: 1.4842	Acc: 0.5529		LR: 5.00e-05
Epoch 5/200		Train Loss: 1.3943	Acc: 0.4662		Val Loss: 1.4005	Acc: 0.6000		LR: 6.00e-05
Epoch 6/200		Train Loss: 1.2594	Acc: 0.5890		Val Loss: 1.3096	Acc: 0.6706		LR: 7.00e-05
Epoch 7/200		Train Loss: 1.1600	Acc: 0.6516		Val Loss: 1.2127	Acc: 0.7176		LR: 8.00e-05
Epoch 8/200		Train Loss: 1.0284	Acc: 0.7343		Val Loss: 1.1166	Acc: 0.7647		LR: 9.00e-05
Epoch 9/200		Train Loss: 0.9770	Acc: 0.7544		Val Loss: 1.0308	Acc: 0.7765		LR: 1.00e-04
Epoch 10/200		Train Loss: 0.8987	Acc: 0.7970		Val Loss: 0.9589	Acc: 0.7882		LR: 1.00e-04
Epoch 11/200		Train Loss: 0.8417	Acc: 0.8296		Val Loss: 0.8918	Acc: 0.7882		LR: 1.00e-04
Epoch 12/200		Train Loss: 0.7930	Acc: 0.8446		Val Loss: 0.8453	Acc: 0.8000		LR: 1.00e-04
Epoch 13/200		Train Loss: 0.7290	Acc: 0.8672		Val Loss: 0.8055	Acc: 0.8118		LR: 9.99e-05
Epoch 14/200		Train Loss: 0.6975	Acc: 0.8972		Val Loss: 0.7696	Acc: 0.8353		LR: 9.99e-05
Epoch 15/200		Train Loss: 0.6768	Acc: 0.9023		Val Loss: 0.7453	Acc: 0.8471		LR: 9.98e-05
Epoch 16/200		Train Loss: 0.6527	Acc: 0.9148		Val Loss: 0.7265	Acc: 0.9059		LR: 9.98e-05
Epoch 17/200		Train Loss: 0.6423	Acc: 0.9148		Val Loss: 0.7042	Acc: 0.9059		LR: 9.97e-05
Epoch 18/200		Train Loss: 0.6102	Acc: 0.9373		Val Loss: 0.6902	Acc: 0.9059		LR: 9.96e-05
Epoch 19/200		Train Loss: 0.6093	Acc: 0.9524		Val Loss: 0.6810	Acc: 0.9059		LR: 9.94e-05
Epoch 20/200		Train Loss: 0.5904	Acc: 0.9373		Val Loss: 0.6717	Acc: 0.9059		LR: 9.93e-05
Epoch 21/200		Train Loss: 0.5832	Acc: 0.9524		Val Loss: 0.6641	Acc: 0.9059		LR: 9.92e-05
Epoch 22/200		Train Loss: 0.5868	Acc: 0.9524		Val Loss: 0.6566	Acc: 0.8941		LR:

9.90e-05
Epoch 23/200 | Train Loss: 0.5799 Acc: 0.9424 | Val Loss: 0.6521 Acc: 0.8706 | LR: 9.88e-05
Epoch 24/200 | Train Loss: 0.5874 Acc: 0.9474 | Val Loss: 0.6414 Acc: 0.8941 | LR: 9.87e-05
Epoch 25/200 | Train Loss: 0.5514 Acc: 0.9474 | Val Loss: 0.6366 Acc: 0.8941 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.5526 Acc: 0.9599 | Val Loss: 0.6322 Acc: 0.8824 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.5423 Acc: 0.9624 | Val Loss: 0.6281 Acc: 0.8706 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.5458 Acc: 0.9624 | Val Loss: 0.6263 Acc: 0.8706 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.5291 Acc: 0.9724 | Val Loss: 0.6220 Acc: 0.8588 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.5377 Acc: 0.9599 | Val Loss: 0.6172 Acc: 0.8706 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.5197 Acc: 0.9699 | Val Loss: 0.6182 Acc: 0.8588 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.5247 Acc: 0.9674 | Val Loss: 0.6169 Acc: 0.8588 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.5051 Acc: 0.9799 | Val Loss: 0.6119 Acc: 0.8588 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.5163 Acc: 0.9674 | Val Loss: 0.6106 Acc: 0.8588 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.5165 Acc: 0.9674 | Val Loss: 0.6101 Acc: 0.8588 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.4974 Acc: 0.9774 | Val Loss: 0.6053 Acc: 0.8588 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.5010 Acc: 0.9774 | Val Loss: 0.6017 Acc: 0.8941 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.4991 Acc: 0.9774 | Val Loss: 0.6004 Acc: 0.8941 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.4969 Acc: 0.9774 | Val Loss: 0.5967 Acc: 0.8941 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.4930 Acc: 0.9799 | Val Loss: 0.5936 Acc: 0.8941 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.4878 Acc: 0.9850 | Val Loss: 0.5918 Acc: 0.8824 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.4781 Acc: 0.9875 | Val Loss: 0.5935 Acc: 0.8706 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.4814 Acc: 0.9875 | Val Loss: 0.5940 Acc: 0.8706 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.4796 Acc: 0.9825 | Val Loss: 0.5887 Acc: 0.8824 | LR: 9.23e-05
Epoch 45/200 | Train Loss: 0.4910 Acc: 0.9749 | Val Loss: 0.5864 Acc: 0.8941 | LR: 9.19e-05
Epoch 46/200 | Train Loss: 0.4700 Acc: 0.9850 | Val Loss: 0.5853 Acc: 0.8824 | LR: 9.14e-05
Epoch 47/200 | Train Loss: 0.4871 Acc: 0.9850 | Val Loss: 0.5842 Acc: 0.8824 | LR: 9.09e-05
Epoch 48/200 | Train Loss: 0.4736 Acc: 0.9850 | Val Loss: 0.5863 Acc: 0.8824 | LR: 9.05e-05
Epoch 49/200 | Train Loss: 0.4848 Acc: 0.9799 | Val Loss: 0.5872 Acc: 0.8706 | LR: 9.00e-05
Epoch 50/200 | Train Loss: 0.4746 Acc: 0.9825 | Val Loss: 0.5819 Acc: 0.8824 | LR:

8.95e-05
Epoch 51/200 | Train Loss: 0.4689 Acc: 0.9875 | Val Loss: 0.5780 Acc: 0.8824 | LR: 8.89e-05
Epoch 52/200 | Train Loss: 0.4704 Acc: 0.9925 | Val Loss: 0.5767 Acc: 0.8824 | LR: 8.84e-05
Epoch 53/200 | Train Loss: 0.4858 Acc: 0.9850 | Val Loss: 0.5766 Acc: 0.8941 | LR: 8.79e-05
Epoch 54/200 | Train Loss: 0.4698 Acc: 0.9925 | Val Loss: 0.5726 Acc: 0.8941 | LR: 8.73e-05
Epoch 55/200 | Train Loss: 0.4704 Acc: 0.9875 | Val Loss: 0.5703 Acc: 0.8941 | LR: 8.68e-05
Epoch 56/200 | Train Loss: 0.4614 Acc: 0.9925 | Val Loss: 0.5736 Acc: 0.8824 | LR: 8.62e-05
Epoch 57/200 | Train Loss: 0.4602 Acc: 0.9900 | Val Loss: 0.5721 Acc: 0.8941 | LR: 8.56e-05
Epoch 58/200 | Train Loss: 0.4648 Acc: 0.9875 | Val Loss: 0.5715 Acc: 0.8941 | LR: 8.51e-05
Epoch 59/200 | Train Loss: 0.4573 Acc: 0.9900 | Val Loss: 0.5709 Acc: 0.8941 | LR: 8.45e-05
Epoch 60/200 | Train Loss: 0.4618 Acc: 0.9900 | Val Loss: 0.5716 Acc: 0.8941 | LR: 8.39e-05
Epoch 61/200 | Train Loss: 0.4592 Acc: 0.9900 | Val Loss: 0.5716 Acc: 0.8941 | LR: 8.33e-05
Epoch 62/200 | Train Loss: 0.4582 Acc: 0.9950 | Val Loss: 0.5750 Acc: 0.8824 | LR: 8.26e-05
Epoch 63/200 | Train Loss: 0.4574 Acc: 0.9925 | Val Loss: 0.5704 Acc: 0.8941 | LR: 8.20e-05
Epoch 64/200 | Train Loss: 0.4505 Acc: 0.9900 | Val Loss: 0.5696 Acc: 0.8941 | LR: 8.14e-05
Epoch 65/200 | Train Loss: 0.4517 Acc: 0.9975 | Val Loss: 0.5651 Acc: 0.8941 | LR: 8.07e-05
Epoch 66/200 | Train Loss: 0.4641 Acc: 0.9900 | Val Loss: 0.5624 Acc: 0.8941 | LR: 8.01e-05
Epoch 67/200 | Train Loss: 0.4528 Acc: 0.9950 | Val Loss: 0.5619 Acc: 0.8941 | LR: 7.94e-05
Epoch 68/200 | Train Loss: 0.4539 Acc: 0.9925 | Val Loss: 0.5668 Acc: 0.8941 | LR: 7.87e-05
Epoch 69/200 | Train Loss: 0.4541 Acc: 0.9950 | Val Loss: 0.5671 Acc: 0.8941 | LR: 7.80e-05
Epoch 70/200 | Train Loss: 0.4519 Acc: 0.9925 | Val Loss: 0.5720 Acc: 0.8824 | LR: 7.73e-05
Epoch 71/200 | Train Loss: 0.4536 Acc: 0.9975 | Val Loss: 0.5705 Acc: 0.9059 | LR: 7.67e-05
Epoch 72/200 | Train Loss: 0.4532 Acc: 0.9950 | Val Loss: 0.5699 Acc: 0.8824 | LR: 7.59e-05
Epoch 73/200 | Train Loss: 0.4463 Acc: 0.9950 | Val Loss: 0.5681 Acc: 0.8824 | LR: 7.52e-05
Epoch 74/200 | Train Loss: 0.4497 Acc: 0.9925 | Val Loss: 0.5587 Acc: 0.8941 | LR: 7.45e-05
Epoch 75/200 | Train Loss: 0.4444 Acc: 0.9950 | Val Loss: 0.5552 Acc: 0.8941 | LR: 7.38e-05
Epoch 76/200 | Train Loss: 0.4452 Acc: 0.9975 | Val Loss: 0.5555 Acc: 0.8941 | LR: 7.31e-05
Epoch 77/200 | Train Loss: 0.4535 Acc: 0.9900 | Val Loss: 0.5525 Acc: 0.9059 | LR: 7.23e-05
Epoch 78/200 | Train Loss: 0.4446 Acc: 0.9950 | Val Loss: 0.5540 Acc: 0.9059 | LR:

7.16e-05
Epoch 79/200 | Train Loss: 0.4421 Acc: 0.9950 | Val Loss: 0.5554 Acc: 0.9059 | LR: 7.08e-05
Epoch 80/200 | Train Loss: 0.4494 Acc: 0.9900 | Val Loss: 0.5594 Acc: 0.9059 | LR: 7.01e-05
Epoch 81/200 | Train Loss: 0.4456 Acc: 0.9950 | Val Loss: 0.5573 Acc: 0.9059 | LR: 6.93e-05
Epoch 82/200 | Train Loss: 0.4481 Acc: 0.9975 | Val Loss: 0.5595 Acc: 0.9059 | LR: 6.86e-05
Epoch 83/200 | Train Loss: 0.4454 Acc: 0.9975 | Val Loss: 0.5590 Acc: 0.9059 | LR: 6.78e-05
Epoch 84/200 | Train Loss: 0.4410 Acc: 0.9975 | Val Loss: 0.5621 Acc: 0.9059 | LR: 6.70e-05
Epoch 85/200 | Train Loss: 0.4525 Acc: 0.9950 | Val Loss: 0.5554 Acc: 0.9059 | LR: 6.62e-05
Epoch 86/200 | Train Loss: 0.4443 Acc: 0.9950 | Val Loss: 0.5608 Acc: 0.9059 | LR: 6.55e-05
Epoch 87/200 | Train Loss: 0.4438 Acc: 0.9975 | Val Loss: 0.5661 Acc: 0.9059 | LR: 6.47e-05
Epoch 88/200 | Train Loss: 0.4417 Acc: 1.0000 | Val Loss: 0.5629 Acc: 0.8941 | LR: 6.39e-05
Epoch 89/200 | Train Loss: 0.4420 Acc: 0.9925 | Val Loss: 0.5600 Acc: 0.9059 | LR: 6.31e-05
Epoch 90/200 | Train Loss: 0.4438 Acc: 1.0000 | Val Loss: 0.5600 Acc: 0.9059 | LR: 6.23e-05
Epoch 91/200 | Train Loss: 0.4381 Acc: 0.9975 | Val Loss: 0.5603 Acc: 0.9059 | LR: 6.15e-05
Epoch 92/200 | Train Loss: 0.4406 Acc: 0.9975 | Val Loss: 0.5636 Acc: 0.9059 | LR: 6.07e-05
Epoch 93/200 | Train Loss: 0.4411 Acc: 1.0000 | Val Loss: 0.5653 Acc: 0.9059 | LR: 5.99e-05
Epoch 94/200 | Train Loss: 0.4490 Acc: 0.9925 | Val Loss: 0.5659 Acc: 0.9059 | LR: 5.90e-05
Epoch 95/200 | Train Loss: 0.4358 Acc: 0.9975 | Val Loss: 0.5632 Acc: 0.8941 | LR: 5.82e-05
Epoch 96/200 | Train Loss: 0.4361 Acc: 1.0000 | Val Loss: 0.5609 Acc: 0.8941 | LR: 5.74e-05
Epoch 97/200 | Train Loss: 0.4504 Acc: 0.9950 | Val Loss: 0.5611 Acc: 0.9059 | LR: 5.66e-05
Early stopping triggered.
Training complete in 7.6 minutes. Best Val Loss: 0.5525
Best model for fold saved to: new_results_Multimodal_DINO_Concat_KFold\best_model_fold_1.pth



Saved training history plot to: new_results_Multimodal_DINO_Concat_KFold\training_history_fold_1.png

=====

OVERALL PERFORMANCE (FOLD 1)

=====

	precision	recall	f1-score	support
Herpes	1.00	0.71	0.83	7
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.86	1.00	0.92	18
Sarampion	0.90	0.90	0.90	29
Varicela	0.96	0.92	0.94	24
accuracy			0.91	81
macro avg	0.94	0.91	0.92	81
weighted avg	0.92	0.91	0.91	81

--- Overall Scalar Metrics ---

Accuracy: 0.9136

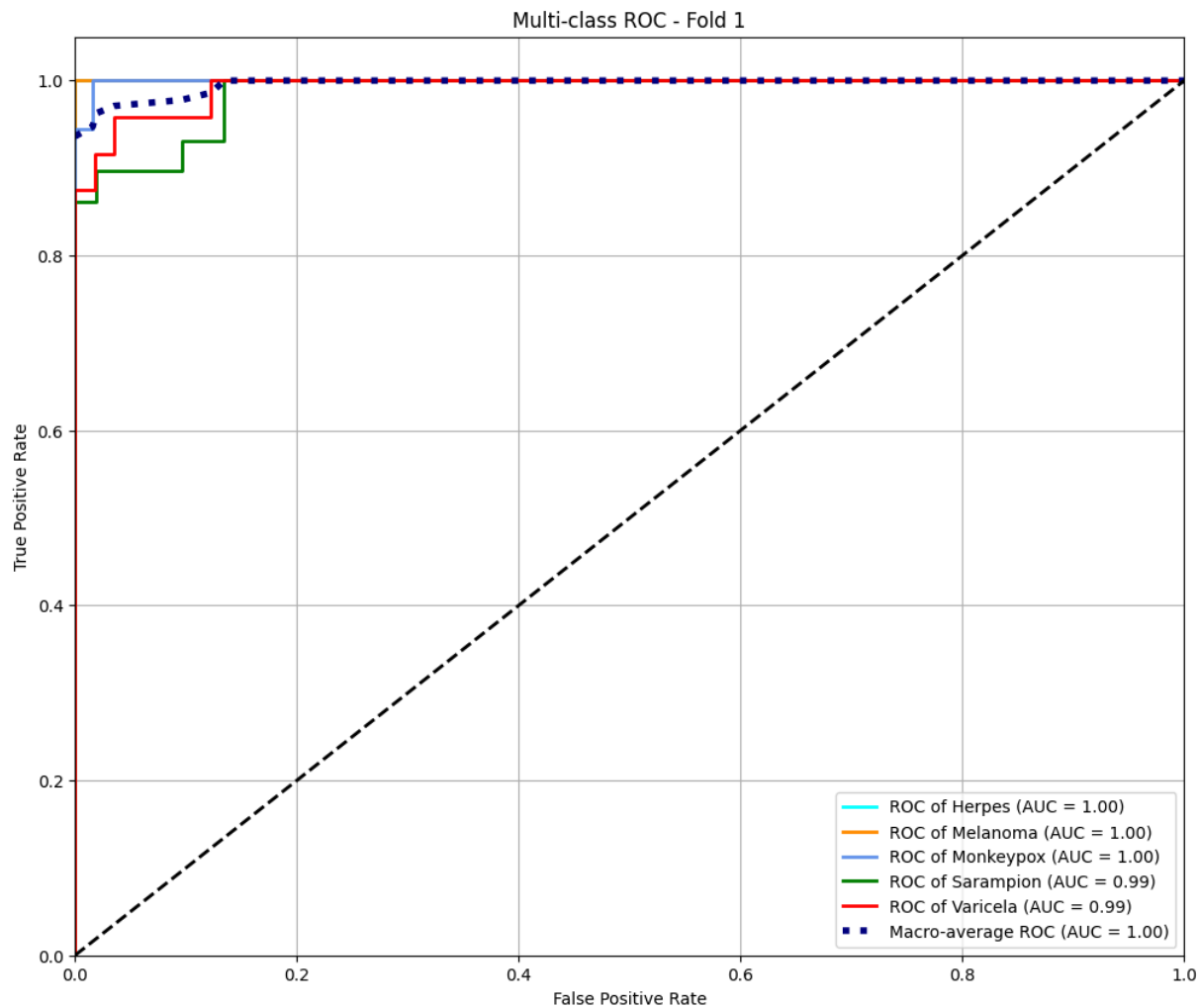
Macro ROC AUC: 0.9957

Cohen's Kappa: 0.8805

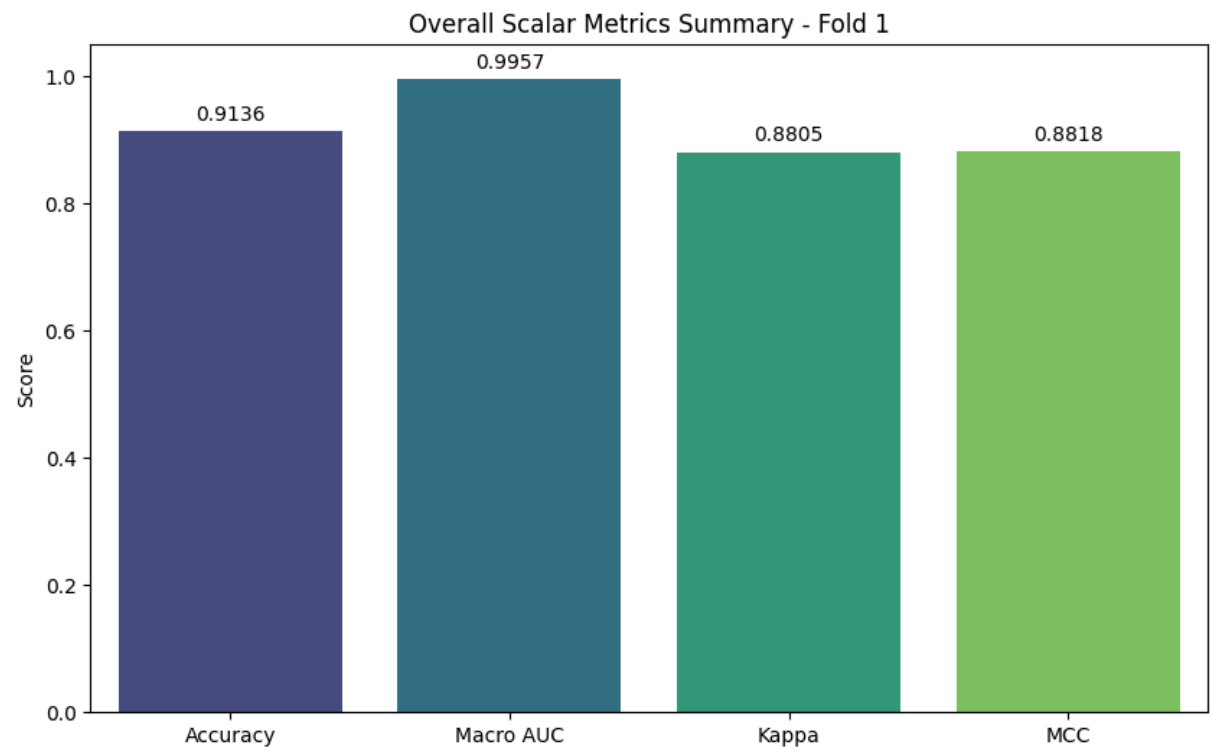
Matthews CorrCoef: 0.8818



Saved normalized confusion matrix to: new_results_Multimodal_DINO_Concat_KFold\confu
sion_matrix_fold_1.png



Saved ROC curves plot to: new_results_Multimodal_DINO_Concat_KFold\roc_curves_fold_1.png



Saved scalar metrics plot to: new_results_Multimodal_DINO_Concat_KFold\scalar_metrics_fold_1.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	1.00	0.33	0.50	3
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.85	0.92	0.88	12
Varicela	0.91	1.00	0.95	10
accuracy			0.89	27
macro avg	0.75	0.65	0.67	27
weighted avg	0.90	0.89	0.87	27

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	4
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.84	1.00	0.91	16
Sarampion	0.94	0.88	0.91	17
Varicela	1.00	0.86	0.92	14
accuracy			0.93	54
macro avg	0.96	0.95	0.95	54
weighted avg	0.93	0.93	0.93	54

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.80	1.00	0.89	4
Varicela	1.00	1.00	1.00	4
accuracy			0.91	11
macro avg	0.56	0.60	0.58	11
weighted avg	0.84	0.91	0.87	11

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	1.00	0.83	0.91	6
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.84	1.00	0.91	16
Sarampion	0.92	0.88	0.90	25
Varicela	0.95	0.90	0.92	20
accuracy			0.91	70
macro avg	0.94	0.92	0.93	70
weighted avg	0.92	0.91	0.91	70

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	0.50	1.00	0.67	1
Varicela	1.00	1.00	1.00	4
accuracy			0.89	9
macro avg	0.50	0.60	0.53	9
weighted avg	0.83	0.89	0.85	9

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	7
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	9
macro avg	0.60	0.60	0.60	9
weighted avg	1.00	1.00	1.00	9

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	4
Varicela	0.00	0.00	0.00	0
accuracy			1.00	8

macro avg	0.60	0.60	0.60	8
weighted avg	1.00	1.00	1.00	8

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	1.00	1.00	1.00	4
Varicela	0.00	0.00	0.00	0
accuracy			1.00	5
macro avg	0.40	0.40	0.40	5
weighted avg	1.00	1.00	1.00	5

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	1.00	0.75	0.86	4
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.50	1.00	0.67	3
Sarampion	0.89	0.84	0.86	19
Varicela	0.95	0.90	0.92	20
accuracy			0.88	49
macro avg	0.87	0.90	0.86	49
weighted avg	0.90	0.88	0.88	49

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.0000 EOD: 0.8600
Worst pair: 0-17|female|asian vs 18-59|male|white
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.8261 EOD: 0.7218
Worst pair: 18-59|female|white vs 18-59|male|white
Groups: ['18-59|female|white', '18-59|male|white']
=====
```

```
#####  
####  
### FOLD 2 ###  
#####  
####  
Trainable params: 0.56M
```

START TRAINING (DINOv3 + Concat) for 200 epochs...

Epoch 1/200		Train Loss: 1.7216	Acc: 0.2105		Val Loss: 1.5922	Acc: 0.3294		LR: 2.00e-05
Epoch 2/200		Train Loss: 1.6793	Acc: 0.2506		Val Loss: 1.5733	Acc: 0.3412		LR: 3.00e-05
Epoch 3/200		Train Loss: 1.5418	Acc: 0.3333		Val Loss: 1.5346	Acc: 0.4235		LR: 4.00e-05
Epoch 4/200		Train Loss: 1.4800	Acc: 0.4010		Val Loss: 1.4712	Acc: 0.5765		LR: 5.00e-05
Epoch 5/200		Train Loss: 1.3583	Acc: 0.5414		Val Loss: 1.3825	Acc: 0.6353		LR: 6.00e-05
Epoch 6/200		Train Loss: 1.2137	Acc: 0.6316		Val Loss: 1.2784	Acc: 0.6824		LR: 7.00e-05
Epoch 7/200		Train Loss: 1.1009	Acc: 0.7193		Val Loss: 1.1694	Acc: 0.6941		LR: 8.00e-05
Epoch 8/200		Train Loss: 0.9995	Acc: 0.7744		Val Loss: 1.0697	Acc: 0.7294		LR: 9.00e-05
Epoch 9/200		Train Loss: 0.9160	Acc: 0.7945		Val Loss: 0.9934	Acc: 0.7765		LR: 1.00e-04
Epoch 10/200		Train Loss: 0.8570	Acc: 0.8045		Val Loss: 0.9335	Acc: 0.7765		LR: 1.00e-04
Epoch 11/200		Train Loss: 0.8007	Acc: 0.8446		Val Loss: 0.8878	Acc: 0.8000		LR: 1.00e-04
Epoch 12/200		Train Loss: 0.7389	Acc: 0.8571		Val Loss: 0.8447	Acc: 0.8000		LR: 1.00e-04
Epoch 13/200		Train Loss: 0.7248	Acc: 0.8722		Val Loss: 0.8114	Acc: 0.8000		LR: 9.99e-05
Epoch 14/200		Train Loss: 0.6826	Acc: 0.8697		Val Loss: 0.7837	Acc: 0.8588		LR: 9.99e-05
Epoch 15/200		Train Loss: 0.6472	Acc: 0.9198		Val Loss: 0.7676	Acc: 0.8588		LR: 9.98e-05
Epoch 16/200		Train Loss: 0.6315	Acc: 0.9123		Val Loss: 0.7499	Acc: 0.8588		LR: 9.98e-05
Epoch 17/200		Train Loss: 0.6196	Acc: 0.9323		Val Loss: 0.7346	Acc: 0.8588		LR: 9.97e-05
Epoch 18/200		Train Loss: 0.5991	Acc: 0.9449		Val Loss: 0.7210	Acc: 0.8706		LR: 9.96e-05
Epoch 19/200		Train Loss: 0.5938	Acc: 0.9273		Val Loss: 0.7100	Acc: 0.8706		LR: 9.94e-05
Epoch 20/200		Train Loss: 0.5673	Acc: 0.9474		Val Loss: 0.7001	Acc: 0.8588		LR: 9.93e-05
Epoch 21/200		Train Loss: 0.5501	Acc: 0.9524		Val Loss: 0.6934	Acc: 0.8588		LR: 9.92e-05
Epoch 22/200		Train Loss: 0.5441	Acc: 0.9524		Val Loss: 0.6872	Acc: 0.8588		LR: 9.90e-05
Epoch 23/200		Train Loss: 0.5509	Acc: 0.9599		Val Loss: 0.6838	Acc: 0.8588		LR: 9.88e-05
Epoch 24/200		Train Loss: 0.5426	Acc: 0.9549		Val Loss: 0.6798	Acc: 0.8588		LR:

9.87e-05
Epoch 25/200 | Train Loss: 0.5409 Acc: 0.9599 | Val Loss: 0.6716 Acc: 0.8588 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.5436 Acc: 0.9474 | Val Loss: 0.6673 Acc: 0.8588 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.5123 Acc: 0.9699 | Val Loss: 0.6640 Acc: 0.8588 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.5130 Acc: 0.9699 | Val Loss: 0.6573 Acc: 0.8588 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.5140 Acc: 0.9699 | Val Loss: 0.6501 Acc: 0.8471 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.5091 Acc: 0.9699 | Val Loss: 0.6464 Acc: 0.8588 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.4998 Acc: 0.9799 | Val Loss: 0.6429 Acc: 0.8588 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.4975 Acc: 0.9774 | Val Loss: 0.6452 Acc: 0.8471 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.5047 Acc: 0.9674 | Val Loss: 0.6461 Acc: 0.8588 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.4989 Acc: 0.9799 | Val Loss: 0.6463 Acc: 0.8471 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.4880 Acc: 0.9850 | Val Loss: 0.6468 Acc: 0.8588 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.4914 Acc: 0.9799 | Val Loss: 0.6466 Acc: 0.8588 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.4968 Acc: 0.9699 | Val Loss: 0.6412 Acc: 0.8588 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.5007 Acc: 0.9699 | Val Loss: 0.6396 Acc: 0.8588 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.4862 Acc: 0.9724 | Val Loss: 0.6419 Acc: 0.8824 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.4834 Acc: 0.9850 | Val Loss: 0.6368 Acc: 0.8706 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.4729 Acc: 0.9825 | Val Loss: 0.6298 Acc: 0.8588 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.4700 Acc: 0.9875 | Val Loss: 0.6269 Acc: 0.8588 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.4824 Acc: 0.9875 | Val Loss: 0.6293 Acc: 0.8824 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.4672 Acc: 0.9850 | Val Loss: 0.6274 Acc: 0.8824 | LR: 9.23e-05
Epoch 45/200 | Train Loss: 0.4820 Acc: 0.9799 | Val Loss: 0.6223 Acc: 0.8706 | LR: 9.19e-05
Epoch 46/200 | Train Loss: 0.4733 Acc: 0.9900 | Val Loss: 0.6209 Acc: 0.8824 | LR: 9.14e-05
Epoch 47/200 | Train Loss: 0.4798 Acc: 0.9850 | Val Loss: 0.6201 Acc: 0.8824 | LR: 9.09e-05
Epoch 48/200 | Train Loss: 0.4700 Acc: 0.9774 | Val Loss: 0.6177 Acc: 0.8824 | LR: 9.05e-05
Epoch 49/200 | Train Loss: 0.4612 Acc: 0.9925 | Val Loss: 0.6178 Acc: 0.8824 | LR: 9.00e-05
Epoch 50/200 | Train Loss: 0.4626 Acc: 0.9900 | Val Loss: 0.6143 Acc: 0.8824 | LR: 8.95e-05
Epoch 51/200 | Train Loss: 0.4654 Acc: 0.9900 | Val Loss: 0.6149 Acc: 0.8824 | LR: 8.89e-05
Epoch 52/200 | Train Loss: 0.4549 Acc: 0.9950 | Val Loss: 0.6084 Acc: 0.9059 | LR:

8.84e-05
Epoch 53/200 | Train Loss: 0.4460 Acc: 0.9925 | Val Loss: 0.6081 Acc: 0.9059 | LR: 8.79e-05
Epoch 54/200 | Train Loss: 0.4545 Acc: 0.9925 | Val Loss: 0.6067 Acc: 0.9059 | LR: 8.73e-05
Epoch 55/200 | Train Loss: 0.4576 Acc: 0.9975 | Val Loss: 0.6063 Acc: 0.9059 | LR: 8.68e-05
Epoch 56/200 | Train Loss: 0.4549 Acc: 0.9950 | Val Loss: 0.6039 Acc: 0.9059 | LR: 8.62e-05
Epoch 57/200 | Train Loss: 0.4553 Acc: 0.9950 | Val Loss: 0.6051 Acc: 0.8941 | LR: 8.56e-05
Epoch 58/200 | Train Loss: 0.4535 Acc: 0.9950 | Val Loss: 0.6058 Acc: 0.8941 | LR: 8.51e-05
Epoch 59/200 | Train Loss: 0.4582 Acc: 0.9925 | Val Loss: 0.6053 Acc: 0.8941 | LR: 8.45e-05
Epoch 60/200 | Train Loss: 0.4517 Acc: 0.9950 | Val Loss: 0.6069 Acc: 0.8941 | LR: 8.39e-05
Epoch 61/200 | Train Loss: 0.4556 Acc: 0.9950 | Val Loss: 0.6052 Acc: 0.9059 | LR: 8.33e-05
Epoch 62/200 | Train Loss: 0.4505 Acc: 0.9950 | Val Loss: 0.6046 Acc: 0.9059 | LR: 8.26e-05
Epoch 63/200 | Train Loss: 0.4454 Acc: 1.0000 | Val Loss: 0.6021 Acc: 0.9059 | LR: 8.20e-05
Epoch 64/200 | Train Loss: 0.4514 Acc: 0.9975 | Val Loss: 0.6026 Acc: 0.9059 | LR: 8.14e-05
Epoch 65/200 | Train Loss: 0.4472 Acc: 0.9975 | Val Loss: 0.6008 Acc: 0.8941 | LR: 8.07e-05
Epoch 66/200 | Train Loss: 0.4505 Acc: 0.9925 | Val Loss: 0.6004 Acc: 0.8941 | LR: 8.01e-05
Epoch 67/200 | Train Loss: 0.4506 Acc: 0.9925 | Val Loss: 0.6017 Acc: 0.8941 | LR: 7.94e-05
Epoch 68/200 | Train Loss: 0.4459 Acc: 0.9950 | Val Loss: 0.6023 Acc: 0.8941 | LR: 7.87e-05
Epoch 69/200 | Train Loss: 0.4474 Acc: 0.9950 | Val Loss: 0.6020 Acc: 0.8941 | LR: 7.80e-05
Epoch 70/200 | Train Loss: 0.4460 Acc: 0.9950 | Val Loss: 0.6016 Acc: 0.8941 | LR: 7.73e-05
Epoch 71/200 | Train Loss: 0.4457 Acc: 0.9950 | Val Loss: 0.5992 Acc: 0.9059 | LR: 7.67e-05
Epoch 72/200 | Train Loss: 0.4448 Acc: 0.9975 | Val Loss: 0.5989 Acc: 0.9059 | LR: 7.59e-05
Epoch 73/200 | Train Loss: 0.4420 Acc: 0.9950 | Val Loss: 0.5974 Acc: 0.9059 | LR: 7.52e-05
Epoch 74/200 | Train Loss: 0.4398 Acc: 0.9975 | Val Loss: 0.5951 Acc: 0.8941 | LR: 7.45e-05
Epoch 75/200 | Train Loss: 0.4433 Acc: 0.9925 | Val Loss: 0.5926 Acc: 0.9059 | LR: 7.38e-05
Epoch 76/200 | Train Loss: 0.4472 Acc: 0.9975 | Val Loss: 0.5960 Acc: 0.9059 | LR: 7.31e-05
Epoch 77/200 | Train Loss: 0.4439 Acc: 0.9950 | Val Loss: 0.5931 Acc: 0.9059 | LR: 7.23e-05
Epoch 78/200 | Train Loss: 0.4405 Acc: 0.9975 | Val Loss: 0.5952 Acc: 0.9059 | LR: 7.16e-05
Epoch 79/200 | Train Loss: 0.4404 Acc: 0.9975 | Val Loss: 0.5944 Acc: 0.9059 | LR: 7.08e-05
Epoch 80/200 | Train Loss: 0.4489 Acc: 0.9950 | Val Loss: 0.5960 Acc: 0.8941 | LR:

7.01e-05
Epoch 81/200 | Train Loss: 0.4477 Acc: 0.9925 | Val Loss: 0.5938 Acc: 0.8941 | LR: 6.93e-05
Epoch 82/200 | Train Loss: 0.4382 Acc: 1.0000 | Val Loss: 0.5917 Acc: 0.8941 | LR: 6.86e-05
Epoch 83/200 | Train Loss: 0.4449 Acc: 0.9975 | Val Loss: 0.5900 Acc: 0.8941 | LR: 6.78e-05
Epoch 84/200 | Train Loss: 0.4387 Acc: 0.9975 | Val Loss: 0.5920 Acc: 0.8941 | LR: 6.70e-05
Epoch 85/200 | Train Loss: 0.4401 Acc: 0.9975 | Val Loss: 0.5887 Acc: 0.9176 | LR: 6.62e-05
Epoch 86/200 | Train Loss: 0.4446 Acc: 0.9925 | Val Loss: 0.5882 Acc: 0.9176 | LR: 6.55e-05
Epoch 87/200 | Train Loss: 0.4428 Acc: 0.9975 | Val Loss: 0.5877 Acc: 0.9176 | LR: 6.47e-05
Epoch 88/200 | Train Loss: 0.4437 Acc: 0.9950 | Val Loss: 0.5928 Acc: 0.8941 | LR: 6.39e-05
Epoch 89/200 | Train Loss: 0.4433 Acc: 1.0000 | Val Loss: 0.5928 Acc: 0.8941 | LR: 6.31e-05
Epoch 90/200 | Train Loss: 0.4495 Acc: 0.9925 | Val Loss: 0.5969 Acc: 0.8941 | LR: 6.23e-05
Epoch 91/200 | Train Loss: 0.4359 Acc: 0.9975 | Val Loss: 0.5957 Acc: 0.8941 | LR: 6.15e-05
Epoch 92/200 | Train Loss: 0.4411 Acc: 0.9950 | Val Loss: 0.5914 Acc: 0.8941 | LR: 6.07e-05
Epoch 93/200 | Train Loss: 0.4377 Acc: 0.9975 | Val Loss: 0.5905 Acc: 0.8941 | LR: 5.99e-05
Epoch 94/200 | Train Loss: 0.4416 Acc: 0.9975 | Val Loss: 0.5939 Acc: 0.8941 | LR: 5.90e-05
Epoch 95/200 | Train Loss: 0.4354 Acc: 0.9975 | Val Loss: 0.5908 Acc: 0.8941 | LR: 5.82e-05
Epoch 96/200 | Train Loss: 0.4412 Acc: 1.0000 | Val Loss: 0.5904 Acc: 0.8941 | LR: 5.74e-05
Epoch 97/200 | Train Loss: 0.4462 Acc: 0.9975 | Val Loss: 0.5878 Acc: 0.8941 | LR: 5.66e-05
Epoch 98/200 | Train Loss: 0.4396 Acc: 0.9950 | Val Loss: 0.5878 Acc: 0.8941 | LR: 5.58e-05
Epoch 99/200 | Train Loss: 0.4373 Acc: 1.0000 | Val Loss: 0.5876 Acc: 0.8941 | LR: 5.50e-05
Epoch 100/200 | Train Loss: 0.4373 Acc: 0.9975 | Val Loss: 0.5879 Acc: 0.8941 | LR: 5.41e-05
Epoch 101/200 | Train Loss: 0.4381 Acc: 1.0000 | Val Loss: 0.5863 Acc: 0.8941 | LR: 5.33e-05
Epoch 102/200 | Train Loss: 0.4364 Acc: 1.0000 | Val Loss: 0.5871 Acc: 0.8941 | LR: 5.25e-05
Epoch 103/200 | Train Loss: 0.4387 Acc: 0.9975 | Val Loss: 0.5833 Acc: 0.8941 | LR: 5.17e-05
Epoch 104/200 | Train Loss: 0.4415 Acc: 0.9975 | Val Loss: 0.5818 Acc: 0.8941 | LR: 5.08e-05
Epoch 105/200 | Train Loss: 0.4339 Acc: 0.9975 | Val Loss: 0.5839 Acc: 0.8941 | LR: 5.00e-05
Epoch 106/200 | Train Loss: 0.4420 Acc: 0.9975 | Val Loss: 0.5803 Acc: 0.9059 | LR: 4.92e-05
Epoch 107/200 | Train Loss: 0.4399 Acc: 1.0000 | Val Loss: 0.5824 Acc: 0.8941 | LR: 4.83e-05
Epoch 108/200 | Train Loss: 0.4378 Acc: 0.9975 | Val Loss: 0.5839 Acc: 0.8941 | LR:

4.75e-05

Epoch 109/200 | Train Loss: 0.4340 Acc: 0.9975 | Val Loss: 0.5812 Acc: 0.8941 | LR:

4.67e-05

Epoch 110/200 | Train Loss: 0.4433 Acc: 0.9900 | Val Loss: 0.5806 Acc: 0.8941 | LR:

4.59e-05

Epoch 111/200 | Train Loss: 0.4362 Acc: 1.0000 | Val Loss: 0.5826 Acc: 0.8941 | LR:

4.50e-05

Epoch 112/200 | Train Loss: 0.4394 Acc: 0.9950 | Val Loss: 0.5857 Acc: 0.8941 | LR:

4.42e-05

Epoch 113/200 | Train Loss: 0.4392 Acc: 0.9900 | Val Loss: 0.5841 Acc: 0.8941 | LR:

4.34e-05

Epoch 114/200 | Train Loss: 0.4373 Acc: 0.9975 | Val Loss: 0.5823 Acc: 0.8941 | LR:

4.26e-05

Epoch 115/200 | Train Loss: 0.4379 Acc: 0.9975 | Val Loss: 0.5825 Acc: 0.8941 | LR:

4.18e-05

Epoch 116/200 | Train Loss: 0.4355 Acc: 1.0000 | Val Loss: 0.5878 Acc: 0.8941 | LR:

4.10e-05

Epoch 117/200 | Train Loss: 0.4380 Acc: 1.0000 | Val Loss: 0.5848 Acc: 0.8941 | LR:

4.01e-05

Epoch 118/200 | Train Loss: 0.4314 Acc: 1.0000 | Val Loss: 0.5841 Acc: 0.8941 | LR:

3.93e-05

Epoch 119/200 | Train Loss: 0.4332 Acc: 0.9975 | Val Loss: 0.5872 Acc: 0.9059 | LR:

3.85e-05

Epoch 120/200 | Train Loss: 0.4404 Acc: 0.9925 | Val Loss: 0.5855 Acc: 0.8941 | LR:

3.77e-05

Epoch 121/200 | Train Loss: 0.4338 Acc: 1.0000 | Val Loss: 0.5867 Acc: 0.9059 | LR:

3.69e-05

Epoch 122/200 | Train Loss: 0.4355 Acc: 0.9975 | Val Loss: 0.5842 Acc: 0.9059 | LR:

3.61e-05

Epoch 123/200 | Train Loss: 0.4366 Acc: 0.9975 | Val Loss: 0.5866 Acc: 0.8941 | LR:

3.53e-05

Epoch 124/200 | Train Loss: 0.4312 Acc: 1.0000 | Val Loss: 0.5891 Acc: 0.9059 | LR:

3.45e-05

Epoch 125/200 | Train Loss: 0.4339 Acc: 0.9950 | Val Loss: 0.5881 Acc: 0.9294 | LR:

3.38e-05

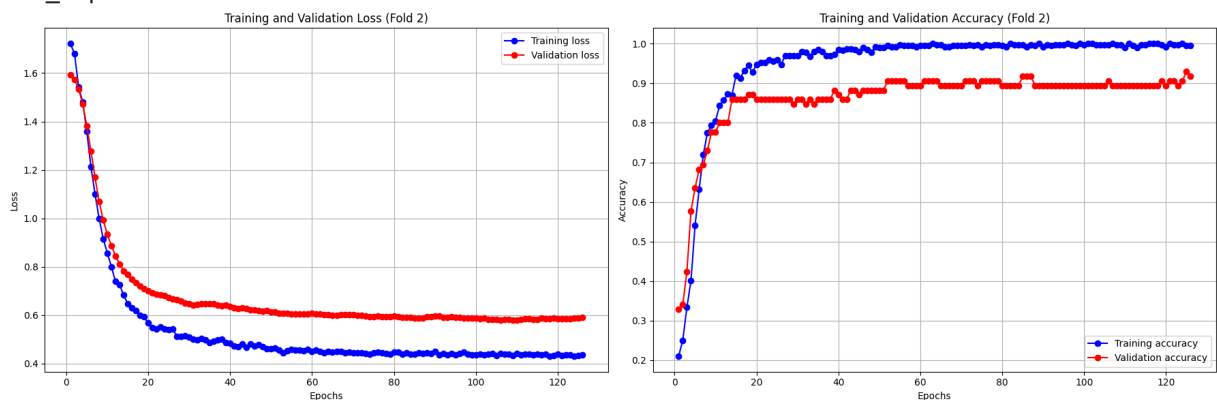
Epoch 126/200 | Train Loss: 0.4364 Acc: 0.9950 | Val Loss: 0.5899 Acc: 0.9176 | LR:

3.30e-05

Early stopping triggered.

Training complete in 10.0 minutes. Best Val Loss: 0.5803

Best model for fold saved to: new_results_Multimodal_DINO_Concat_KFold\best_model_fold_2.pth



Saved training history plot to: new_results_Multimodal_DINO_Concat_KFold\training_history_fold_2.png

=====

OVERALL PERFORMANCE (FOLD 2)

=====

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	7
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.94	0.89	0.91	18
Sarampion	0.90	0.93	0.92	29
Varicela	0.92	0.92	0.92	24
accuracy			0.93	81
macro avg	0.95	0.95	0.95	81
weighted avg	0.93	0.93	0.93	81

--- Overall Scalar Metrics ---

Accuracy: 0.9259

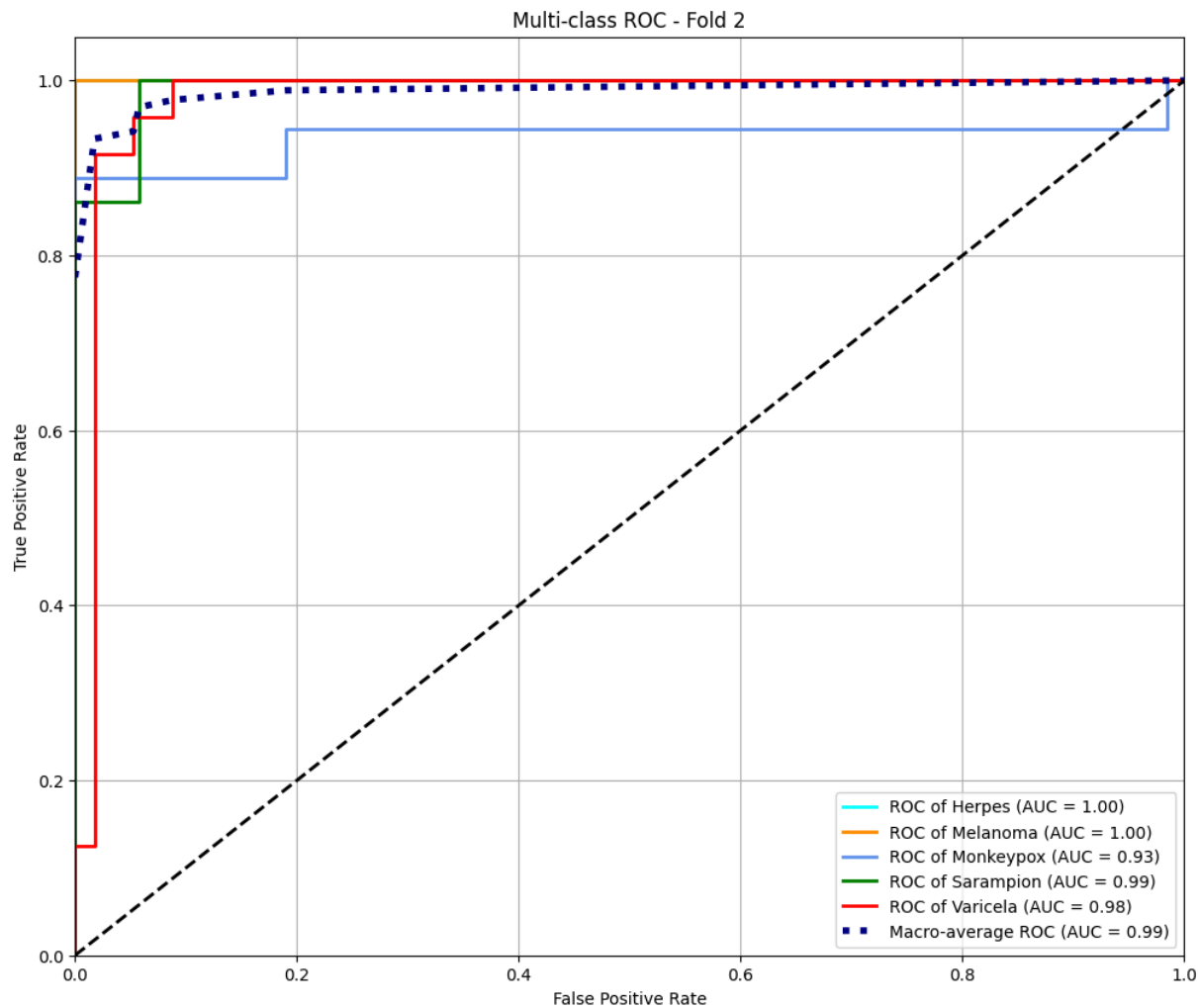
Macro ROC AUC: 0.9814

Cohen's Kappa: 0.8977

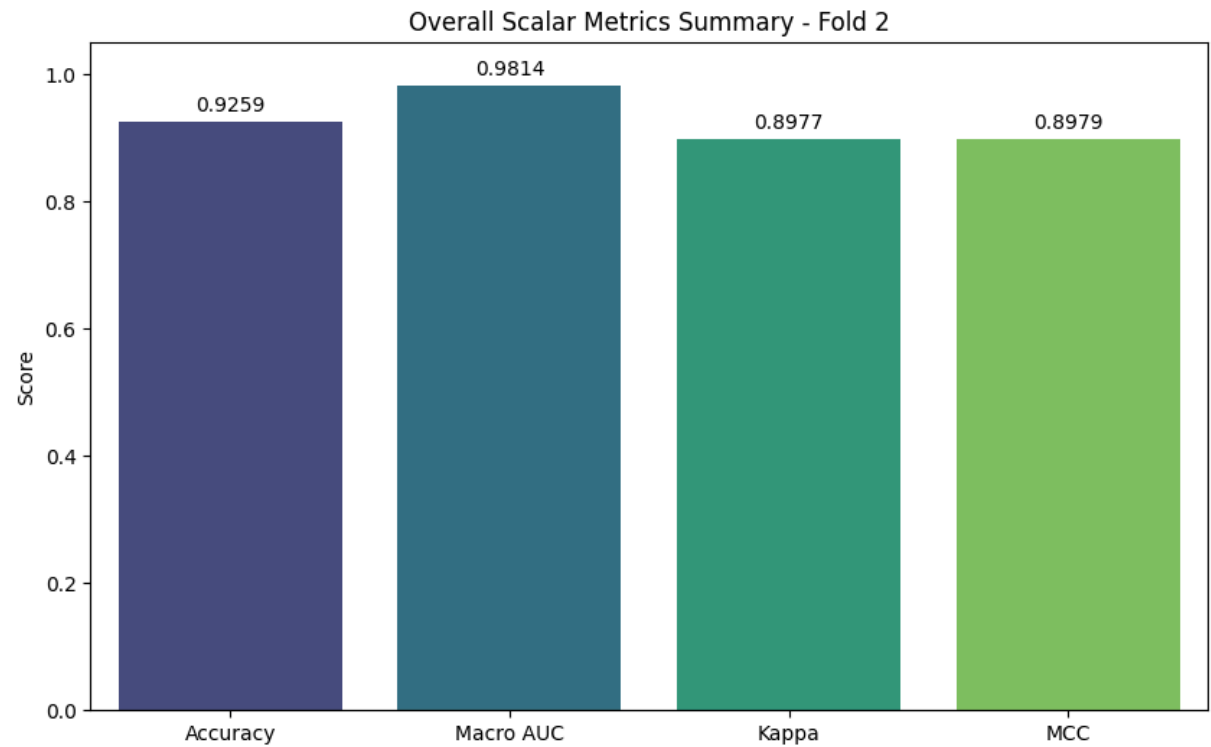
Matthews CorrCoef: 0.8979



Saved normalized confusion matrix to: new_results_Multimodal_DINO_Concat_KFold\confu
sion_matrix_fold_2.png



Saved ROC curves plot to: new_results_Multimodal_DINO_Concat_KFold\roc_curves_fold_2.png



Saved scalar metrics plot to: new_results_Multimodal_DINO_Concat_KFold\scalar_metrics_fold_2.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	2
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.50	0.67	4
Sarampion	0.87	0.93	0.90	14
Varicela	0.86	0.92	0.89	13
accuracy			0.88	33
macro avg	0.74	0.67	0.69	33
weighted avg	0.89	0.88	0.87	33

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	5
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.93	1.00	0.97	14
Sarampion	0.93	0.93	0.93	15
Varicela	1.00	0.91	0.95	11
accuracy			0.96	48
macro avg	0.97	0.97	0.97	48
weighted avg	0.96	0.96	0.96	48

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	1
Sarampion	1.00	1.00	1.00	3
Varicela	0.80	1.00	0.89	4
accuracy			0.88	8
macro avg	0.36	0.40	0.38	8
weighted avg	0.78	0.88	0.82	8

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	6
Melanoma	1.00	1.00	1.00	3
Monkeypox	0.94	0.94	0.94	17
Sarampion	0.89	0.92	0.91	26
Varicela	0.95	0.90	0.92	20
accuracy			0.93	72
macro avg	0.96	0.95	0.95	72
weighted avg	0.93	0.93	0.93	72

--- Subgroup: 60+ ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	3
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	1.00	1.00	2
Varicela	1.00	1.00	1.00	2
accuracy			1.00	11
macro avg	0.80	0.80	0.80	11
weighted avg	1.00	1.00	1.00	11

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	2

macro avg	0.20	0.20	0.20	2
weighted avg	1.00	1.00	1.00	2

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	2
Varicela	0.00	0.00	0.00	0
accuracy			1.00	4
macro avg	0.40	0.40	0.40	4
weighted avg	1.00	1.00	1.00	4

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.67	0.80	3
Sarampion	0.88	1.00	0.93	7
Varicela	0.00	0.00	0.00	0
accuracy			0.91	11
macro avg	0.57	0.53	0.55	11
weighted avg	0.92	0.91	0.90	11

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.75	1.00	0.86	3
Sarampion	0.00	0.00	0.00	1
Varicela	0.00	0.00	0.00	1
accuracy			0.60	5
macro avg	0.15	0.20	0.17	5
weighted avg	0.45	0.60	0.51	5

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	3
Melanoma	1.00	1.00	1.00	3
Monkeypox	1.00	0.75	0.86	4
Sarampion	0.94	0.94	0.94	17
Varicela	0.91	0.95	0.93	21
accuracy			0.94	48

macro avg	0.97	0.93	0.95	48
weighted avg	0.94	0.94	0.94	48

=====

FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.6000 EOD: 0.8000

Worst pair: 0-17|female|white vs 18-59|male|asian

=====

=====

FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.8889 EOD: 0.6444

Worst pair: 18-59|female|white vs 18-59|male|white

Groups: ['18-59|female|white', '18-59|male|white']

=====

#####

####

FOLD 3

#####

####

Trainable params: 0.56M

START TRAINING (DINOv3 + Concat) for 200 epochs...

Epoch 1/200	Train Loss: 1.7458	Acc: 0.1604	Val Loss: 1.6183	Acc: 0.1647	LR: 2.00e-05
Epoch 2/200	Train Loss: 1.6730	Acc: 0.2080	Val Loss: 1.5936	Acc: 0.2000	LR: 3.00e-05
Epoch 3/200	Train Loss: 1.5983	Acc: 0.2531	Val Loss: 1.5480	Acc: 0.3294	LR: 4.00e-05
Epoch 4/200	Train Loss: 1.4968	Acc: 0.3584	Val Loss: 1.4753	Acc: 0.5765	LR: 5.00e-05
Epoch 5/200	Train Loss: 1.3655	Acc: 0.5038	Val Loss: 1.3731	Acc: 0.7176	LR: 6.00e-05
Epoch 6/200	Train Loss: 1.2819	Acc: 0.5865	Val Loss: 1.2533	Acc: 0.7529	LR: 7.00e-05
Epoch 7/200	Train Loss: 1.1660	Acc: 0.6917	Val Loss: 1.1280	Acc: 0.8235	LR: 8.00e-05
Epoch 8/200	Train Loss: 1.0453	Acc: 0.7469	Val Loss: 1.0152	Acc: 0.8118	LR: 9.00e-05
Epoch 9/200	Train Loss: 0.9832	Acc: 0.7619	Val Loss: 0.9192	Acc: 0.8235	LR: 1.00e-04
Epoch 10/200	Train Loss: 0.9179	Acc: 0.7895	Val Loss: 0.8523	Acc: 0.8118	LR: 1.00e-04
Epoch 11/200	Train Loss: 0.8430	Acc: 0.8371	Val Loss: 0.7899	Acc: 0.8471	LR: 1.00e-04
Epoch 12/200	Train Loss: 0.7879	Acc: 0.8546	Val Loss: 0.7464	Acc: 0.8706	LR: 1.00e-04
Epoch 13/200	Train Loss: 0.7486	Acc: 0.8722	Val Loss: 0.7135	Acc: 0.8824	LR: 9.99e-05
Epoch 14/200	Train Loss: 0.7147	Acc: 0.8897	Val Loss: 0.6787	Acc: 0.8824	LR: 9.99e-05
Epoch 15/200	Train Loss: 0.6861	Acc: 0.8922	Val Loss: 0.6578	Acc: 0.8824	LR: 9.98e-05
Epoch 16/200	Train Loss: 0.6580	Acc: 0.9148	Val Loss: 0.6415	Acc: 0.8941	LR: 9.98e-05
Epoch 17/200	Train Loss: 0.6457	Acc: 0.9223	Val Loss: 0.6318	Acc: 0.9294	LR:

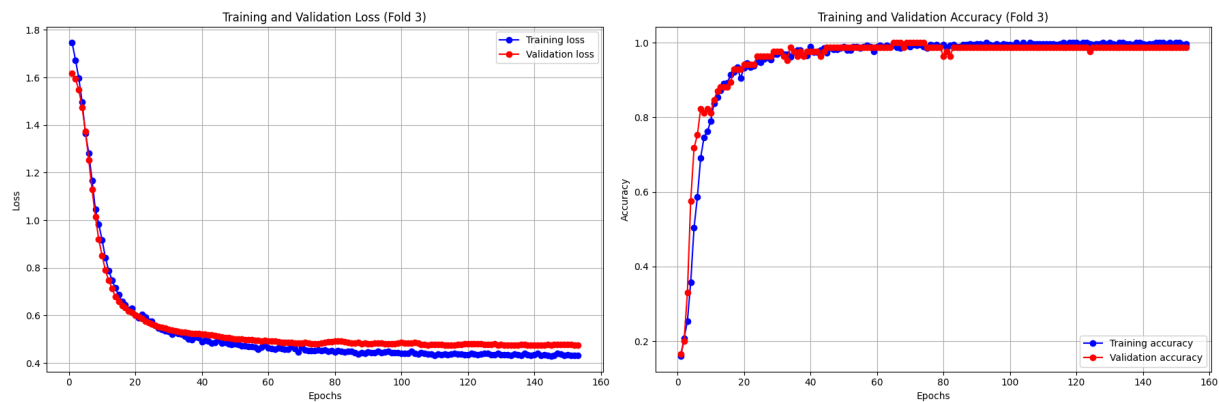
9.97e-05
Epoch 18/200 | Train Loss: 0.6240 Acc: 0.9348 | Val Loss: 0.6198 Acc: 0.9294 | LR: 9.96e-05
Epoch 19/200 | Train Loss: 0.6287 Acc: 0.9048 | Val Loss: 0.6117 Acc: 0.9294 | LR: 9.94e-05
Epoch 20/200 | Train Loss: 0.6016 Acc: 0.9323 | Val Loss: 0.6022 Acc: 0.9412 | LR: 9.93e-05
Epoch 21/200 | Train Loss: 0.5905 Acc: 0.9449 | Val Loss: 0.5955 Acc: 0.9412 | LR: 9.92e-05
Epoch 22/200 | Train Loss: 0.6029 Acc: 0.9348 | Val Loss: 0.5864 Acc: 0.9412 | LR: 9.90e-05
Epoch 23/200 | Train Loss: 0.5919 Acc: 0.9373 | Val Loss: 0.5749 Acc: 0.9412 | LR: 9.88e-05
Epoch 24/200 | Train Loss: 0.5763 Acc: 0.9499 | Val Loss: 0.5689 Acc: 0.9647 | LR: 9.87e-05
Epoch 25/200 | Train Loss: 0.5745 Acc: 0.9474 | Val Loss: 0.5630 Acc: 0.9647 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.5587 Acc: 0.9574 | Val Loss: 0.5578 Acc: 0.9647 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.5474 Acc: 0.9599 | Val Loss: 0.5538 Acc: 0.9647 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.5401 Acc: 0.9549 | Val Loss: 0.5501 Acc: 0.9647 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.5340 Acc: 0.9699 | Val Loss: 0.5463 Acc: 0.9765 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.5316 Acc: 0.9699 | Val Loss: 0.5407 Acc: 0.9765 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.5220 Acc: 0.9749 | Val Loss: 0.5385 Acc: 0.9765 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.5289 Acc: 0.9699 | Val Loss: 0.5354 Acc: 0.9647 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.5231 Acc: 0.9699 | Val Loss: 0.5338 Acc: 0.9529 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.5200 Acc: 0.9624 | Val Loss: 0.5301 Acc: 0.9882 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.5121 Acc: 0.9749 | Val Loss: 0.5283 Acc: 0.9765 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.5020 Acc: 0.9799 | Val Loss: 0.5263 Acc: 0.9647 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.4968 Acc: 0.9799 | Val Loss: 0.5243 Acc: 0.9765 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.5106 Acc: 0.9649 | Val Loss: 0.5251 Acc: 0.9647 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.5066 Acc: 0.9649 | Val Loss: 0.5236 Acc: 0.9765 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.4885 Acc: 0.9900 | Val Loss: 0.5222 Acc: 0.9765 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.4958 Acc: 0.9749 | Val Loss: 0.5210 Acc: 0.9765 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.4930 Acc: 0.9749 | Val Loss: 0.5194 Acc: 0.9765 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.4835 Acc: 0.9799 | Val Loss: 0.5170 Acc: 0.9647 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.4865 Acc: 0.9850 | Val Loss: 0.5150 Acc: 0.9765 | LR: 9.23e-05
Epoch 45/200 | Train Loss: 0.4975 Acc: 0.9724 | Val Loss: 0.5123 Acc: 0.9882 | LR:

9.19e-05
Epoch 46/200 | Train Loss: 0.4833 Acc: 0.9850 | Val Loss: 0.5098 Acc: 0.9882 | LR: 9.14e-05
Epoch 47/200 | Train Loss: 0.4877 Acc: 0.9825 | Val Loss: 0.5073 Acc: 0.9882 | LR: 9.09e-05
Epoch 48/200 | Train Loss: 0.4818 Acc: 0.9825 | Val Loss: 0.5063 Acc: 0.9882 | LR: 9.05e-05
Epoch 49/200 | Train Loss: 0.4769 Acc: 0.9850 | Val Loss: 0.5038 Acc: 0.9882 | LR: 9.00e-05
Epoch 50/200 | Train Loss: 0.4804 Acc: 0.9900 | Val Loss: 0.5018 Acc: 0.9882 | LR: 8.95e-05
Epoch 51/200 | Train Loss: 0.4791 Acc: 0.9799 | Val Loss: 0.5013 Acc: 0.9882 | LR: 8.89e-05
Epoch 52/200 | Train Loss: 0.4715 Acc: 0.9799 | Val Loss: 0.5008 Acc: 0.9882 | LR: 8.84e-05
Epoch 53/200 | Train Loss: 0.4728 Acc: 0.9900 | Val Loss: 0.4984 Acc: 0.9882 | LR: 8.79e-05
Epoch 54/200 | Train Loss: 0.4687 Acc: 0.9900 | Val Loss: 0.4982 Acc: 0.9882 | LR: 8.73e-05
Epoch 55/200 | Train Loss: 0.4694 Acc: 0.9850 | Val Loss: 0.4983 Acc: 0.9882 | LR: 8.68e-05
Epoch 56/200 | Train Loss: 0.4671 Acc: 0.9900 | Val Loss: 0.4951 Acc: 0.9882 | LR: 8.62e-05
Epoch 57/200 | Train Loss: 0.4581 Acc: 0.9925 | Val Loss: 0.4941 Acc: 0.9882 | LR: 8.56e-05
Epoch 58/200 | Train Loss: 0.4653 Acc: 0.9900 | Val Loss: 0.4935 Acc: 0.9882 | LR: 8.51e-05
Epoch 59/200 | Train Loss: 0.4743 Acc: 0.9774 | Val Loss: 0.4942 Acc: 0.9882 | LR: 8.45e-05
Epoch 60/200 | Train Loss: 0.4636 Acc: 0.9900 | Val Loss: 0.4914 Acc: 0.9882 | LR: 8.39e-05
Epoch 61/200 | Train Loss: 0.4605 Acc: 0.9925 | Val Loss: 0.4915 Acc: 0.9882 | LR: 8.33e-05
Epoch 62/200 | Train Loss: 0.4565 Acc: 0.9900 | Val Loss: 0.4919 Acc: 0.9882 | LR: 8.26e-05
Epoch 63/200 | Train Loss: 0.4649 Acc: 0.9925 | Val Loss: 0.4890 Acc: 0.9882 | LR: 8.20e-05
Epoch 64/200 | Train Loss: 0.4602 Acc: 0.9900 | Val Loss: 0.4869 Acc: 0.9882 | LR: 8.14e-05
Epoch 65/200 | Train Loss: 0.4579 Acc: 0.9925 | Val Loss: 0.4872 Acc: 1.0000 | LR: 8.07e-05
Epoch 66/200 | Train Loss: 0.4579 Acc: 0.9875 | Val Loss: 0.4865 Acc: 1.0000 | LR: 8.01e-05
Epoch 67/200 | Train Loss: 0.4681 Acc: 0.9850 | Val Loss: 0.4857 Acc: 1.0000 | LR: 7.94e-05
Epoch 68/200 | Train Loss: 0.4610 Acc: 0.9925 | Val Loss: 0.4848 Acc: 0.9882 | LR: 7.87e-05
Epoch 69/200 | Train Loss: 0.4457 Acc: 0.9925 | Val Loss: 0.4837 Acc: 1.0000 | LR: 7.80e-05
Epoch 70/200 | Train Loss: 0.4659 Acc: 0.9900 | Val Loss: 0.4847 Acc: 1.0000 | LR: 7.73e-05
Epoch 71/200 | Train Loss: 0.4555 Acc: 0.9950 | Val Loss: 0.4861 Acc: 1.0000 | LR: 7.67e-05
Epoch 72/200 | Train Loss: 0.4517 Acc: 0.9925 | Val Loss: 0.4832 Acc: 1.0000 | LR: 7.59e-05
Epoch 73/200 | Train Loss: 0.4520 Acc: 0.9950 | Val Loss: 0.4813 Acc: 1.0000 | LR:

7.52e-05
Epoch 74/200 | Train Loss: 0.4518 Acc: 0.9900 | Val Loss: 0.4810 Acc: 1.0000 | LR: 7.45e-05
Epoch 75/200 | Train Loss: 0.4542 Acc: 0.9850 | Val Loss: 0.4812 Acc: 0.9882 | LR: 7.38e-05
Epoch 76/200 | Train Loss: 0.4516 Acc: 0.9950 | Val Loss: 0.4833 Acc: 0.9882 | LR: 7.31e-05
Epoch 77/200 | Train Loss: 0.4542 Acc: 0.9925 | Val Loss: 0.4872 Acc: 0.9882 | LR: 7.23e-05
Epoch 78/200 | Train Loss: 0.4490 Acc: 0.9950 | Val Loss: 0.4893 Acc: 0.9882 | LR: 7.16e-05
Epoch 79/200 | Train Loss: 0.4535 Acc: 0.9900 | Val Loss: 0.4896 Acc: 0.9882 | LR: 7.08e-05
Epoch 80/200 | Train Loss: 0.4466 Acc: 0.9950 | Val Loss: 0.4922 Acc: 0.9647 | LR: 7.01e-05
Epoch 81/200 | Train Loss: 0.4517 Acc: 0.9875 | Val Loss: 0.4912 Acc: 0.9765 | LR: 6.93e-05
Epoch 82/200 | Train Loss: 0.4487 Acc: 0.9925 | Val Loss: 0.4924 Acc: 0.9647 | LR: 6.86e-05
Epoch 83/200 | Train Loss: 0.4471 Acc: 0.9950 | Val Loss: 0.4881 Acc: 0.9882 | LR: 6.78e-05
Epoch 84/200 | Train Loss: 0.4496 Acc: 0.9900 | Val Loss: 0.4866 Acc: 0.9882 | LR: 6.70e-05
Epoch 85/200 | Train Loss: 0.4483 Acc: 0.9900 | Val Loss: 0.4844 Acc: 0.9882 | LR: 6.62e-05
Epoch 86/200 | Train Loss: 0.4435 Acc: 0.9950 | Val Loss: 0.4843 Acc: 0.9882 | LR: 6.55e-05
Epoch 87/200 | Train Loss: 0.4385 Acc: 0.9950 | Val Loss: 0.4861 Acc: 0.9882 | LR: 6.47e-05
Epoch 88/200 | Train Loss: 0.4427 Acc: 0.9975 | Val Loss: 0.4813 Acc: 0.9882 | LR: 6.39e-05
Epoch 89/200 | Train Loss: 0.4410 Acc: 0.9950 | Val Loss: 0.4826 Acc: 0.9882 | LR: 6.31e-05
Epoch 90/200 | Train Loss: 0.4460 Acc: 0.9975 | Val Loss: 0.4821 Acc: 0.9882 | LR: 6.23e-05
Epoch 91/200 | Train Loss: 0.4434 Acc: 0.9975 | Val Loss: 0.4809 Acc: 0.9882 | LR: 6.15e-05
Epoch 92/200 | Train Loss: 0.4438 Acc: 0.9950 | Val Loss: 0.4824 Acc: 0.9882 | LR: 6.07e-05
Epoch 93/200 | Train Loss: 0.4483 Acc: 1.0000 | Val Loss: 0.4840 Acc: 0.9882 | LR: 5.99e-05
Epoch 94/200 | Train Loss: 0.4440 Acc: 0.9950 | Val Loss: 0.4828 Acc: 0.9882 | LR: 5.90e-05
Epoch 95/200 | Train Loss: 0.4475 Acc: 0.9925 | Val Loss: 0.4808 Acc: 0.9882 | LR: 5.82e-05
Epoch 96/200 | Train Loss: 0.4481 Acc: 0.9975 | Val Loss: 0.4811 Acc: 0.9882 | LR: 5.74e-05
Epoch 97/200 | Train Loss: 0.4423 Acc: 0.9925 | Val Loss: 0.4800 Acc: 0.9882 | LR: 5.66e-05
Epoch 98/200 | Train Loss: 0.4421 Acc: 0.9975 | Val Loss: 0.4807 Acc: 0.9882 | LR: 5.58e-05
Epoch 99/200 | Train Loss: 0.4423 Acc: 0.9950 | Val Loss: 0.4834 Acc: 0.9882 | LR: 5.50e-05
Epoch 100/200 | Train Loss: 0.4406 Acc: 0.9950 | Val Loss: 0.4858 Acc: 0.9882 | LR: 5.41e-05
Epoch 101/200 | Train Loss: 0.4407 Acc: 0.9925 | Val Loss: 0.4830 Acc: 0.9882 | LR:

5.33e-05
Epoch 102/200 | Train Loss: 0.4395 Acc: 1.0000 | Val Loss: 0.4838 Acc: 0.9882 | LR: 5.25e-05
Epoch 103/200 | Train Loss: 0.4489 Acc: 0.9925 | Val Loss: 0.4847 Acc: 0.9882 | LR: 5.17e-05
Epoch 104/200 | Train Loss: 0.4400 Acc: 1.0000 | Val Loss: 0.4866 Acc: 0.9882 | LR: 5.08e-05
Epoch 105/200 | Train Loss: 0.4381 Acc: 0.9925 | Val Loss: 0.4826 Acc: 0.9882 | LR: 5.00e-05
Epoch 106/200 | Train Loss: 0.4446 Acc: 1.0000 | Val Loss: 0.4783 Acc: 0.9882 | LR: 4.92e-05
Epoch 107/200 | Train Loss: 0.4402 Acc: 0.9975 | Val Loss: 0.4766 Acc: 0.9882 | LR: 4.83e-05
Epoch 108/200 | Train Loss: 0.4385 Acc: 0.9975 | Val Loss: 0.4759 Acc: 0.9882 | LR: 4.75e-05
Epoch 109/200 | Train Loss: 0.4366 Acc: 0.9975 | Val Loss: 0.4772 Acc: 0.9882 | LR: 4.67e-05
Epoch 110/200 | Train Loss: 0.4318 Acc: 0.9975 | Val Loss: 0.4782 Acc: 0.9882 | LR: 4.59e-05
Epoch 111/200 | Train Loss: 0.4369 Acc: 0.9975 | Val Loss: 0.4781 Acc: 0.9882 | LR: 4.50e-05
Epoch 112/200 | Train Loss: 0.4362 Acc: 0.9975 | Val Loss: 0.4779 Acc: 0.9882 | LR: 4.42e-05
Epoch 113/200 | Train Loss: 0.4382 Acc: 0.9975 | Val Loss: 0.4760 Acc: 0.9882 | LR: 4.34e-05
Epoch 114/200 | Train Loss: 0.4352 Acc: 0.9950 | Val Loss: 0.4743 Acc: 0.9882 | LR: 4.26e-05
Epoch 115/200 | Train Loss: 0.4384 Acc: 0.9975 | Val Loss: 0.4747 Acc: 0.9882 | LR: 4.18e-05
Epoch 116/200 | Train Loss: 0.4407 Acc: 1.0000 | Val Loss: 0.4743 Acc: 0.9882 | LR: 4.10e-05
Epoch 117/200 | Train Loss: 0.4373 Acc: 0.9975 | Val Loss: 0.4768 Acc: 0.9882 | LR: 4.01e-05
Epoch 118/200 | Train Loss: 0.4380 Acc: 1.0000 | Val Loss: 0.4770 Acc: 0.9882 | LR: 3.93e-05
Epoch 119/200 | Train Loss: 0.4358 Acc: 1.0000 | Val Loss: 0.4801 Acc: 0.9882 | LR: 3.85e-05
Epoch 120/200 | Train Loss: 0.4358 Acc: 1.0000 | Val Loss: 0.4796 Acc: 0.9882 | LR: 3.77e-05
Epoch 121/200 | Train Loss: 0.4417 Acc: 0.9975 | Val Loss: 0.4798 Acc: 0.9882 | LR: 3.69e-05
Epoch 122/200 | Train Loss: 0.4386 Acc: 1.0000 | Val Loss: 0.4802 Acc: 0.9882 | LR: 3.61e-05
Epoch 123/200 | Train Loss: 0.4354 Acc: 1.0000 | Val Loss: 0.4795 Acc: 0.9882 | LR: 3.53e-05
Epoch 124/200 | Train Loss: 0.4365 Acc: 0.9975 | Val Loss: 0.4797 Acc: 0.9765 | LR: 3.45e-05
Epoch 125/200 | Train Loss: 0.4419 Acc: 0.9950 | Val Loss: 0.4788 Acc: 0.9882 | LR: 3.38e-05
Epoch 126/200 | Train Loss: 0.4418 Acc: 0.9975 | Val Loss: 0.4774 Acc: 0.9882 | LR: 3.30e-05
Epoch 127/200 | Train Loss: 0.4343 Acc: 0.9975 | Val Loss: 0.4782 Acc: 0.9882 | LR: 3.22e-05
Epoch 128/200 | Train Loss: 0.4360 Acc: 1.0000 | Val Loss: 0.4796 Acc: 0.9882 | LR: 3.14e-05
Epoch 129/200 | Train Loss: 0.4398 Acc: 0.9975 | Val Loss: 0.4762 Acc: 0.9882 | LR:

3.07e-05
Epoch 130/200 | Train Loss: 0.4340 Acc: 1.0000 | Val Loss: 0.4775 Acc: 0.9882 | LR: 2.99e-05
Epoch 131/200 | Train Loss: 0.4351 Acc: 0.9975 | Val Loss: 0.4763 Acc: 0.9882 | LR: 2.92e-05
Epoch 132/200 | Train Loss: 0.4356 Acc: 0.9950 | Val Loss: 0.4744 Acc: 0.9882 | LR: 2.84e-05
Epoch 133/200 | Train Loss: 0.4328 Acc: 1.0000 | Val Loss: 0.4736 Acc: 0.9882 | LR: 2.77e-05
Epoch 134/200 | Train Loss: 0.4365 Acc: 1.0000 | Val Loss: 0.4752 Acc: 0.9882 | LR: 2.69e-05
Epoch 135/200 | Train Loss: 0.4362 Acc: 0.9975 | Val Loss: 0.4759 Acc: 0.9882 | LR: 2.62e-05
Epoch 136/200 | Train Loss: 0.4321 Acc: 0.9975 | Val Loss: 0.4769 Acc: 0.9882 | LR: 2.55e-05
Epoch 137/200 | Train Loss: 0.4413 Acc: 0.9950 | Val Loss: 0.4768 Acc: 0.9882 | LR: 2.48e-05
Epoch 138/200 | Train Loss: 0.4343 Acc: 1.0000 | Val Loss: 0.4753 Acc: 0.9882 | LR: 2.41e-05
Epoch 139/200 | Train Loss: 0.4319 Acc: 1.0000 | Val Loss: 0.4760 Acc: 0.9882 | LR: 2.33e-05
Epoch 140/200 | Train Loss: 0.4355 Acc: 0.9975 | Val Loss: 0.4762 Acc: 0.9882 | LR: 2.27e-05
Epoch 141/200 | Train Loss: 0.4400 Acc: 0.9950 | Val Loss: 0.4753 Acc: 0.9882 | LR: 2.20e-05
Epoch 142/200 | Train Loss: 0.4327 Acc: 0.9975 | Val Loss: 0.4757 Acc: 0.9882 | LR: 2.13e-05
Epoch 143/200 | Train Loss: 0.4362 Acc: 0.9975 | Val Loss: 0.4742 Acc: 0.9882 | LR: 2.06e-05
Epoch 144/200 | Train Loss: 0.4321 Acc: 1.0000 | Val Loss: 0.4767 Acc: 0.9882 | LR: 1.99e-05
Epoch 145/200 | Train Loss: 0.4296 Acc: 1.0000 | Val Loss: 0.4762 Acc: 0.9882 | LR: 1.93e-05
Epoch 146/200 | Train Loss: 0.4327 Acc: 1.0000 | Val Loss: 0.4768 Acc: 0.9882 | LR: 1.86e-05
Epoch 147/200 | Train Loss: 0.4412 Acc: 0.9975 | Val Loss: 0.4777 Acc: 0.9882 | LR: 1.80e-05
Epoch 148/200 | Train Loss: 0.4374 Acc: 1.0000 | Val Loss: 0.4770 Acc: 0.9882 | LR: 1.74e-05
Epoch 149/200 | Train Loss: 0.4320 Acc: 0.9975 | Val Loss: 0.4789 Acc: 0.9882 | LR: 1.67e-05
Epoch 150/200 | Train Loss: 0.4353 Acc: 1.0000 | Val Loss: 0.4780 Acc: 0.9882 | LR: 1.61e-05
Epoch 151/200 | Train Loss: 0.4329 Acc: 1.0000 | Val Loss: 0.4765 Acc: 0.9882 | LR: 1.55e-05
Epoch 152/200 | Train Loss: 0.4309 Acc: 0.9950 | Val Loss: 0.4746 Acc: 0.9882 | LR: 1.49e-05
Epoch 153/200 | Train Loss: 0.4326 Acc: 0.9975 | Val Loss: 0.4744 Acc: 0.9882 | LR: 1.44e-05
Early stopping triggered.
Training complete in 11.7 minutes. Best Val Loss: 0.4736
Best model for fold saved to: new_results_Multimodal_DINO_Concat_KFold\best_model_fold_3.pth



Saved training history plot to: new_results_Multimodal_DINO_Concat_KFold\training_history_fold_3.png

=====

OVERALL PERFORMANCE (FOLD 3)

=====

	precision	recall	f1-score	support
Herpes	0.86	0.86	0.86	7
Melanoma	1.00	1.00	1.00	3
Monkeypox	1.00	0.94	0.97	18
Sarampion	0.97	0.97	0.97	29
Varicela	0.96	1.00	0.98	24
accuracy			0.96	81
macro avg	0.96	0.95	0.95	81
weighted avg	0.96	0.96	0.96	81

--- Overall Scalar Metrics ---

Accuracy: 0.9630

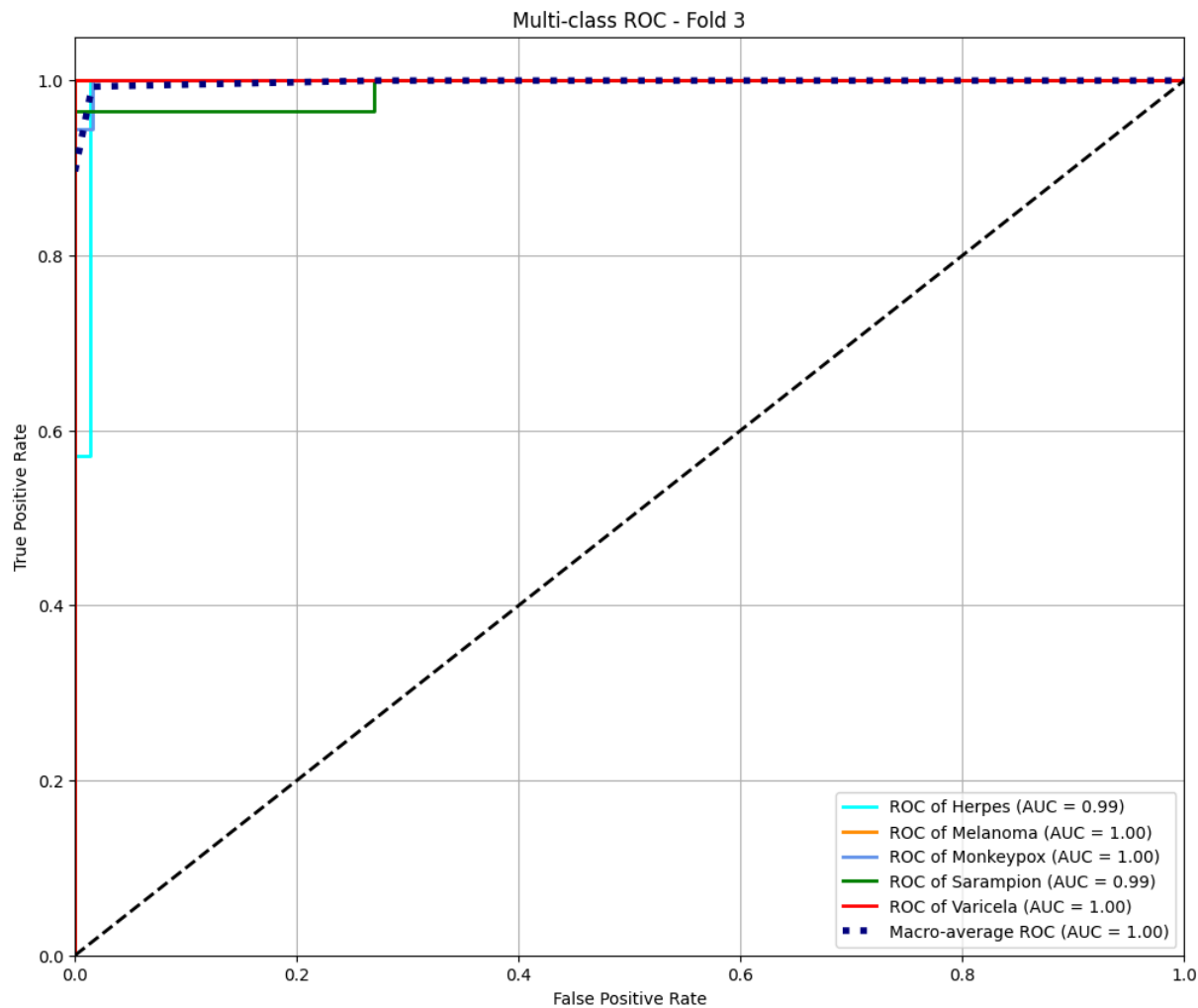
Macro ROC AUC: 0.9968

Cohen's Kappa: 0.9489

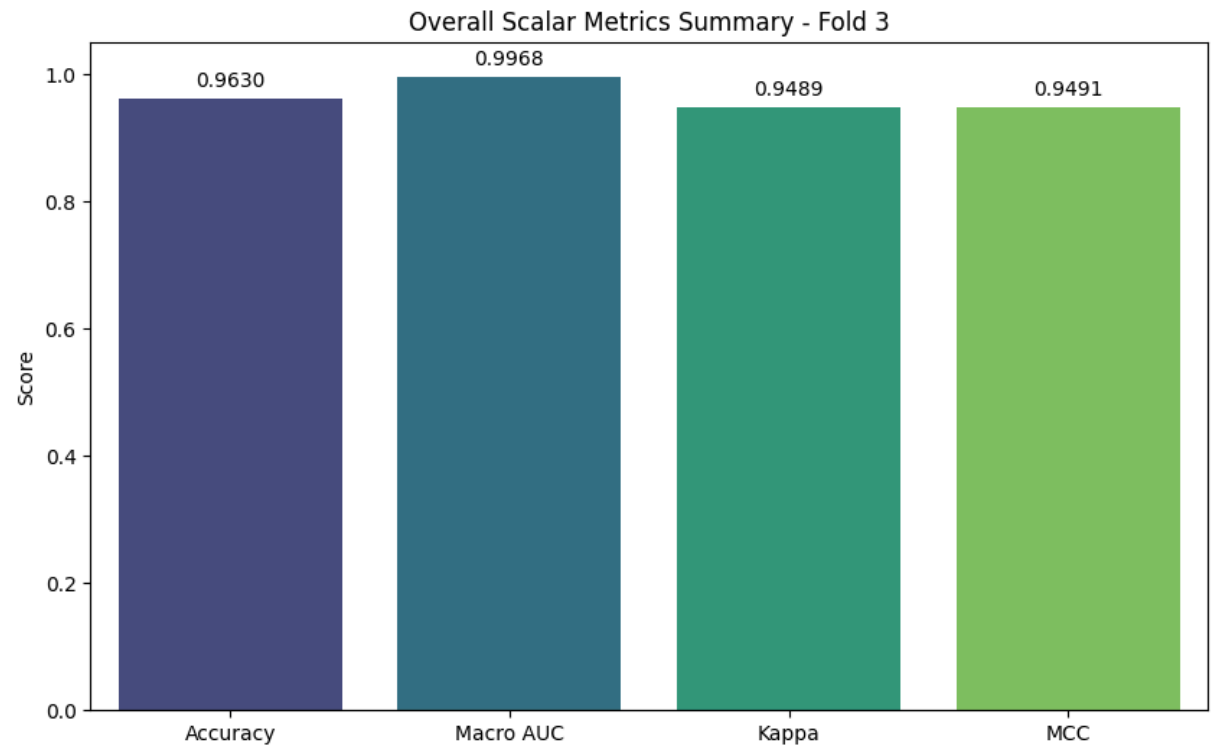
Matthews CorrCoef: 0.9491



Saved normalized confusion matrix to: new_results_Multimodal_DINO_Concat_KFold\confu
sion_matrix_fold_3.png



Saved ROC curves plot to: new_results_Multimodal_DINO_Concat_KFold\roc_curves_fold_3.png



Saved scalar metrics plot to: new_results_Multimodal_DINO_Concat_KFold\scalar_metrics_fold_3.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	0.50	1.00	0.67	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	0.95	0.98	22
Varicela	1.00	1.00	1.00	10
accuracy			0.97	36
macro avg	0.70	0.79	0.73	36
weighted avg	0.99	0.97	0.98	36

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	1.00	0.83	0.91	6
Melanoma	1.00	1.00	1.00	3
Monkeypox	1.00	0.93	0.97	15
Sarampion	0.88	1.00	0.93	7
Varicela	0.93	1.00	0.97	14
accuracy			0.96	45
macro avg	0.96	0.95	0.95	45
weighted avg	0.96	0.96	0.96	45

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	0.67	0.80	3
Varicela	1.00	1.00	1.00	5
accuracy			0.88	8
macro avg	0.40	0.33	0.36	8
weighted avg	1.00	0.88	0.93	8

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	1.00	0.86	0.92	7
Melanoma	1.00	1.00	1.00	3
Monkeypox	1.00	0.94	0.97	18
Sarampion	0.96	1.00	0.98	26
Varicela	0.95	1.00	0.97	19
accuracy			0.97	73
macro avg	0.98	0.96	0.97	73
weighted avg	0.97	0.97	0.97	73

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	2
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.67	0.80	3
Sarampion	1.00	1.00	1.00	4
Varicela	0.67	1.00	0.80	2
accuracy			0.91	11
macro avg	0.73	0.73	0.72	11
weighted avg	0.94	0.91	0.91	11

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	3
macro avg	0.20	0.20	0.20	3
weighted avg	1.00	1.00	1.00	3

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	0.50	1.00	0.67	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	0.00	0.00	0.00	1
Varicela	0.00	0.00	0.00	0
accuracy			0.83	6

macro avg	0.30	0.40	0.33	6
weighted avg	0.75	0.83	0.78	6

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	1.00	1.00	1.00	1
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	7
Varicela	0.00	0.00	0.00	0
accuracy			1.00	10
macro avg	0.60	0.60	0.60	10
weighted avg	1.00	1.00	1.00	10

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	1.00	1.00	1.00	1
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	4
macro avg	0.60	0.60	0.60	4
weighted avg	1.00	1.00	1.00	4

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	1.00	0.75	0.86	4
Melanoma	1.00	1.00	1.00	1
Monkeypox	1.00	1.00	1.00	4
Sarampion	0.94	1.00	0.97	16
Varicela	1.00	1.00	1.00	22
accuracy			0.98	47
macro avg	0.99	0.95	0.97	47
weighted avg	0.98	0.98	0.98	47

=====

FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.0000 EOD: 1.0000

Worst pair: 18-59|female|white vs 18-59|male|middle eastern

=====

=====

FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.9500 EOD: 0.4667

Worst pair: 18-59|female|white vs 18-59|male|white

Groups: ['18-59|female|white', '18-59|male|white']

=====

```
#####  
####  
### FOLD 4 ###  
#####  
####  
Trainable params: 0.56M
```

START TRAINING (DINOv3 + Concat) for 200 epochs...

Epoch 1/200		Train Loss: 1.7549	Acc: 0.1805		Val Loss: 1.6186	Acc: 0.1882		LR: 2.00e-05
Epoch 2/200		Train Loss: 1.6489	Acc: 0.2506		Val Loss: 1.5936	Acc: 0.3294		LR: 3.00e-05
Epoch 3/200		Train Loss: 1.5918	Acc: 0.3283		Val Loss: 1.5471	Acc: 0.4235		LR: 4.00e-05
Epoch 4/200		Train Loss: 1.4925	Acc: 0.4135		Val Loss: 1.4765	Acc: 0.5176		LR: 5.00e-05
Epoch 5/200		Train Loss: 1.3451	Acc: 0.5288		Val Loss: 1.3821	Acc: 0.5647		LR: 6.00e-05
Epoch 6/200		Train Loss: 1.2195	Acc: 0.6266		Val Loss: 1.2671	Acc: 0.6706		LR: 7.00e-05
Epoch 7/200		Train Loss: 1.1458	Acc: 0.6642		Val Loss: 1.1576	Acc: 0.7294		LR: 8.00e-05
Epoch 8/200		Train Loss: 1.0598	Acc: 0.7393		Val Loss: 1.0473	Acc: 0.7765		LR: 9.00e-05
Epoch 9/200		Train Loss: 0.9483	Acc: 0.7694		Val Loss: 0.9551	Acc: 0.7765		LR: 1.00e-04
Epoch 10/200		Train Loss: 0.8878	Acc: 0.7970		Val Loss: 0.8805	Acc: 0.8118		LR: 1.00e-04
Epoch 11/200		Train Loss: 0.8315	Acc: 0.8246		Val Loss: 0.8236	Acc: 0.8118		LR: 1.00e-04
Epoch 12/200		Train Loss: 0.7702	Acc: 0.8521		Val Loss: 0.7796	Acc: 0.8471		LR: 1.00e-04
Epoch 13/200		Train Loss: 0.7247	Acc: 0.8872		Val Loss: 0.7389	Acc: 0.8706		LR: 9.99e-05
Epoch 14/200		Train Loss: 0.6912	Acc: 0.8797		Val Loss: 0.7063	Acc: 0.8824		LR: 9.99e-05
Epoch 15/200		Train Loss: 0.6607	Acc: 0.9073		Val Loss: 0.6827	Acc: 0.8941		LR: 9.98e-05
Epoch 16/200		Train Loss: 0.6470	Acc: 0.9098		Val Loss: 0.6632	Acc: 0.8941		LR: 9.98e-05
Epoch 17/200		Train Loss: 0.6322	Acc: 0.9173		Val Loss: 0.6473	Acc: 0.9059		LR: 9.97e-05
Epoch 18/200		Train Loss: 0.6061	Acc: 0.9524		Val Loss: 0.6290	Acc: 0.9294		LR: 9.96e-05
Epoch 19/200		Train Loss: 0.5974	Acc: 0.9323		Val Loss: 0.6167	Acc: 0.9176		LR: 9.94e-05
Epoch 20/200		Train Loss: 0.5833	Acc: 0.9424		Val Loss: 0.6080	Acc: 0.9294		LR: 9.93e-05
Epoch 21/200		Train Loss: 0.5799	Acc: 0.9574		Val Loss: 0.5990	Acc: 0.9294		LR: 9.92e-05
Epoch 22/200		Train Loss: 0.5684	Acc: 0.9474		Val Loss: 0.5958	Acc: 0.9176		LR: 9.90e-05
Epoch 23/200		Train Loss: 0.5589	Acc: 0.9524		Val Loss: 0.5936	Acc: 0.9176		LR: 9.88e-05
Epoch 24/200		Train Loss: 0.5442	Acc: 0.9624		Val Loss: 0.5918	Acc: 0.9294		LR:

9.87e-05
Epoch 25/200 | Train Loss: 0.5438 Acc: 0.9549 | Val Loss: 0.5911 Acc: 0.9176 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.5306 Acc: 0.9649 | Val Loss: 0.5879 Acc: 0.9059 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.5214 Acc: 0.9699 | Val Loss: 0.5870 Acc: 0.9059 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.5118 Acc: 0.9724 | Val Loss: 0.5866 Acc: 0.9294 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.5063 Acc: 0.9825 | Val Loss: 0.5847 Acc: 0.9412 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.5165 Acc: 0.9774 | Val Loss: 0.5798 Acc: 0.9412 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.5013 Acc: 0.9799 | Val Loss: 0.5749 Acc: 0.9529 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.4977 Acc: 0.9875 | Val Loss: 0.5724 Acc: 0.9529 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.5007 Acc: 0.9825 | Val Loss: 0.5651 Acc: 0.9529 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.4934 Acc: 0.9724 | Val Loss: 0.5647 Acc: 0.9529 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.4857 Acc: 0.9875 | Val Loss: 0.5613 Acc: 0.9529 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.5031 Acc: 0.9749 | Val Loss: 0.5593 Acc: 0.9529 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.4818 Acc: 0.9925 | Val Loss: 0.5573 Acc: 0.9529 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.4833 Acc: 0.9825 | Val Loss: 0.5575 Acc: 0.9529 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.4790 Acc: 0.9875 | Val Loss: 0.5581 Acc: 0.9529 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.4846 Acc: 0.9850 | Val Loss: 0.5551 Acc: 0.9529 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.4757 Acc: 0.9900 | Val Loss: 0.5532 Acc: 0.9529 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.4728 Acc: 0.9900 | Val Loss: 0.5530 Acc: 0.9529 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.4792 Acc: 0.9900 | Val Loss: 0.5523 Acc: 0.9529 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.4689 Acc: 0.9900 | Val Loss: 0.5514 Acc: 0.9529 | LR: 9.23e-05
Epoch 45/200 | Train Loss: 0.4675 Acc: 0.9925 | Val Loss: 0.5513 Acc: 0.9529 | LR: 9.19e-05
Epoch 46/200 | Train Loss: 0.4698 Acc: 0.9875 | Val Loss: 0.5509 Acc: 0.9529 | LR: 9.14e-05
Epoch 47/200 | Train Loss: 0.4673 Acc: 0.9900 | Val Loss: 0.5520 Acc: 0.9529 | LR: 9.09e-05
Epoch 48/200 | Train Loss: 0.4631 Acc: 0.9975 | Val Loss: 0.5491 Acc: 0.9529 | LR: 9.05e-05
Epoch 49/200 | Train Loss: 0.4738 Acc: 0.9925 | Val Loss: 0.5468 Acc: 0.9529 | LR: 9.00e-05
Epoch 50/200 | Train Loss: 0.4558 Acc: 0.9925 | Val Loss: 0.5460 Acc: 0.9529 | LR: 8.95e-05
Epoch 51/200 | Train Loss: 0.4572 Acc: 0.9950 | Val Loss: 0.5459 Acc: 0.9529 | LR: 8.89e-05
Epoch 52/200 | Train Loss: 0.4575 Acc: 0.9900 | Val Loss: 0.5482 Acc: 0.9529 | LR:

8.84e-05
Epoch 53/200 | Train Loss: 0.4647 Acc: 0.9850 | Val Loss: 0.5482 Acc: 0.9529 | LR: 8.79e-05
Epoch 54/200 | Train Loss: 0.4609 Acc: 0.9850 | Val Loss: 0.5500 Acc: 0.9529 | LR: 8.73e-05
Epoch 55/200 | Train Loss: 0.4569 Acc: 0.9925 | Val Loss: 0.5493 Acc: 0.9529 | LR: 8.68e-05
Epoch 56/200 | Train Loss: 0.4531 Acc: 1.0000 | Val Loss: 0.5476 Acc: 0.9412 | LR: 8.62e-05
Epoch 57/200 | Train Loss: 0.4604 Acc: 0.9875 | Val Loss: 0.5501 Acc: 0.9529 | LR: 8.56e-05
Epoch 58/200 | Train Loss: 0.4567 Acc: 0.9925 | Val Loss: 0.5489 Acc: 0.9529 | LR: 8.51e-05
Epoch 59/200 | Train Loss: 0.4511 Acc: 0.9975 | Val Loss: 0.5473 Acc: 0.9412 | LR: 8.45e-05
Epoch 60/200 | Train Loss: 0.4627 Acc: 0.9925 | Val Loss: 0.5468 Acc: 0.9529 | LR: 8.39e-05
Epoch 61/200 | Train Loss: 0.4546 Acc: 0.9950 | Val Loss: 0.5470 Acc: 0.9647 | LR: 8.33e-05
Epoch 62/200 | Train Loss: 0.4572 Acc: 0.9925 | Val Loss: 0.5440 Acc: 0.9529 | LR: 8.26e-05
Epoch 63/200 | Train Loss: 0.4533 Acc: 0.9975 | Val Loss: 0.5470 Acc: 0.9529 | LR: 8.20e-05
Epoch 64/200 | Train Loss: 0.4544 Acc: 0.9950 | Val Loss: 0.5418 Acc: 0.9529 | LR: 8.14e-05
Epoch 65/200 | Train Loss: 0.4514 Acc: 0.9925 | Val Loss: 0.5425 Acc: 0.9529 | LR: 8.07e-05
Epoch 66/200 | Train Loss: 0.4478 Acc: 0.9950 | Val Loss: 0.5378 Acc: 0.9529 | LR: 8.01e-05
Epoch 67/200 | Train Loss: 0.4451 Acc: 0.9950 | Val Loss: 0.5386 Acc: 0.9529 | LR: 7.94e-05
Epoch 68/200 | Train Loss: 0.4448 Acc: 0.9925 | Val Loss: 0.5391 Acc: 0.9529 | LR: 7.87e-05
Epoch 69/200 | Train Loss: 0.4471 Acc: 0.9975 | Val Loss: 0.5371 Acc: 0.9529 | LR: 7.80e-05
Epoch 70/200 | Train Loss: 0.4463 Acc: 0.9950 | Val Loss: 0.5382 Acc: 0.9529 | LR: 7.73e-05
Epoch 71/200 | Train Loss: 0.4430 Acc: 0.9950 | Val Loss: 0.5385 Acc: 0.9529 | LR: 7.67e-05
Epoch 72/200 | Train Loss: 0.4396 Acc: 1.0000 | Val Loss: 0.5388 Acc: 0.9529 | LR: 7.59e-05
Epoch 73/200 | Train Loss: 0.4453 Acc: 0.9925 | Val Loss: 0.5399 Acc: 0.9529 | LR: 7.52e-05
Epoch 74/200 | Train Loss: 0.4469 Acc: 0.9975 | Val Loss: 0.5427 Acc: 0.9529 | LR: 7.45e-05
Epoch 75/200 | Train Loss: 0.4441 Acc: 0.9975 | Val Loss: 0.5419 Acc: 0.9647 | LR: 7.38e-05
Epoch 76/200 | Train Loss: 0.4459 Acc: 0.9950 | Val Loss: 0.5419 Acc: 0.9529 | LR: 7.31e-05
Epoch 77/200 | Train Loss: 0.4411 Acc: 0.9975 | Val Loss: 0.5434 Acc: 0.9529 | LR: 7.23e-05
Epoch 78/200 | Train Loss: 0.4458 Acc: 0.9975 | Val Loss: 0.5417 Acc: 0.9647 | LR: 7.16e-05
Epoch 79/200 | Train Loss: 0.4381 Acc: 0.9975 | Val Loss: 0.5415 Acc: 0.9647 | LR: 7.08e-05
Epoch 80/200 | Train Loss: 0.4442 Acc: 0.9950 | Val Loss: 0.5396 Acc: 0.9529 | LR:

7.01e-05

Epoch 81/200 | Train Loss: 0.4408 Acc: 1.0000 | Val Loss: 0.5418 Acc: 0.9529 | LR: 6.93e-05

Epoch 82/200 | Train Loss: 0.4369 Acc: 1.0000 | Val Loss: 0.5411 Acc: 0.9647 | LR: 6.86e-05

Epoch 83/200 | Train Loss: 0.4354 Acc: 0.9950 | Val Loss: 0.5415 Acc: 0.9529 | LR: 6.78e-05

Epoch 84/200 | Train Loss: 0.4339 Acc: 1.0000 | Val Loss: 0.5417 Acc: 0.9647 | LR: 6.70e-05

Epoch 85/200 | Train Loss: 0.4358 Acc: 1.0000 | Val Loss: 0.5413 Acc: 0.9647 | LR: 6.62e-05

Epoch 86/200 | Train Loss: 0.4396 Acc: 1.0000 | Val Loss: 0.5422 Acc: 0.9647 | LR: 6.55e-05

Epoch 87/200 | Train Loss: 0.4395 Acc: 0.9975 | Val Loss: 0.5397 Acc: 0.9647 | LR: 6.47e-05

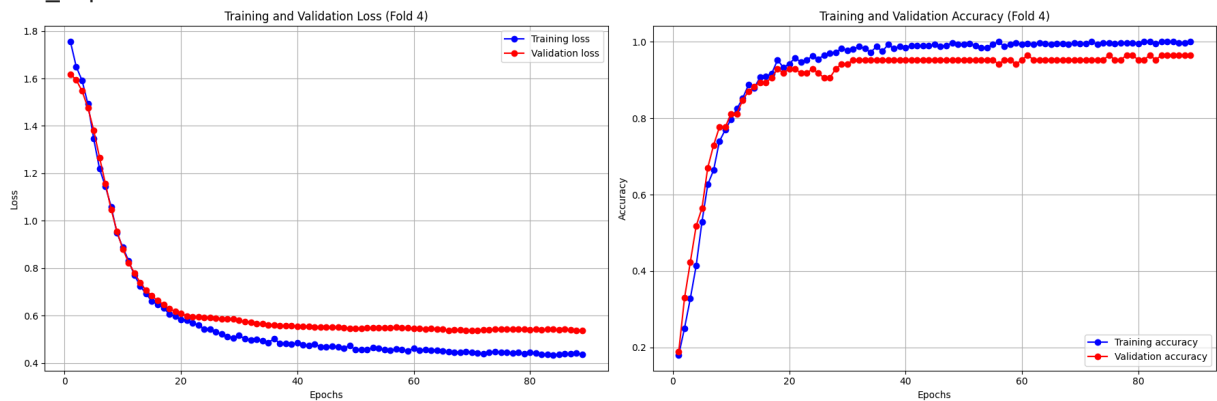
Epoch 88/200 | Train Loss: 0.4432 Acc: 0.9975 | Val Loss: 0.5374 Acc: 0.9647 | LR: 6.39e-05

Epoch 89/200 | Train Loss: 0.4350 Acc: 1.0000 | Val Loss: 0.5372 Acc: 0.9647 | LR: 6.31e-05

Early stopping triggered.

Training complete in 6.1 minutes. Best Val Loss: 0.5371

Best model for fold saved to: new_results_Multimodal_DINO_Concat_KFold\best_model_fold_4.pth



Saved training history plot to: new_results_Multimodal_DINO_Concat_KFold\training_history_fold_4.png

=====

OVERALL PERFORMANCE (FOLD 4)

=====

	precision	recall	f1-score	support
Herpes	1.00	0.83	0.91	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.94	0.84	0.89	19
Sarampion	0.82	0.93	0.88	30
Varicela	0.87	0.83	0.85	24
accuracy			0.88	81
macro avg	0.93	0.89	0.90	81
weighted avg	0.88	0.88	0.88	81

--- Overall Scalar Metrics ---

Accuracy: 0.8765

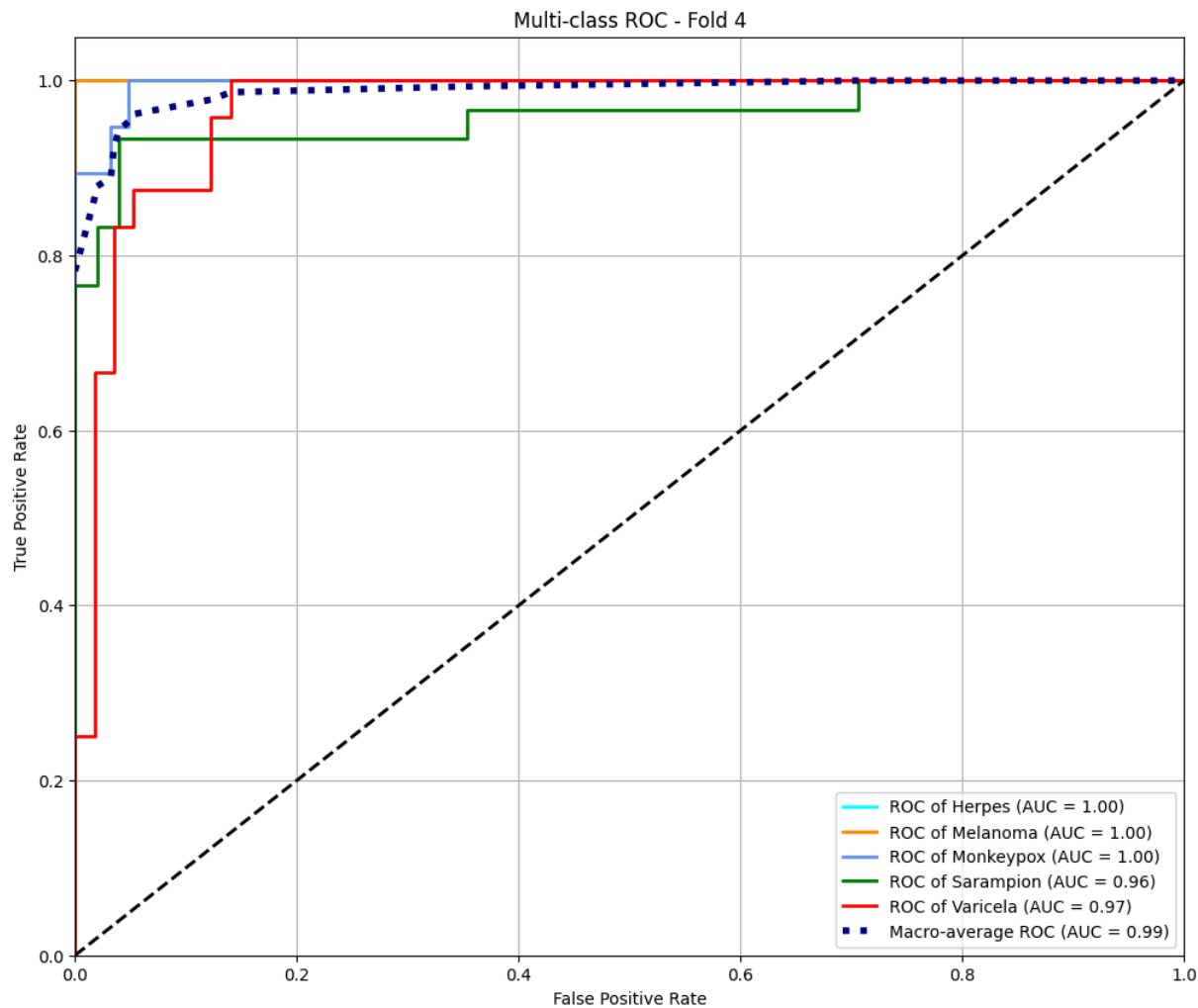
Macro ROC AUC: 0.9848

Cohen's Kappa: 0.8251

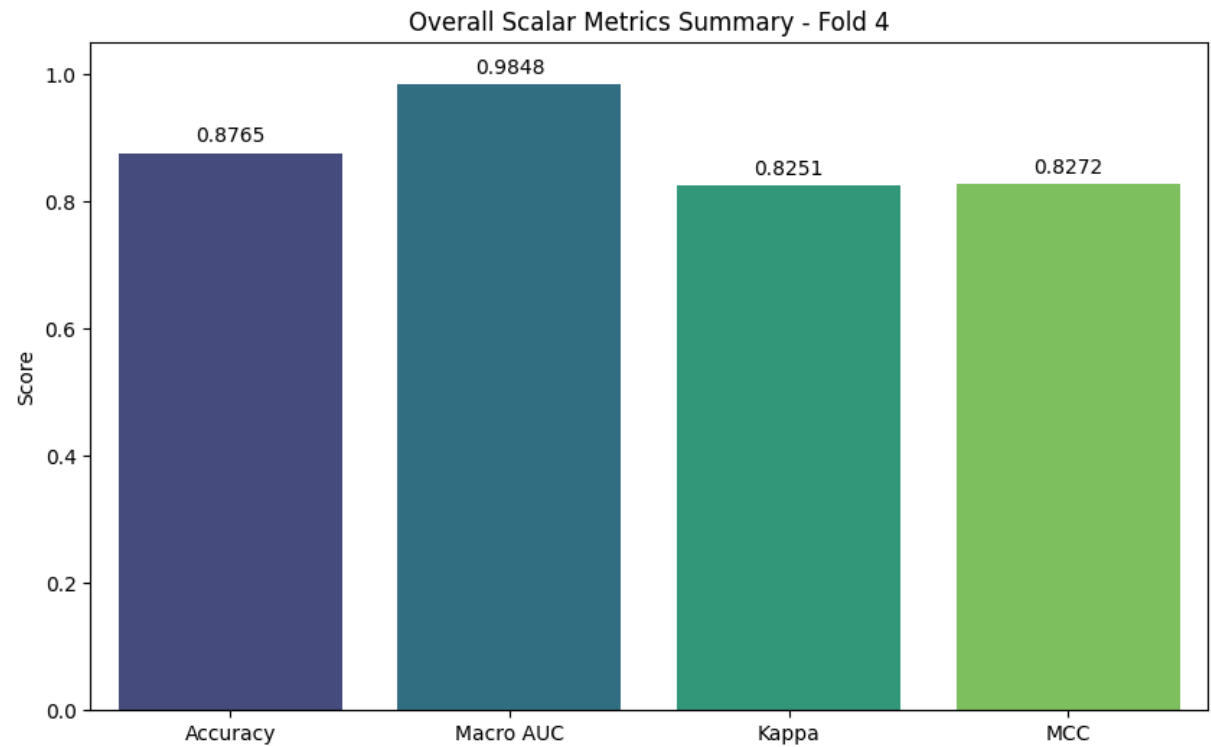
Matthews CorrCoef: 0.8272



Saved normalized confusion matrix to: new_results_Multimodal_DINO_Concat_KFold\confusion_matrix_fold_4.png



Saved ROC curves plot to: new_results_Multimodal_DINO_Concat_KFold\roc_curves_fold_4.png



Saved scalar metrics plot to: new_results_Multimodal_DINO_Concat_KFold\scalar_metrics_fold_4.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.67	0.80	3
Sarampion	0.78	1.00	0.88	14
Varicela	1.00	0.73	0.84	11
accuracy			0.86	29
macro avg	0.76	0.68	0.70	29
weighted avg	0.89	0.86	0.86	29

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	1.00	0.80	0.89	5
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.93	0.88	0.90	16
Sarampion	0.88	0.88	0.88	16
Varicela	0.80	0.92	0.86	13
accuracy			0.88	52
macro avg	0.92	0.89	0.90	52
weighted avg	0.89	0.88	0.89	52

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	0.50	0.67	2
Varicela	0.75	1.00	0.86	3
accuracy			0.80	5
macro avg	0.35	0.30	0.30	5
weighted avg	0.85	0.80	0.78	5

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	1.00	0.83	0.91	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.94	0.84	0.89	19
Sarampion	0.82	0.96	0.89	28
Varicela	0.89	0.81	0.85	21
accuracy			0.88	76
macro avg	0.93	0.89	0.91	76
weighted avg	0.89	0.88	0.88	76

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.50	0.67	0.57	3
Varicela	0.80	0.80	0.80	5
accuracy			0.70	10
macro avg	0.46	0.49	0.47	10
weighted avg	0.65	0.70	0.67	10

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	2
Varicela	0.00	0.00	0.00	0
accuracy			1.00	5
macro avg	0.40	0.40	0.40	5
weighted avg	1.00	1.00	1.00	5

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.80	1.00	0.89	4
Varicela	0.00	0.00	0.00	1
accuracy			0.86	7

macro avg	0.36	0.40	0.38	7
weighted avg	0.74	0.86	0.79	7

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.67	0.80	3
Sarampion	0.80	1.00	0.89	4
Varicela	0.00	0.00	0.00	0
accuracy			0.86	7
macro avg	0.36	0.33	0.34	7
weighted avg	0.89	0.86	0.85	7

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	1.00	1.00	1.00	1
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	4
macro avg	0.60	0.60	0.60	4
weighted avg	1.00	1.00	1.00	4

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	5
Melanoma	1.00	1.00	1.00	1
Monkeypox	0.86	0.75	0.80	8
Sarampion	0.88	0.94	0.91	16
Varicela	0.89	0.89	0.89	18
accuracy			0.90	48
macro avg	0.93	0.92	0.92	48
weighted avg	0.90	0.90	0.89	48

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.0000 EOD: 0.8778
Worst pair: 0-17|male|asian vs 18-59|male|white
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.8462 EOD: 0.4111
Worst pair: 18-59|female|white vs 18-59|male|white
Groups: ['18-59|female|white', '18-59|male|white']
=====
```

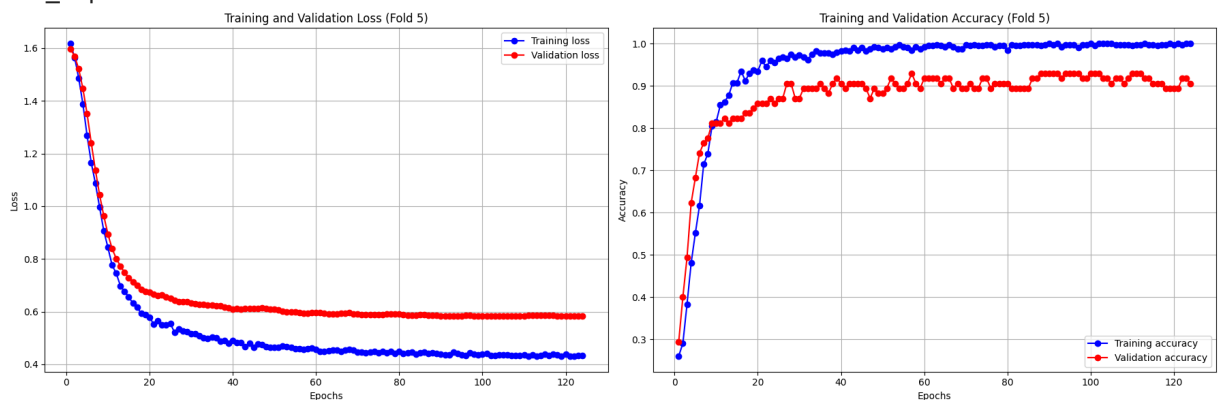
```
#####  
####  
### FOLD 5 ###  
#####  
####  
Trainable params: 0.56M  
  
START TRAINING (DINOv3 + Concat) for 200 epochs...  
Epoch 1/200 | Train Loss: 1.6173 Acc: 0.2607 | Val Loss: 1.5977 Acc: 0.2941 | LR: 2.00e-05  
Epoch 2/200 | Train Loss: 1.5649 Acc: 0.2907 | Val Loss: 1.5696 Acc: 0.4000 | LR: 3.00e-05  
Epoch 3/200 | Train Loss: 1.4851 Acc: 0.3835 | Val Loss: 1.5213 Acc: 0.4941 | LR: 4.00e-05  
Epoch 4/200 | Train Loss: 1.3876 Acc: 0.4812 | Val Loss: 1.4464 Acc: 0.6235 | LR: 5.00e-05  
Epoch 5/200 | Train Loss: 1.2682 Acc: 0.5514 | Val Loss: 1.3505 Acc: 0.6824 | LR: 6.00e-05  
Epoch 6/200 | Train Loss: 1.1651 Acc: 0.6165 | Val Loss: 1.2406 Acc: 0.7412 | LR: 7.00e-05  
Epoch 7/200 | Train Loss: 1.0866 Acc: 0.7143 | Val Loss: 1.1375 Acc: 0.7647 | LR: 8.00e-05  
Epoch 8/200 | Train Loss: 0.9968 Acc: 0.7393 | Val Loss: 1.0436 Acc: 0.7765 | LR: 9.00e-05  
Epoch 9/200 | Train Loss: 0.9077 Acc: 0.8045 | Val Loss: 0.9643 Acc: 0.8118 | LR: 1.00e-04  
Epoch 10/200 | Train Loss: 0.8438 Acc: 0.8145 | Val Loss: 0.8948 Acc: 0.8118 | LR: 1.00e-04  
Epoch 11/200 | Train Loss: 0.7771 Acc: 0.8546 | Val Loss: 0.8397 Acc: 0.8118 | LR: 1.00e-04  
Epoch 12/200 | Train Loss: 0.7455 Acc: 0.8622 | Val Loss: 0.8018 Acc: 0.8235 | LR: 1.00e-04  
Epoch 13/200 | Train Loss: 0.6984 Acc: 0.8772 | Val Loss: 0.7735 Acc: 0.8118 | LR: 9.99e-05  
Epoch 14/200 | Train Loss: 0.6765 Acc: 0.9073 | Val Loss: 0.7489 Acc: 0.8235 | LR: 9.99e-05  
Epoch 15/200 | Train Loss: 0.6566 Acc: 0.9073 | Val Loss: 0.7289 Acc: 0.8235 | LR: 9.98e-05  
Epoch 16/200 | Train Loss: 0.6320 Acc: 0.9348 | Val Loss: 0.7136 Acc: 0.8235 | LR: 9.98e-05  
Epoch 17/200 | Train Loss: 0.6178 Acc: 0.9123 | Val Loss: 0.6990 Acc: 0.8353 | LR: 9.97e-05  
Epoch 18/200 | Train Loss: 0.5927 Acc: 0.9298 | Val Loss: 0.6857 Acc: 0.8353 | LR: 9.96e-05  
Epoch 19/200 | Train Loss: 0.5894 Acc: 0.9373 | Val Loss: 0.6768 Acc: 0.8471 | LR: 9.94e-05  
Epoch 20/200 | Train Loss: 0.5797 Acc: 0.9348 | Val Loss: 0.6734 Acc: 0.8588 | LR: 9.93e-05  
Epoch 21/200 | Train Loss: 0.5521 Acc: 0.9599 | Val Loss: 0.6670 Acc: 0.8588 | LR: 9.92e-05  
Epoch 22/200 | Train Loss: 0.5655 Acc: 0.9449 | Val Loss: 0.6611 Acc: 0.8588 | LR: 9.90e-05  
Epoch 23/200 | Train Loss: 0.5494 Acc: 0.9599 | Val Loss: 0.6642 Acc: 0.8706 | LR: 9.88e-05  
Epoch 24/200 | Train Loss: 0.5508 Acc: 0.9549 | Val Loss: 0.6565 Acc: 0.8588 | LR:
```

9.87e-05
Epoch 25/200 | Train Loss: 0.5550 Acc: 0.9649 | Val Loss: 0.6517 Acc: 0.8706 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.5218 Acc: 0.9674 | Val Loss: 0.6422 Acc: 0.8706 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.5354 Acc: 0.9649 | Val Loss: 0.6387 Acc: 0.9059 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.5272 Acc: 0.9749 | Val Loss: 0.6368 Acc: 0.9059 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.5240 Acc: 0.9674 | Val Loss: 0.6379 Acc: 0.8706 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.5168 Acc: 0.9724 | Val Loss: 0.6337 Acc: 0.8706 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.5159 Acc: 0.9674 | Val Loss: 0.6306 Acc: 0.8941 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.5086 Acc: 0.9624 | Val Loss: 0.6280 Acc: 0.8941 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.4996 Acc: 0.9749 | Val Loss: 0.6270 Acc: 0.8941 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.4974 Acc: 0.9825 | Val Loss: 0.6256 Acc: 0.8941 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.5037 Acc: 0.9774 | Val Loss: 0.6237 Acc: 0.9059 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.4997 Acc: 0.9774 | Val Loss: 0.6216 Acc: 0.8941 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.4880 Acc: 0.9774 | Val Loss: 0.6212 Acc: 0.8824 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.4908 Acc: 0.9749 | Val Loss: 0.6161 Acc: 0.9059 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.4790 Acc: 0.9799 | Val Loss: 0.6138 Acc: 0.9176 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.4905 Acc: 0.9825 | Val Loss: 0.6102 Acc: 0.9059 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.4823 Acc: 0.9850 | Val Loss: 0.6122 Acc: 0.8941 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.4826 Acc: 0.9825 | Val Loss: 0.6100 Acc: 0.9059 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.4683 Acc: 0.9900 | Val Loss: 0.6122 Acc: 0.9059 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.4792 Acc: 0.9850 | Val Loss: 0.6120 Acc: 0.9059 | LR: 9.23e-05
Epoch 45/200 | Train Loss: 0.4643 Acc: 0.9900 | Val Loss: 0.6113 Acc: 0.9059 | LR: 9.19e-05
Epoch 46/200 | Train Loss: 0.4772 Acc: 0.9825 | Val Loss: 0.6130 Acc: 0.8941 | LR: 9.14e-05
Epoch 47/200 | Train Loss: 0.4743 Acc: 0.9875 | Val Loss: 0.6149 Acc: 0.8706 | LR: 9.09e-05
Epoch 48/200 | Train Loss: 0.4667 Acc: 0.9925 | Val Loss: 0.6115 Acc: 0.8941 | LR: 9.05e-05
Epoch 49/200 | Train Loss: 0.4657 Acc: 0.9900 | Val Loss: 0.6098 Acc: 0.8824 | LR: 9.00e-05
Epoch 50/200 | Train Loss: 0.4636 Acc: 0.9875 | Val Loss: 0.6107 Acc: 0.8824 | LR: 8.95e-05
Epoch 51/200 | Train Loss: 0.4654 Acc: 0.9900 | Val Loss: 0.6071 Acc: 0.8941 | LR: 8.89e-05
Epoch 52/200 | Train Loss: 0.4710 Acc: 0.9875 | Val Loss: 0.6012 Acc: 0.9176 | LR:

8.84e-05
Epoch 53/200 | Train Loss: 0.4663 Acc: 0.9925 | Val Loss: 0.5993 Acc: 0.9059 | LR: 8.79e-05
Epoch 54/200 | Train Loss: 0.4655 Acc: 0.9975 | Val Loss: 0.6004 Acc: 0.8941 | LR: 8.73e-05
Epoch 55/200 | Train Loss: 0.4603 Acc: 0.9925 | Val Loss: 0.5989 Acc: 0.8941 | LR: 8.68e-05
Epoch 56/200 | Train Loss: 0.4585 Acc: 0.9900 | Val Loss: 0.5976 Acc: 0.9059 | LR: 8.62e-05
Epoch 57/200 | Train Loss: 0.4560 Acc: 0.9850 | Val Loss: 0.5950 Acc: 0.9294 | LR: 8.56e-05
Epoch 58/200 | Train Loss: 0.4595 Acc: 0.9925 | Val Loss: 0.5931 Acc: 0.9059 | LR: 8.51e-05
Epoch 59/200 | Train Loss: 0.4631 Acc: 0.9875 | Val Loss: 0.5952 Acc: 0.8941 | LR: 8.45e-05
Epoch 60/200 | Train Loss: 0.4563 Acc: 0.9925 | Val Loss: 0.5963 Acc: 0.9176 | LR: 8.39e-05
Epoch 61/200 | Train Loss: 0.4487 Acc: 0.9950 | Val Loss: 0.5969 Acc: 0.9176 | LR: 8.33e-05
Epoch 62/200 | Train Loss: 0.4500 Acc: 0.9950 | Val Loss: 0.5952 Acc: 0.9176 | LR: 8.26e-05
Epoch 63/200 | Train Loss: 0.4525 Acc: 0.9975 | Val Loss: 0.5920 Acc: 0.9176 | LR: 8.20e-05
Epoch 64/200 | Train Loss: 0.4537 Acc: 0.9950 | Val Loss: 0.5910 Acc: 0.9059 | LR: 8.14e-05
Epoch 65/200 | Train Loss: 0.4540 Acc: 0.9925 | Val Loss: 0.5907 Acc: 0.9176 | LR: 8.07e-05
Epoch 66/200 | Train Loss: 0.4483 Acc: 0.9975 | Val Loss: 0.5942 Acc: 0.9176 | LR: 8.01e-05
Epoch 67/200 | Train Loss: 0.4552 Acc: 0.9925 | Val Loss: 0.5950 Acc: 0.8941 | LR: 7.94e-05
Epoch 68/200 | Train Loss: 0.4562 Acc: 0.9875 | Val Loss: 0.5954 Acc: 0.9059 | LR: 7.87e-05
Epoch 69/200 | Train Loss: 0.4536 Acc: 0.9875 | Val Loss: 0.5915 Acc: 0.8941 | LR: 7.80e-05
Epoch 70/200 | Train Loss: 0.4453 Acc: 0.9975 | Val Loss: 0.5905 Acc: 0.8941 | LR: 7.73e-05
Epoch 71/200 | Train Loss: 0.4460 Acc: 0.9950 | Val Loss: 0.5893 Acc: 0.9059 | LR: 7.67e-05
Epoch 72/200 | Train Loss: 0.4444 Acc: 0.9975 | Val Loss: 0.5884 Acc: 0.8941 | LR: 7.59e-05
Epoch 73/200 | Train Loss: 0.4468 Acc: 0.9950 | Val Loss: 0.5888 Acc: 0.8941 | LR: 7.52e-05
Epoch 74/200 | Train Loss: 0.4485 Acc: 0.9950 | Val Loss: 0.5885 Acc: 0.9176 | LR: 7.45e-05
Epoch 75/200 | Train Loss: 0.4433 Acc: 0.9975 | Val Loss: 0.5878 Acc: 0.9176 | LR: 7.38e-05
Epoch 76/200 | Train Loss: 0.4478 Acc: 0.9975 | Val Loss: 0.5892 Acc: 0.8941 | LR: 7.31e-05
Epoch 77/200 | Train Loss: 0.4445 Acc: 0.9925 | Val Loss: 0.5905 Acc: 0.9059 | LR: 7.23e-05
Epoch 78/200 | Train Loss: 0.4502 Acc: 0.9950 | Val Loss: 0.5910 Acc: 0.9059 | LR: 7.16e-05
Epoch 79/200 | Train Loss: 0.4418 Acc: 0.9950 | Val Loss: 0.5920 Acc: 0.9059 | LR: 7.08e-05
Epoch 80/200 | Train Loss: 0.4487 Acc: 0.9850 | Val Loss: 0.5912 Acc: 0.9059 | LR:

7.01e-05
Epoch 81/200 | Train Loss: 0.4410 Acc: 0.9975 | Val Loss: 0.5887 Acc: 0.8941 | LR: 6.93e-05
Epoch 82/200 | Train Loss: 0.4429 Acc: 0.9950 | Val Loss: 0.5874 Acc: 0.8941 | LR: 6.86e-05
Epoch 83/200 | Train Loss: 0.4455 Acc: 0.9950 | Val Loss: 0.5858 Acc: 0.8941 | LR: 6.78e-05
Epoch 84/200 | Train Loss: 0.4380 Acc: 0.9975 | Val Loss: 0.5862 Acc: 0.8941 | LR: 6.70e-05
Epoch 85/200 | Train Loss: 0.4432 Acc: 0.9975 | Val Loss: 0.5889 Acc: 0.8941 | LR: 6.62e-05
Epoch 86/200 | Train Loss: 0.4459 Acc: 0.9975 | Val Loss: 0.5876 Acc: 0.9176 | LR: 6.55e-05
Epoch 87/200 | Train Loss: 0.4410 Acc: 0.9975 | Val Loss: 0.5863 Acc: 0.9176 | LR: 6.47e-05
Epoch 88/200 | Train Loss: 0.4428 Acc: 0.9950 | Val Loss: 0.5852 Acc: 0.9294 | LR: 6.39e-05
Epoch 89/200 | Train Loss: 0.4426 Acc: 0.9975 | Val Loss: 0.5858 Acc: 0.9294 | LR: 6.31e-05
Epoch 90/200 | Train Loss: 0.4382 Acc: 1.0000 | Val Loss: 0.5844 Acc: 0.9294 | LR: 6.23e-05
Epoch 91/200 | Train Loss: 0.4368 Acc: 0.9975 | Val Loss: 0.5834 Acc: 0.9294 | LR: 6.15e-05
Epoch 92/200 | Train Loss: 0.4368 Acc: 1.0000 | Val Loss: 0.5848 Acc: 0.9294 | LR: 6.07e-05
Epoch 93/200 | Train Loss: 0.4455 Acc: 0.9925 | Val Loss: 0.5849 Acc: 0.9176 | LR: 5.99e-05
Epoch 94/200 | Train Loss: 0.4421 Acc: 0.9975 | Val Loss: 0.5842 Acc: 0.9294 | LR: 5.90e-05
Epoch 95/200 | Train Loss: 0.4372 Acc: 0.9975 | Val Loss: 0.5840 Acc: 0.9294 | LR: 5.82e-05
Epoch 96/200 | Train Loss: 0.4343 Acc: 0.9975 | Val Loss: 0.5860 Acc: 0.9294 | LR: 5.74e-05
Epoch 97/200 | Train Loss: 0.4449 Acc: 0.9900 | Val Loss: 0.5851 Acc: 0.9294 | LR: 5.66e-05
Epoch 98/200 | Train Loss: 0.4392 Acc: 0.9975 | Val Loss: 0.5829 Acc: 0.9176 | LR: 5.58e-05
Epoch 99/200 | Train Loss: 0.4373 Acc: 0.9975 | Val Loss: 0.5838 Acc: 0.9176 | LR: 5.50e-05
Epoch 100/200 | Train Loss: 0.4383 Acc: 1.0000 | Val Loss: 0.5825 Acc: 0.9294 | LR: 5.41e-05
Epoch 101/200 | Train Loss: 0.4417 Acc: 0.9950 | Val Loss: 0.5837 Acc: 0.9294 | LR: 5.33e-05
Epoch 102/200 | Train Loss: 0.4332 Acc: 1.0000 | Val Loss: 0.5839 Acc: 0.9294 | LR: 5.25e-05
Epoch 103/200 | Train Loss: 0.4329 Acc: 1.0000 | Val Loss: 0.5827 Acc: 0.9176 | LR: 5.17e-05
Epoch 104/200 | Train Loss: 0.4369 Acc: 1.0000 | Val Loss: 0.5823 Acc: 0.9176 | LR: 5.08e-05
Epoch 105/200 | Train Loss: 0.4362 Acc: 1.0000 | Val Loss: 0.5833 Acc: 0.9059 | LR: 5.00e-05
Epoch 106/200 | Train Loss: 0.4351 Acc: 0.9975 | Val Loss: 0.5843 Acc: 0.9176 | LR: 4.92e-05
Epoch 107/200 | Train Loss: 0.4326 Acc: 0.9975 | Val Loss: 0.5845 Acc: 0.9176 | LR: 4.83e-05
Epoch 108/200 | Train Loss: 0.4347 Acc: 0.9975 | Val Loss: 0.5836 Acc: 0.9059 | LR:

4.75e-05
Epoch 109/200 | Train Loss: 0.4335 Acc: 0.9975 | Val Loss: 0.5833 Acc: 0.9176 | LR: 4.67e-05
Epoch 110/200 | Train Loss: 0.4375 Acc: 0.9950 | Val Loss: 0.5845 Acc: 0.9294 | LR: 4.59e-05
Epoch 111/200 | Train Loss: 0.4322 Acc: 0.9975 | Val Loss: 0.5852 Acc: 0.9294 | LR: 4.50e-05
Epoch 112/200 | Train Loss: 0.4360 Acc: 0.9975 | Val Loss: 0.5864 Acc: 0.9294 | LR: 4.42e-05
Epoch 113/200 | Train Loss: 0.4317 Acc: 1.0000 | Val Loss: 0.5855 Acc: 0.9176 | LR: 4.34e-05
Epoch 114/200 | Train Loss: 0.4328 Acc: 0.9975 | Val Loss: 0.5849 Acc: 0.9176 | LR: 4.26e-05
Epoch 115/200 | Train Loss: 0.4378 Acc: 0.9975 | Val Loss: 0.5861 Acc: 0.9059 | LR: 4.18e-05
Epoch 116/200 | Train Loss: 0.4329 Acc: 0.9950 | Val Loss: 0.5852 Acc: 0.9059 | LR: 4.10e-05
Epoch 117/200 | Train Loss: 0.4390 Acc: 0.9975 | Val Loss: 0.5853 Acc: 0.9059 | LR: 4.01e-05
Epoch 118/200 | Train Loss: 0.4363 Acc: 0.9975 | Val Loss: 0.5827 Acc: 0.8941 | LR: 3.93e-05
Epoch 119/200 | Train Loss: 0.4316 Acc: 1.0000 | Val Loss: 0.5832 Acc: 0.8941 | LR: 3.85e-05
Epoch 120/200 | Train Loss: 0.4378 Acc: 0.9975 | Val Loss: 0.5825 Acc: 0.8941 | LR: 3.77e-05
Epoch 121/200 | Train Loss: 0.4322 Acc: 1.0000 | Val Loss: 0.5825 Acc: 0.8941 | LR: 3.69e-05
Epoch 122/200 | Train Loss: 0.4319 Acc: 0.9975 | Val Loss: 0.5834 Acc: 0.9176 | LR: 3.61e-05
Epoch 123/200 | Train Loss: 0.4340 Acc: 1.0000 | Val Loss: 0.5836 Acc: 0.9176 | LR: 3.53e-05
Epoch 124/200 | Train Loss: 0.4327 Acc: 1.0000 | Val Loss: 0.5846 Acc: 0.9059 | LR: 3.45e-05
Early stopping triggered.
Training complete in 8.5 minutes. Best Val Loss: 0.5823
Best model for fold saved to: new_results_Multimodal_DINO_Concat_KFold\best_model_fold_5.pth



Saved training history plot to: new_results_Multimodal_DINO_Concat_KFold\training_history_fold_5.png

=====

OVERALL PERFORMANCE (FOLD 5)

=====

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	0.95	0.97	19
Sarampion	1.00	0.97	0.98	30
Varicela	0.92	1.00	0.96	24
accuracy			0.98	81
macro avg	0.98	0.98	0.98	81
weighted avg	0.98	0.98	0.98	81

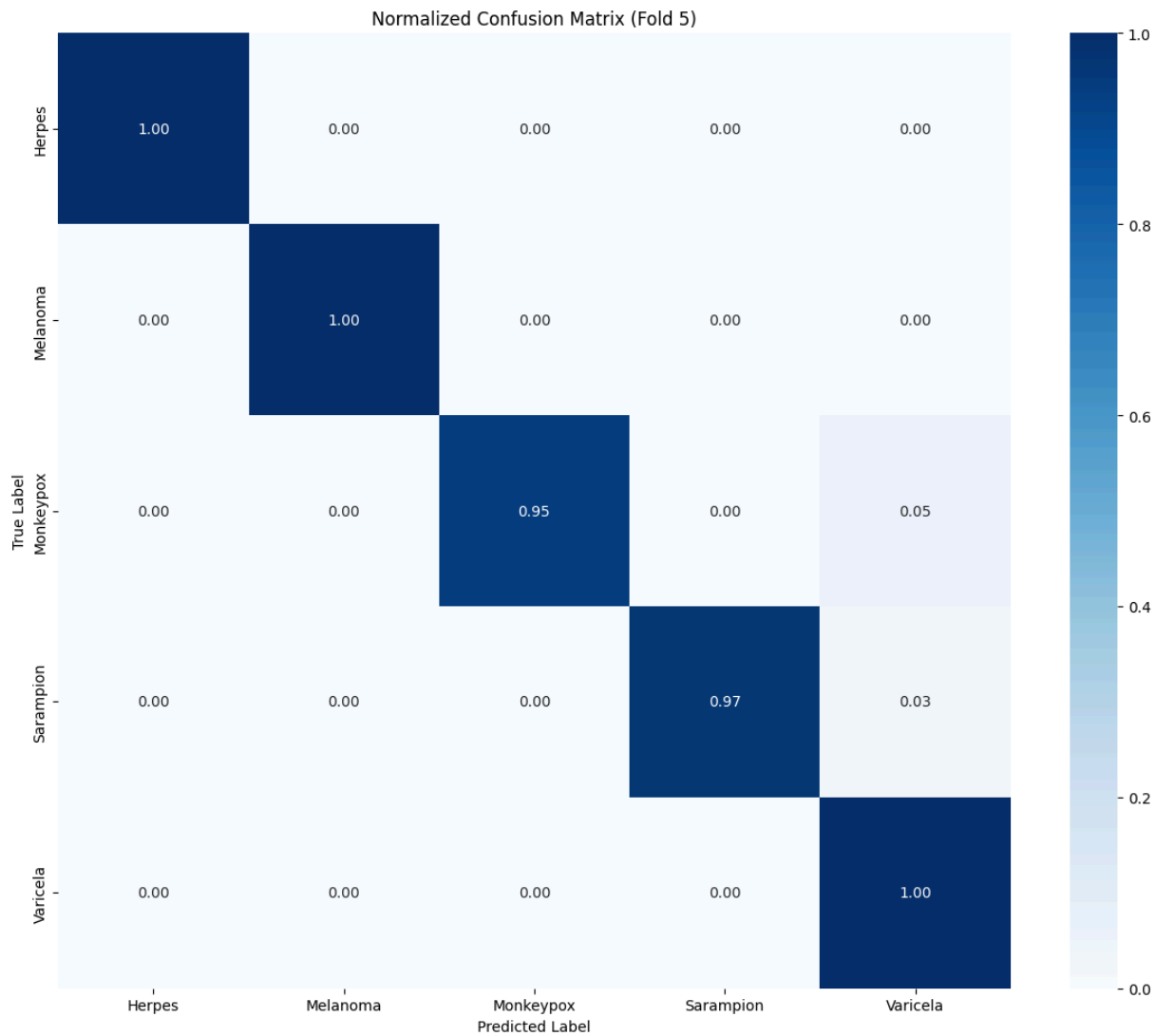
--- Overall Scalar Metrics ---

Accuracy: 0.9753

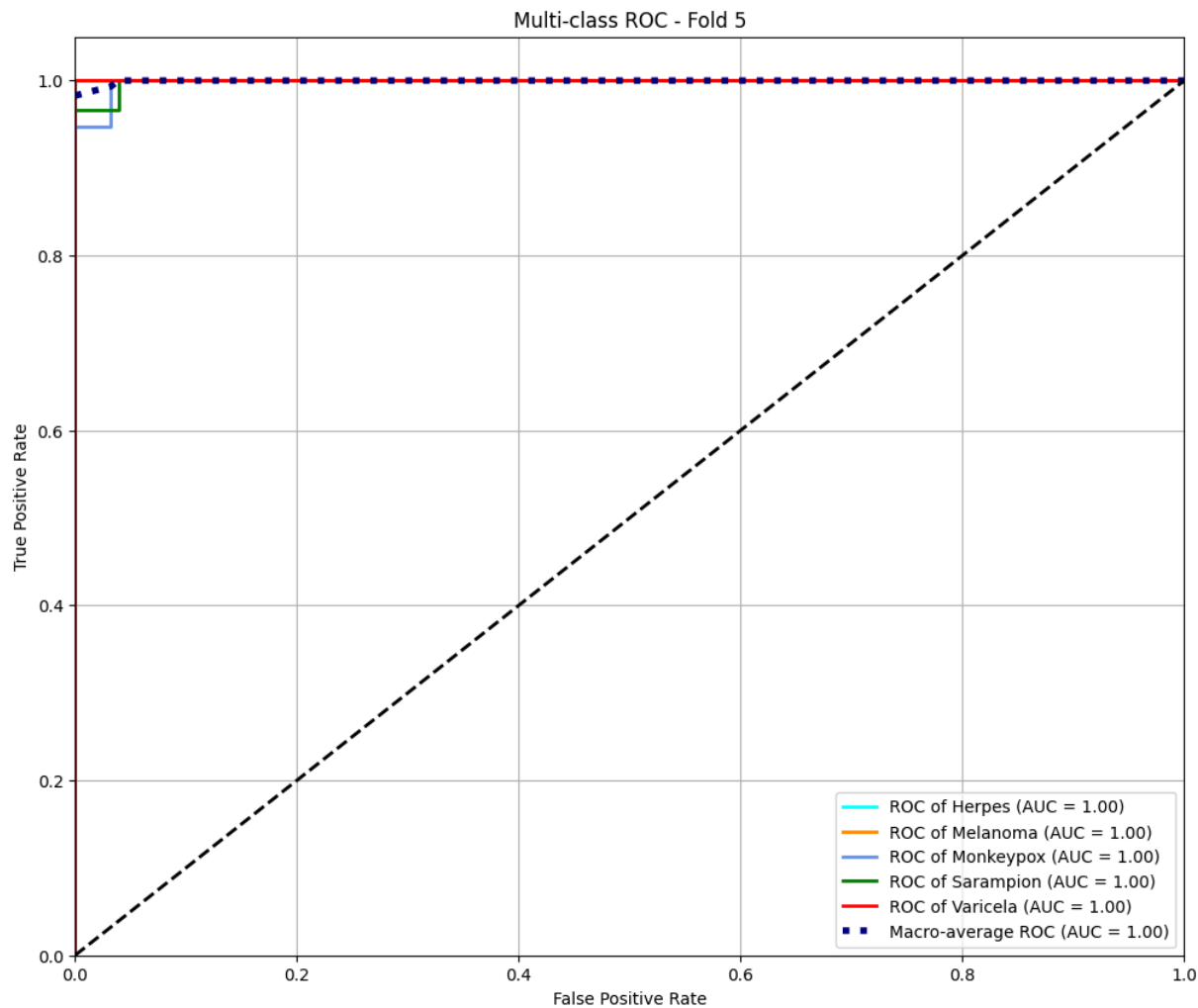
Macro ROC AUC: 0.9994

Cohen's Kappa: 0.9654

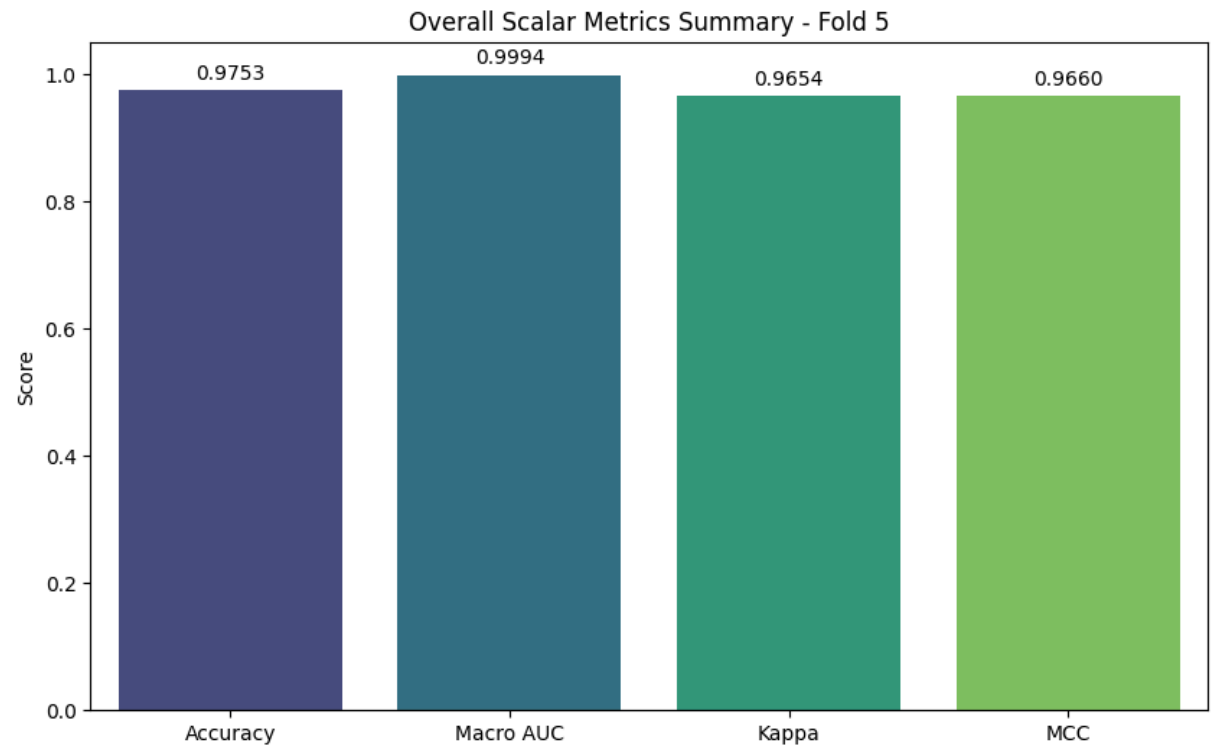
Matthews CorrCoef: 0.9660



Saved normalized confusion matrix to: new_results_Multimodal_DINO_Concat_KFold\confu
sion_matrix_fold_5.png



Saved ROC curves plot to: new_results_Multimodal_DINO_Concat_KFold\roc_curves_fold_5.png



Saved scalar metrics plot to: new_results_Multimodal_DINO_Concat_KFold\scalar_metrics_fold_5.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	2
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	1.00	1.00	15
Varicela	1.00	1.00	1.00	7
accuracy			1.00	28
macro avg	0.80	0.80	0.80	28
weighted avg	1.00	1.00	1.00	28

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	4
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	0.93	0.97	15
Sarampion	1.00	0.93	0.97	15
Varicela	0.89	1.00	0.94	17
accuracy			0.96	53
macro avg	0.98	0.97	0.98	53
weighted avg	0.97	0.96	0.96	53

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	1.00	1.00	1
Varicela	1.00	1.00	1.00	5
accuracy			1.00	6
macro avg	0.40	0.40	0.40	6
weighted avg	1.00	1.00	1.00	6

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	0.95	0.97	19
Sarampion	1.00	0.97	0.98	29
Varicela	0.90	1.00	0.95	19
accuracy			0.97	75
macro avg	0.98	0.98	0.98	75
weighted avg	0.98	0.97	0.97	75

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	1.00	0.75	0.86	4
Varicela	0.75	1.00	0.86	3
accuracy			0.89	9
macro avg	0.75	0.75	0.74	9
weighted avg	0.92	0.89	0.89	9

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	6
macro avg	0.60	0.60	0.60	6
weighted avg	1.00	1.00	1.00	6

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	4
Varicela	1.00	1.00	1.00	3
accuracy			1.00	10

macro avg	0.60	0.60	0.60	10
weighted avg	1.00	1.00	1.00	10

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	7
Varicela	0.00	0.00	0.00	0
accuracy			1.00	9
macro avg	0.40	0.40	0.40	9
weighted avg	1.00	1.00	1.00	9

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	2
macro avg	0.40	0.40	0.40	2
weighted avg	1.00	1.00	1.00	2

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	4
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	0.88	0.93	8
Sarampion	1.00	1.00	1.00	13
Varicela	0.95	1.00	0.97	18
accuracy			0.98	45
macro avg	0.99	0.97	0.98	45
weighted avg	0.98	0.98	0.98	45

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.8000 EOD: 0.8000
Worst pair: 18-59|male|black vs 18-59|male|indian
=====

=====
FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.9615 EOD: 0.0000
Worst pair: None vs None
Groups: ['18-59|male|white']
=====
```

```
#####  
####  
### FOLD 6 ###  
#####  
####  
Trainable params: 0.56M
```

START TRAINING (DINOv3 + Concat) for 200 epochs...

Epoch 1/200		Train Loss: 1.6944	Acc: 0.1950		Val Loss: 1.6010	Acc: 0.2941		LR: 2.00e-05
Epoch 2/200		Train Loss: 1.5920	Acc: 0.2925		Val Loss: 1.5756	Acc: 0.3059		LR: 3.00e-05
Epoch 3/200		Train Loss: 1.5138	Acc: 0.3825		Val Loss: 1.5253	Acc: 0.5059		LR: 4.00e-05
Epoch 4/200		Train Loss: 1.3984	Acc: 0.5000		Val Loss: 1.4498	Acc: 0.6353		LR: 5.00e-05
Epoch 5/200		Train Loss: 1.2782	Acc: 0.6025		Val Loss: 1.3509	Acc: 0.6471		LR: 6.00e-05
Epoch 6/200		Train Loss: 1.1547	Acc: 0.6625		Val Loss: 1.2378	Acc: 0.7059		LR: 7.00e-05
Epoch 7/200		Train Loss: 1.0774	Acc: 0.7450		Val Loss: 1.1219	Acc: 0.7059		LR: 8.00e-05
Epoch 8/200		Train Loss: 0.9806	Acc: 0.7575		Val Loss: 1.0245	Acc: 0.7765		LR: 9.00e-05
Epoch 9/200		Train Loss: 0.9063	Acc: 0.7900		Val Loss: 0.9406	Acc: 0.7882		LR: 1.00e-04
Epoch 10/200		Train Loss: 0.8268	Acc: 0.8425		Val Loss: 0.8782	Acc: 0.7882		LR: 1.00e-04
Epoch 11/200		Train Loss: 0.7919	Acc: 0.8300		Val Loss: 0.8251	Acc: 0.8000		LR: 1.00e-04
Epoch 12/200		Train Loss: 0.7486	Acc: 0.8600		Val Loss: 0.7840	Acc: 0.8118		LR: 1.00e-04
Epoch 13/200		Train Loss: 0.7138	Acc: 0.8800		Val Loss: 0.7526	Acc: 0.8353		LR: 9.99e-05
Epoch 14/200		Train Loss: 0.6707	Acc: 0.9150		Val Loss: 0.7271	Acc: 0.8353		LR: 9.99e-05
Epoch 15/200		Train Loss: 0.6631	Acc: 0.8950		Val Loss: 0.7040	Acc: 0.8706		LR: 9.98e-05
Epoch 16/200		Train Loss: 0.6298	Acc: 0.9150		Val Loss: 0.6911	Acc: 0.8824		LR: 9.98e-05
Epoch 17/200		Train Loss: 0.6107	Acc: 0.9275		Val Loss: 0.6810	Acc: 0.8824		LR: 9.97e-05
Epoch 18/200		Train Loss: 0.6063	Acc: 0.9500		Val Loss: 0.6715	Acc: 0.8824		LR: 9.96e-05
Epoch 19/200		Train Loss: 0.5877	Acc: 0.9400		Val Loss: 0.6549	Acc: 0.8824		LR: 9.94e-05
Epoch 20/200		Train Loss: 0.5894	Acc: 0.9400		Val Loss: 0.6461	Acc: 0.8941		LR: 9.93e-05
Epoch 21/200		Train Loss: 0.5618	Acc: 0.9625		Val Loss: 0.6395	Acc: 0.8941		LR: 9.92e-05
Epoch 22/200		Train Loss: 0.5561	Acc: 0.9525		Val Loss: 0.6314	Acc: 0.8941		LR: 9.90e-05
Epoch 23/200		Train Loss: 0.5522	Acc: 0.9675		Val Loss: 0.6235	Acc: 0.9059		LR: 9.88e-05
Epoch 24/200		Train Loss: 0.5475	Acc: 0.9625		Val Loss: 0.6218	Acc: 0.8941		LR:

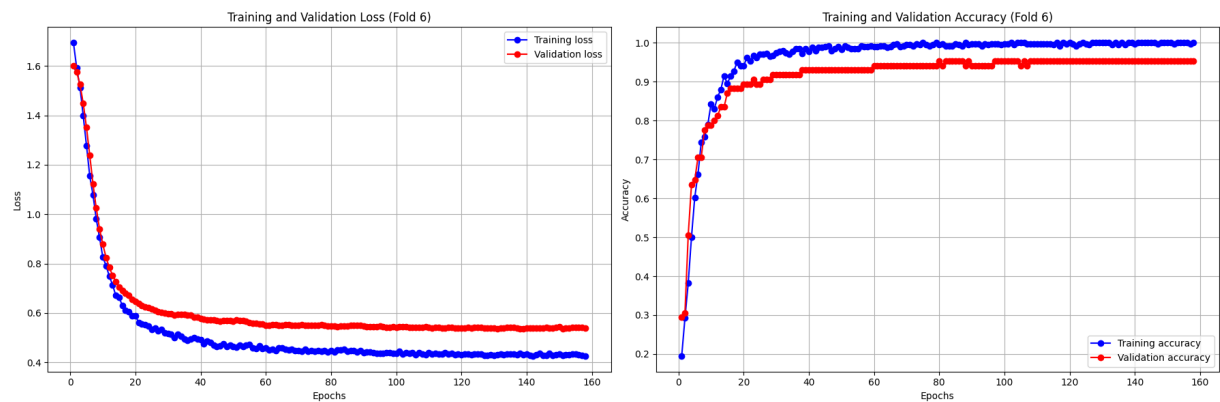
9.87e-05
Epoch 25/200 | Train Loss: 0.5332 Acc: 0.9700 | Val Loss: 0.6163 Acc: 0.8941 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.5392 Acc: 0.9700 | Val Loss: 0.6101 Acc: 0.9059 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.5278 Acc: 0.9725 | Val Loss: 0.6064 Acc: 0.9059 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.5328 Acc: 0.9650 | Val Loss: 0.6021 Acc: 0.9059 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.5184 Acc: 0.9675 | Val Loss: 0.5996 Acc: 0.9176 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.5159 Acc: 0.9750 | Val Loss: 0.5971 Acc: 0.9176 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.5137 Acc: 0.9775 | Val Loss: 0.5971 Acc: 0.9176 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.4994 Acc: 0.9800 | Val Loss: 0.5924 Acc: 0.9176 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.5131 Acc: 0.9750 | Val Loss: 0.5931 Acc: 0.9176 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.5053 Acc: 0.9700 | Val Loss: 0.5943 Acc: 0.9176 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.4946 Acc: 0.9775 | Val Loss: 0.5928 Acc: 0.9176 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.4883 Acc: 0.9850 | Val Loss: 0.5923 Acc: 0.9176 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.4949 Acc: 0.9850 | Val Loss: 0.5904 Acc: 0.9176 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.5014 Acc: 0.9725 | Val Loss: 0.5839 Acc: 0.9294 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.4952 Acc: 0.9850 | Val Loss: 0.5819 Acc: 0.9294 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.4908 Acc: 0.9775 | Val Loss: 0.5780 Acc: 0.9294 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.4763 Acc: 0.9875 | Val Loss: 0.5759 Acc: 0.9294 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.4850 Acc: 0.9800 | Val Loss: 0.5730 Acc: 0.9294 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.4808 Acc: 0.9875 | Val Loss: 0.5716 Acc: 0.9294 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.4697 Acc: 0.9875 | Val Loss: 0.5708 Acc: 0.9294 | LR: 9.23e-05
Epoch 45/200 | Train Loss: 0.4643 Acc: 0.9900 | Val Loss: 0.5684 Acc: 0.9294 | LR: 9.19e-05
Epoch 46/200 | Train Loss: 0.4664 Acc: 0.9925 | Val Loss: 0.5669 Acc: 0.9294 | LR: 9.14e-05
Epoch 47/200 | Train Loss: 0.4742 Acc: 0.9800 | Val Loss: 0.5701 Acc: 0.9294 | LR: 9.09e-05
Epoch 48/200 | Train Loss: 0.4655 Acc: 0.9850 | Val Loss: 0.5692 Acc: 0.9294 | LR: 9.05e-05
Epoch 49/200 | Train Loss: 0.4694 Acc: 0.9900 | Val Loss: 0.5699 Acc: 0.9294 | LR: 9.00e-05
Epoch 50/200 | Train Loss: 0.4637 Acc: 0.9825 | Val Loss: 0.5675 Acc: 0.9294 | LR: 8.95e-05
Epoch 51/200 | Train Loss: 0.4621 Acc: 0.9925 | Val Loss: 0.5711 Acc: 0.9294 | LR: 8.89e-05
Epoch 52/200 | Train Loss: 0.4706 Acc: 0.9875 | Val Loss: 0.5706 Acc: 0.9294 | LR:

8.84e-05
Epoch 53/200 | Train Loss: 0.4640 Acc: 0.9850 | Val Loss: 0.5693 Acc: 0.9294 | LR: 8.79e-05
Epoch 54/200 | Train Loss: 0.4694 Acc: 0.9850 | Val Loss: 0.5672 Acc: 0.9294 | LR: 8.73e-05
Epoch 55/200 | Train Loss: 0.4730 Acc: 0.9850 | Val Loss: 0.5614 Acc: 0.9294 | LR: 8.68e-05
Epoch 56/200 | Train Loss: 0.4593 Acc: 0.9925 | Val Loss: 0.5593 Acc: 0.9294 | LR: 8.62e-05
Epoch 57/200 | Train Loss: 0.4563 Acc: 0.9900 | Val Loss: 0.5577 Acc: 0.9294 | LR: 8.56e-05
Epoch 58/200 | Train Loss: 0.4678 Acc: 0.9900 | Val Loss: 0.5565 Acc: 0.9294 | LR: 8.51e-05
Epoch 59/200 | Train Loss: 0.4565 Acc: 0.9925 | Val Loss: 0.5548 Acc: 0.9294 | LR: 8.45e-05
Epoch 60/200 | Train Loss: 0.4581 Acc: 0.9900 | Val Loss: 0.5511 Acc: 0.9412 | LR: 8.39e-05
Epoch 61/200 | Train Loss: 0.4491 Acc: 0.9900 | Val Loss: 0.5506 Acc: 0.9412 | LR: 8.33e-05
Epoch 62/200 | Train Loss: 0.4531 Acc: 0.9925 | Val Loss: 0.5517 Acc: 0.9412 | LR: 8.26e-05
Epoch 63/200 | Train Loss: 0.4487 Acc: 0.9925 | Val Loss: 0.5532 Acc: 0.9412 | LR: 8.20e-05
Epoch 64/200 | Train Loss: 0.4598 Acc: 0.9875 | Val Loss: 0.5505 Acc: 0.9412 | LR: 8.14e-05
Epoch 65/200 | Train Loss: 0.4579 Acc: 0.9900 | Val Loss: 0.5490 Acc: 0.9412 | LR: 8.07e-05
Epoch 66/200 | Train Loss: 0.4518 Acc: 0.9950 | Val Loss: 0.5517 Acc: 0.9412 | LR: 8.01e-05
Epoch 67/200 | Train Loss: 0.4502 Acc: 0.9975 | Val Loss: 0.5519 Acc: 0.9412 | LR: 7.94e-05
Epoch 68/200 | Train Loss: 0.4540 Acc: 0.9900 | Val Loss: 0.5499 Acc: 0.9412 | LR: 7.87e-05
Epoch 69/200 | Train Loss: 0.4489 Acc: 0.9925 | Val Loss: 0.5486 Acc: 0.9412 | LR: 7.80e-05
Epoch 70/200 | Train Loss: 0.4487 Acc: 0.9950 | Val Loss: 0.5496 Acc: 0.9412 | LR: 7.73e-05
Epoch 71/200 | Train Loss: 0.4459 Acc: 0.9950 | Val Loss: 0.5520 Acc: 0.9412 | LR: 7.67e-05
Epoch 72/200 | Train Loss: 0.4529 Acc: 0.9925 | Val Loss: 0.5504 Acc: 0.9412 | LR: 7.59e-05
Epoch 73/200 | Train Loss: 0.4454 Acc: 0.9975 | Val Loss: 0.5495 Acc: 0.9412 | LR: 7.52e-05
Epoch 74/200 | Train Loss: 0.4489 Acc: 0.9950 | Val Loss: 0.5507 Acc: 0.9412 | LR: 7.45e-05
Epoch 75/200 | Train Loss: 0.4442 Acc: 1.0000 | Val Loss: 0.5490 Acc: 0.9412 | LR: 7.38e-05
Epoch 76/200 | Train Loss: 0.4489 Acc: 0.9950 | Val Loss: 0.5500 Acc: 0.9412 | LR: 7.31e-05
Epoch 77/200 | Train Loss: 0.4442 Acc: 0.9925 | Val Loss: 0.5522 Acc: 0.9412 | LR: 7.23e-05
Epoch 78/200 | Train Loss: 0.4477 Acc: 0.9950 | Val Loss: 0.5496 Acc: 0.9412 | LR: 7.16e-05
Epoch 79/200 | Train Loss: 0.4427 Acc: 1.0000 | Val Loss: 0.5472 Acc: 0.9412 | LR: 7.08e-05
Epoch 80/200 | Train Loss: 0.4465 Acc: 0.9950 | Val Loss: 0.5482 Acc: 0.9529 | LR:

7.01e-05
Epoch 81/200 | Train Loss: 0.4409 Acc: 0.9975 | Val Loss: 0.5472 Acc: 0.9412 | LR: 6.93e-05
Epoch 82/200 | Train Loss: 0.4500 Acc: 0.9925 | Val Loss: 0.5453 Acc: 0.9529 | LR: 6.86e-05
Epoch 83/200 | Train Loss: 0.4496 Acc: 0.9925 | Val Loss: 0.5468 Acc: 0.9529 | LR: 6.78e-05
Epoch 84/200 | Train Loss: 0.4524 Acc: 0.9925 | Val Loss: 0.5468 Acc: 0.9529 | LR: 6.70e-05
Epoch 85/200 | Train Loss: 0.4437 Acc: 0.9975 | Val Loss: 0.5472 Acc: 0.9529 | LR: 6.62e-05
Epoch 86/200 | Train Loss: 0.4488 Acc: 0.9950 | Val Loss: 0.5488 Acc: 0.9529 | LR: 6.55e-05
Epoch 87/200 | Train Loss: 0.4486 Acc: 0.9925 | Val Loss: 0.5487 Acc: 0.9529 | LR: 6.47e-05
Epoch 88/200 | Train Loss: 0.4422 Acc: 1.0000 | Val Loss: 0.5500 Acc: 0.9412 | LR: 6.39e-05
Epoch 89/200 | Train Loss: 0.4487 Acc: 0.9950 | Val Loss: 0.5489 Acc: 0.9529 | LR: 6.31e-05
Epoch 90/200 | Train Loss: 0.4401 Acc: 0.9975 | Val Loss: 0.5462 Acc: 0.9412 | LR: 6.23e-05
Epoch 91/200 | Train Loss: 0.4412 Acc: 0.9975 | Val Loss: 0.5454 Acc: 0.9412 | LR: 6.15e-05
Epoch 92/200 | Train Loss: 0.4411 Acc: 0.9975 | Val Loss: 0.5445 Acc: 0.9412 | LR: 6.07e-05
Epoch 93/200 | Train Loss: 0.4399 Acc: 0.9925 | Val Loss: 0.5454 Acc: 0.9412 | LR: 5.99e-05
Epoch 94/200 | Train Loss: 0.4364 Acc: 0.9975 | Val Loss: 0.5451 Acc: 0.9412 | LR: 5.90e-05
Epoch 95/200 | Train Loss: 0.4378 Acc: 0.9950 | Val Loss: 0.5459 Acc: 0.9412 | LR: 5.82e-05
Epoch 96/200 | Train Loss: 0.4367 Acc: 0.9975 | Val Loss: 0.5434 Acc: 0.9412 | LR: 5.74e-05
Epoch 97/200 | Train Loss: 0.4405 Acc: 0.9975 | Val Loss: 0.5426 Acc: 0.9529 | LR: 5.66e-05
Epoch 98/200 | Train Loss: 0.4390 Acc: 0.9975 | Val Loss: 0.5425 Acc: 0.9529 | LR: 5.58e-05
Epoch 99/200 | Train Loss: 0.4373 Acc: 0.9950 | Val Loss: 0.5444 Acc: 0.9529 | LR: 5.50e-05
Epoch 100/200 | Train Loss: 0.4362 Acc: 0.9975 | Val Loss: 0.5415 Acc: 0.9529 | LR: 5.41e-05
Epoch 101/200 | Train Loss: 0.4435 Acc: 0.9975 | Val Loss: 0.5446 Acc: 0.9529 | LR: 5.33e-05
Epoch 102/200 | Train Loss: 0.4350 Acc: 1.0000 | Val Loss: 0.5456 Acc: 0.9529 | LR: 5.25e-05
Epoch 103/200 | Train Loss: 0.4398 Acc: 0.9950 | Val Loss: 0.5457 Acc: 0.9529 | LR: 5.17e-05
Epoch 104/200 | Train Loss: 0.4355 Acc: 1.0000 | Val Loss: 0.5427 Acc: 0.9529 | LR: 5.08e-05
Epoch 105/200 | Train Loss: 0.4381 Acc: 1.0000 | Val Loss: 0.5428 Acc: 0.9412 | LR: 5.00e-05
Epoch 106/200 | Train Loss: 0.4316 Acc: 1.0000 | Val Loss: 0.5404 Acc: 0.9529 | LR: 4.92e-05
Epoch 107/200 | Train Loss: 0.4403 Acc: 0.9975 | Val Loss: 0.5425 Acc: 0.9412 | LR: 4.83e-05
Epoch 108/200 | Train Loss: 0.4334 Acc: 0.9975 | Val Loss: 0.5418 Acc: 0.9529 | LR:

4.75e-05
Epoch 109/200 | Train Loss: 0.4312 Acc: 0.9975 | Val Loss: 0.5442 Acc: 0.9529 | LR: 4.67e-05
Epoch 110/200 | Train Loss: 0.4390 Acc: 0.9975 | Val Loss: 0.5409 Acc: 0.9529 | LR: 4.59e-05
Epoch 111/200 | Train Loss: 0.4332 Acc: 0.9975 | Val Loss: 0.5402 Acc: 0.9529 | LR: 4.50e-05
Epoch 112/200 | Train Loss: 0.4367 Acc: 0.9975 | Val Loss: 0.5407 Acc: 0.9529 | LR: 4.42e-05
Epoch 113/200 | Train Loss: 0.4337 Acc: 0.9975 | Val Loss: 0.5407 Acc: 0.9529 | LR: 4.34e-05
Epoch 114/200 | Train Loss: 0.4331 Acc: 0.9975 | Val Loss: 0.5387 Acc: 0.9529 | LR: 4.26e-05
Epoch 115/200 | Train Loss: 0.4380 Acc: 0.9950 | Val Loss: 0.5387 Acc: 0.9529 | LR: 4.18e-05
Epoch 116/200 | Train Loss: 0.4340 Acc: 1.0000 | Val Loss: 0.5402 Acc: 0.9529 | LR: 4.10e-05
Epoch 117/200 | Train Loss: 0.4364 Acc: 0.9925 | Val Loss: 0.5412 Acc: 0.9529 | LR: 4.01e-05
Epoch 118/200 | Train Loss: 0.4320 Acc: 1.0000 | Val Loss: 0.5394 Acc: 0.9529 | LR: 3.93e-05
Epoch 119/200 | Train Loss: 0.4327 Acc: 0.9975 | Val Loss: 0.5400 Acc: 0.9529 | LR: 3.85e-05
Epoch 120/200 | Train Loss: 0.4312 Acc: 1.0000 | Val Loss: 0.5394 Acc: 0.9529 | LR: 3.77e-05
Epoch 121/200 | Train Loss: 0.4334 Acc: 0.9975 | Val Loss: 0.5378 Acc: 0.9529 | LR: 3.69e-05
Epoch 122/200 | Train Loss: 0.4350 Acc: 0.9925 | Val Loss: 0.5402 Acc: 0.9529 | LR: 3.61e-05
Epoch 123/200 | Train Loss: 0.4317 Acc: 0.9975 | Val Loss: 0.5394 Acc: 0.9529 | LR: 3.53e-05
Epoch 124/200 | Train Loss: 0.4333 Acc: 1.0000 | Val Loss: 0.5424 Acc: 0.9529 | LR: 3.45e-05
Epoch 125/200 | Train Loss: 0.4324 Acc: 0.9975 | Val Loss: 0.5409 Acc: 0.9529 | LR: 3.38e-05
Epoch 126/200 | Train Loss: 0.4344 Acc: 0.9950 | Val Loss: 0.5412 Acc: 0.9529 | LR: 3.30e-05
Epoch 127/200 | Train Loss: 0.4283 Acc: 1.0000 | Val Loss: 0.5395 Acc: 0.9529 | LR: 3.22e-05
Epoch 128/200 | Train Loss: 0.4268 Acc: 1.0000 | Val Loss: 0.5375 Acc: 0.9529 | LR: 3.14e-05
Epoch 129/200 | Train Loss: 0.4318 Acc: 1.0000 | Val Loss: 0.5375 Acc: 0.9529 | LR: 3.07e-05
Epoch 130/200 | Train Loss: 0.4293 Acc: 1.0000 | Val Loss: 0.5381 Acc: 0.9529 | LR: 2.99e-05
Epoch 131/200 | Train Loss: 0.4300 Acc: 1.0000 | Val Loss: 0.5374 Acc: 0.9529 | LR: 2.92e-05
Epoch 132/200 | Train Loss: 0.4332 Acc: 1.0000 | Val Loss: 0.5386 Acc: 0.9529 | LR: 2.84e-05
Epoch 133/200 | Train Loss: 0.4317 Acc: 0.9950 | Val Loss: 0.5389 Acc: 0.9529 | LR: 2.77e-05
Epoch 134/200 | Train Loss: 0.4334 Acc: 1.0000 | Val Loss: 0.5403 Acc: 0.9529 | LR: 2.69e-05
Epoch 135/200 | Train Loss: 0.4336 Acc: 0.9950 | Val Loss: 0.5406 Acc: 0.9529 | LR: 2.62e-05
Epoch 136/200 | Train Loss: 0.4333 Acc: 1.0000 | Val Loss: 0.5409 Acc: 0.9529 | LR:

2.55e-05
Epoch 137/200 | Train Loss: 0.4306 Acc: 0.9950 | Val Loss: 0.5392 Acc: 0.9529 | LR: 2.48e-05
Epoch 138/200 | Train Loss: 0.4374 Acc: 1.0000 | Val Loss: 0.5351 Acc: 0.9529 | LR: 2.41e-05
Epoch 139/200 | Train Loss: 0.4297 Acc: 1.0000 | Val Loss: 0.5358 Acc: 0.9529 | LR: 2.33e-05
Epoch 140/200 | Train Loss: 0.4334 Acc: 0.9975 | Val Loss: 0.5379 Acc: 0.9529 | LR: 2.27e-05
Epoch 141/200 | Train Loss: 0.4288 Acc: 1.0000 | Val Loss: 0.5390 Acc: 0.9529 | LR: 2.20e-05
Epoch 142/200 | Train Loss: 0.4264 Acc: 1.0000 | Val Loss: 0.5388 Acc: 0.9529 | LR: 2.13e-05
Epoch 143/200 | Train Loss: 0.4320 Acc: 1.0000 | Val Loss: 0.5388 Acc: 0.9529 | LR: 2.06e-05
Epoch 144/200 | Train Loss: 0.4355 Acc: 1.0000 | Val Loss: 0.5399 Acc: 0.9529 | LR: 1.99e-05
Epoch 145/200 | Train Loss: 0.4286 Acc: 1.0000 | Val Loss: 0.5395 Acc: 0.9529 | LR: 1.93e-05
Epoch 146/200 | Train Loss: 0.4280 Acc: 1.0000 | Val Loss: 0.5409 Acc: 0.9529 | LR: 1.86e-05
Epoch 147/200 | Train Loss: 0.4363 Acc: 0.9975 | Val Loss: 0.5389 Acc: 0.9529 | LR: 1.80e-05
Epoch 148/200 | Train Loss: 0.4282 Acc: 1.0000 | Val Loss: 0.5394 Acc: 0.9529 | LR: 1.74e-05
Epoch 149/200 | Train Loss: 0.4315 Acc: 1.0000 | Val Loss: 0.5427 Acc: 0.9529 | LR: 1.67e-05
Epoch 150/200 | Train Loss: 0.4331 Acc: 1.0000 | Val Loss: 0.5453 Acc: 0.9529 | LR: 1.61e-05
Epoch 151/200 | Train Loss: 0.4288 Acc: 1.0000 | Val Loss: 0.5369 Acc: 0.9529 | LR: 1.55e-05
Epoch 152/200 | Train Loss: 0.4317 Acc: 0.9975 | Val Loss: 0.5388 Acc: 0.9529 | LR: 1.49e-05
Epoch 153/200 | Train Loss: 0.4314 Acc: 1.0000 | Val Loss: 0.5401 Acc: 0.9529 | LR: 1.44e-05
Epoch 154/200 | Train Loss: 0.4337 Acc: 1.0000 | Val Loss: 0.5414 Acc: 0.9529 | LR: 1.38e-05
Epoch 155/200 | Train Loss: 0.4336 Acc: 1.0000 | Val Loss: 0.5410 Acc: 0.9529 | LR: 1.32e-05
Epoch 156/200 | Train Loss: 0.4314 Acc: 1.0000 | Val Loss: 0.5413 Acc: 0.9529 | LR: 1.27e-05
Epoch 157/200 | Train Loss: 0.4290 Acc: 0.9975 | Val Loss: 0.5405 Acc: 0.9529 | LR: 1.21e-05
Epoch 158/200 | Train Loss: 0.4260 Acc: 1.0000 | Val Loss: 0.5390 Acc: 0.9529 | LR: 1.16e-05
Early stopping triggered.
Training complete in 11.0 minutes. Best Val Loss: 0.5351
Best model for fold saved to: new_results_Multimodal_DINO_Concat_KFold\best_model_fold_6.pth



Saved training history plot to: new_results_Multimodal_DINO_Concat_KFold\training_history_fold_6.png

=====

OVERALL PERFORMANCE (FOLD 6)

=====

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	1.00	1.00	18
Sarampion	0.93	0.93	0.93	29
Varicela	0.92	0.92	0.92	25
accuracy			0.95	80
macro avg	0.97	0.97	0.97	80
weighted avg	0.95	0.95	0.95	80

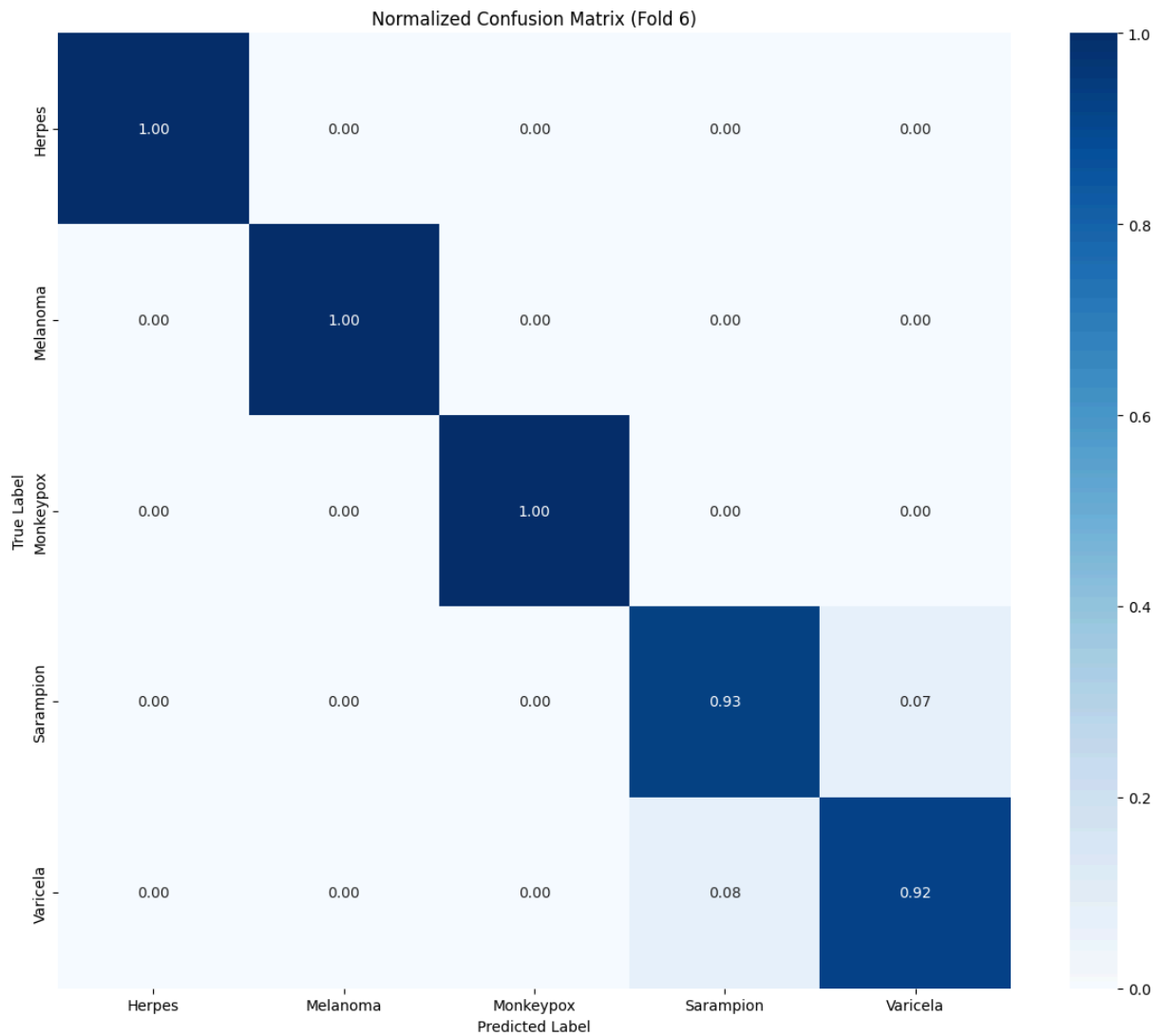
--- Overall Scalar Metrics ---

Accuracy: 0.9500

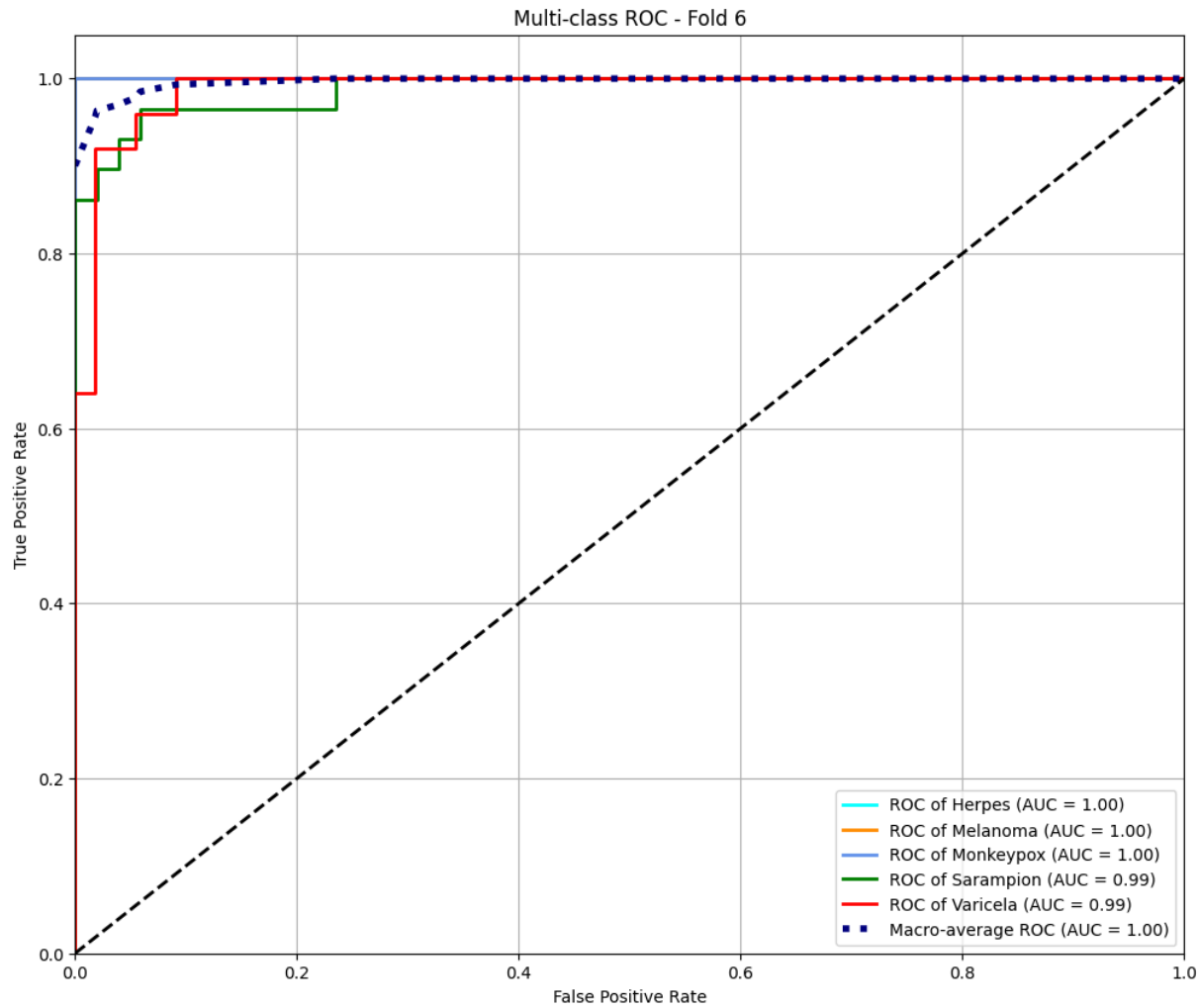
Macro ROC AUC: 0.9954

Cohen's Kappa: 0.9300

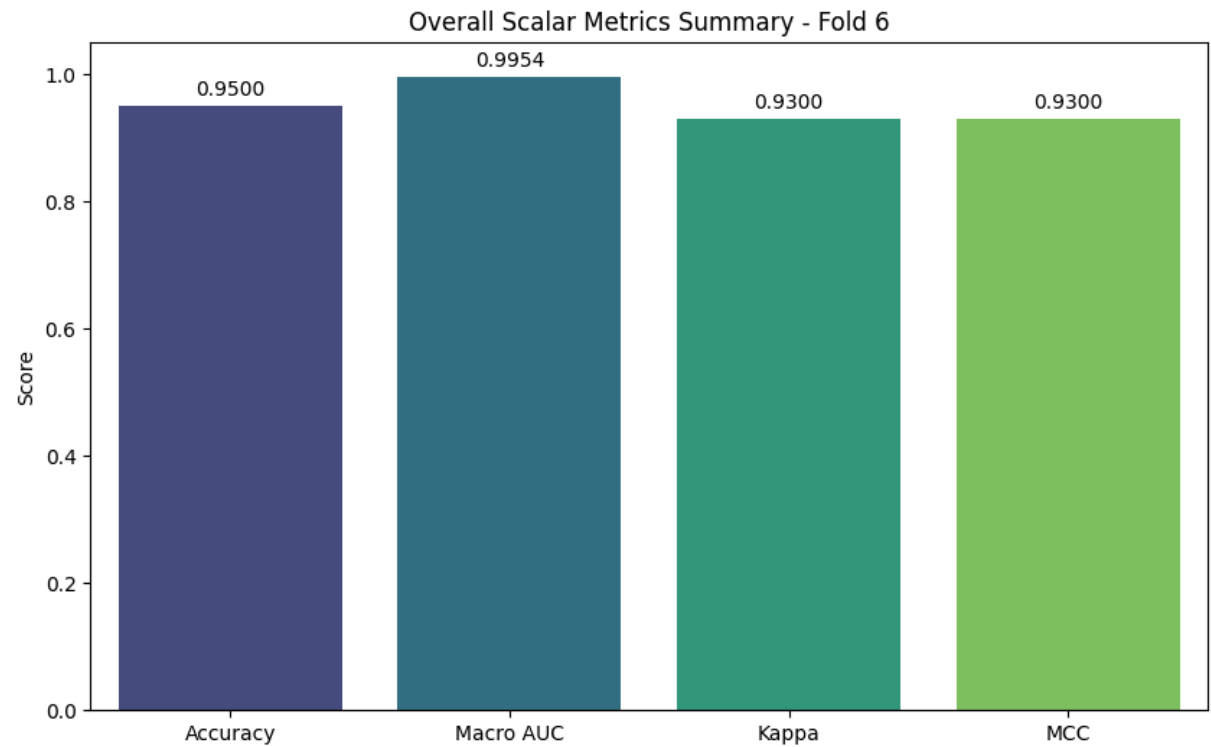
Matthews CorrCoef: 0.9300



Saved normalized confusion matrix to: new_results_Multimodal_DINO_Concat_KFold\confusion_matrix_fold_6.png



Saved ROC curves plot to: new_results_Multimodal_DINO_Concat_KFold\roc_curves_fold_6.png



Saved scalar metrics plot to: new_results_Multimodal_DINO_Concat_KFold\scalar_metrics_fold_6.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.91	1.00	0.95	10
Varicela	1.00	0.93	0.96	14
accuracy			0.96	27
macro avg	0.78	0.79	0.78	27
weighted avg	0.97	0.96	0.96	27

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	5
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	1.00	1.00	16
Sarampion	0.94	0.89	0.92	19
Varicela	0.83	0.91	0.87	11
accuracy			0.94	53
macro avg	0.96	0.96	0.96	53
weighted avg	0.95	0.94	0.94	53

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.75	0.75	0.75	4
Varicela	0.80	0.80	0.80	5
accuracy			0.83	12
macro avg	0.71	0.71	0.71	12
weighted avg	0.83	0.83	0.83	12

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	4
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	1.00	1.00	16
Sarampion	0.96	0.96	0.96	25
Varicela	0.95	0.95	0.95	20
accuracy			0.97	67
macro avg	0.98	0.98	0.98	67
weighted avg	0.97	0.97	0.97	67

--- Subgroup: 60+ ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.50	1.00	0.67	1
Varicela	1.00	0.80	0.89	5
accuracy			0.86	7
macro avg	0.50	0.56	0.51	7
weighted avg	0.93	0.86	0.87	7

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	4

macro avg	0.40	0.40	0.40	4
weighted avg	1.00	1.00	1.00	4

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	7
Varicela	1.00	1.00	1.00	1
accuracy			1.00	10
macro avg	0.60	0.60	0.60	10
weighted avg	1.00	1.00	1.00	10

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	1.00	1.00	3
Varicela	0.00	0.00	0.00	0
accuracy			1.00	3
macro avg	0.20	0.20	0.20	3
weighted avg	1.00	1.00	1.00	3

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	1.00	1.00	11
Sarampion	0.94	0.88	0.91	17
Varicela	0.90	0.95	0.92	19
accuracy			0.95	55

macro avg	0.97	0.97	0.97	55
weighted avg	0.95	0.95	0.95	55

=====

FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.5000 EOD: 0.8400

Worst pair: 18-59|female|latino hispanic vs 18-59|male|white

=====

=====

FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.9655 EOD: 0.4400

Worst pair: 18-59|female|white vs 18-59|male|white

Groups: ['18-59|female|white', '18-59|male|white']

=====

#####

####

FOLD 7

#####

####

Trainable params: 0.56M

START TRAINING (DINOv3 + Concat) for 200 epochs...

Epoch 1/200	Train Loss: 1.5947	Acc: 0.3275	Val Loss: 1.6001	Acc: 0.2471	LR: 2.00e-05
Epoch 2/200	Train Loss: 1.5371	Acc: 0.3975	Val Loss: 1.5700	Acc: 0.3882	LR: 3.00e-05
Epoch 3/200	Train Loss: 1.4756	Acc: 0.4250	Val Loss: 1.5190	Acc: 0.5176	LR: 4.00e-05
Epoch 4/200	Train Loss: 1.3859	Acc: 0.4600	Val Loss: 1.4431	Acc: 0.5765	LR: 5.00e-05
Epoch 5/200	Train Loss: 1.3099	Acc: 0.5250	Val Loss: 1.3485	Acc: 0.6235	LR: 6.00e-05
Epoch 6/200	Train Loss: 1.1614	Acc: 0.6375	Val Loss: 1.2387	Acc: 0.6588	LR: 7.00e-05
Epoch 7/200	Train Loss: 1.0695	Acc: 0.6925	Val Loss: 1.1248	Acc: 0.6824	LR: 8.00e-05
Epoch 8/200	Train Loss: 0.9830	Acc: 0.7425	Val Loss: 1.0195	Acc: 0.7882	LR: 9.00e-05
Epoch 9/200	Train Loss: 0.9098	Acc: 0.7825	Val Loss: 0.9291	Acc: 0.8118	LR: 1.00e-04
Epoch 10/200	Train Loss: 0.8341	Acc: 0.8150	Val Loss: 0.8539	Acc: 0.8235	LR: 1.00e-04
Epoch 11/200	Train Loss: 0.7714	Acc: 0.8550	Val Loss: 0.7949	Acc: 0.8471	LR: 1.00e-04
Epoch 12/200	Train Loss: 0.7372	Acc: 0.8675	Val Loss: 0.7533	Acc: 0.8706	LR: 1.00e-04
Epoch 13/200	Train Loss: 0.7143	Acc: 0.8800	Val Loss: 0.7223	Acc: 0.8824	LR: 9.99e-05
Epoch 14/200	Train Loss: 0.6578	Acc: 0.9125	Val Loss: 0.6954	Acc: 0.8824	LR: 9.99e-05
Epoch 15/200	Train Loss: 0.6516	Acc: 0.9150	Val Loss: 0.6772	Acc: 0.9059	LR: 9.98e-05
Epoch 16/200	Train Loss: 0.6281	Acc: 0.9200	Val Loss: 0.6637	Acc: 0.9059	LR: 9.98e-05
Epoch 17/200	Train Loss: 0.6150	Acc: 0.9275	Val Loss: 0.6530	Acc: 0.9059	LR:

9.97e-05
Epoch 18/200 | Train Loss: 0.6012 Acc: 0.9350 | Val Loss: 0.6458 Acc: 0.8941 | LR: 9.96e-05
Epoch 19/200 | Train Loss: 0.6051 Acc: 0.9250 | Val Loss: 0.6358 Acc: 0.9176 | LR: 9.94e-05
Epoch 20/200 | Train Loss: 0.5562 Acc: 0.9575 | Val Loss: 0.6323 Acc: 0.9176 | LR: 9.93e-05
Epoch 21/200 | Train Loss: 0.5738 Acc: 0.9400 | Val Loss: 0.6265 Acc: 0.9176 | LR: 9.92e-05
Epoch 22/200 | Train Loss: 0.5736 Acc: 0.9350 | Val Loss: 0.6214 Acc: 0.9176 | LR: 9.90e-05
Epoch 23/200 | Train Loss: 0.5611 Acc: 0.9450 | Val Loss: 0.6120 Acc: 0.9294 | LR: 9.88e-05
Epoch 24/200 | Train Loss: 0.5413 Acc: 0.9650 | Val Loss: 0.6093 Acc: 0.9176 | LR: 9.87e-05
Epoch 25/200 | Train Loss: 0.5414 Acc: 0.9625 | Val Loss: 0.6081 Acc: 0.9176 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.5214 Acc: 0.9700 | Val Loss: 0.6056 Acc: 0.9176 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.5355 Acc: 0.9550 | Val Loss: 0.6034 Acc: 0.9176 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.5235 Acc: 0.9700 | Val Loss: 0.5968 Acc: 0.9294 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.5348 Acc: 0.9700 | Val Loss: 0.5905 Acc: 0.9294 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.5142 Acc: 0.9625 | Val Loss: 0.5875 Acc: 0.9412 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.5047 Acc: 0.9775 | Val Loss: 0.5843 Acc: 0.9294 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.5056 Acc: 0.9775 | Val Loss: 0.5805 Acc: 0.9412 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.5067 Acc: 0.9700 | Val Loss: 0.5757 Acc: 0.9412 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.4941 Acc: 0.9825 | Val Loss: 0.5768 Acc: 0.9176 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.4971 Acc: 0.9725 | Val Loss: 0.5725 Acc: 0.9176 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.5028 Acc: 0.9800 | Val Loss: 0.5717 Acc: 0.9294 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.4979 Acc: 0.9775 | Val Loss: 0.5703 Acc: 0.9176 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.4933 Acc: 0.9775 | Val Loss: 0.5686 Acc: 0.9294 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.4844 Acc: 0.9825 | Val Loss: 0.5723 Acc: 0.9294 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.4842 Acc: 0.9825 | Val Loss: 0.5722 Acc: 0.9294 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.4747 Acc: 0.9825 | Val Loss: 0.5714 Acc: 0.9294 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.4801 Acc: 0.9875 | Val Loss: 0.5670 Acc: 0.9294 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.4814 Acc: 0.9725 | Val Loss: 0.5652 Acc: 0.9294 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.4739 Acc: 0.9775 | Val Loss: 0.5641 Acc: 0.9294 | LR: 9.23e-05
Epoch 45/200 | Train Loss: 0.4818 Acc: 0.9725 | Val Loss: 0.5655 Acc: 0.9294 | LR:

9.19e-05
Epoch 46/200 | Train Loss: 0.4777 Acc: 0.9900 | Val Loss: 0.5656 Acc: 0.9294 | LR: 9.14e-05
Epoch 47/200 | Train Loss: 0.4676 Acc: 0.9950 | Val Loss: 0.5615 Acc: 0.9294 | LR: 9.09e-05
Epoch 48/200 | Train Loss: 0.4639 Acc: 0.9925 | Val Loss: 0.5609 Acc: 0.9412 | LR: 9.05e-05
Epoch 49/200 | Train Loss: 0.4716 Acc: 0.9950 | Val Loss: 0.5665 Acc: 0.9294 | LR: 9.00e-05
Epoch 50/200 | Train Loss: 0.4640 Acc: 0.9825 | Val Loss: 0.5692 Acc: 0.9294 | LR: 8.95e-05
Epoch 51/200 | Train Loss: 0.4605 Acc: 0.9925 | Val Loss: 0.5677 Acc: 0.9294 | LR: 8.89e-05
Epoch 52/200 | Train Loss: 0.4596 Acc: 0.9925 | Val Loss: 0.5701 Acc: 0.9176 | LR: 8.84e-05
Epoch 53/200 | Train Loss: 0.4634 Acc: 0.9850 | Val Loss: 0.5686 Acc: 0.9294 | LR: 8.79e-05
Epoch 54/200 | Train Loss: 0.4668 Acc: 0.9800 | Val Loss: 0.5640 Acc: 0.9294 | LR: 8.73e-05
Epoch 55/200 | Train Loss: 0.4619 Acc: 0.9825 | Val Loss: 0.5615 Acc: 0.9176 | LR: 8.68e-05
Epoch 56/200 | Train Loss: 0.4528 Acc: 0.9900 | Val Loss: 0.5598 Acc: 0.9294 | LR: 8.62e-05
Epoch 57/200 | Train Loss: 0.4687 Acc: 0.9750 | Val Loss: 0.5569 Acc: 0.9529 | LR: 8.56e-05
Epoch 58/200 | Train Loss: 0.4499 Acc: 0.9950 | Val Loss: 0.5581 Acc: 0.9529 | LR: 8.51e-05
Epoch 59/200 | Train Loss: 0.4515 Acc: 0.9950 | Val Loss: 0.5571 Acc: 0.9412 | LR: 8.45e-05
Epoch 60/200 | Train Loss: 0.4491 Acc: 0.9925 | Val Loss: 0.5583 Acc: 0.9412 | LR: 8.39e-05
Epoch 61/200 | Train Loss: 0.4532 Acc: 0.9950 | Val Loss: 0.5578 Acc: 0.9412 | LR: 8.33e-05
Epoch 62/200 | Train Loss: 0.4576 Acc: 0.9900 | Val Loss: 0.5576 Acc: 0.9529 | LR: 8.26e-05
Epoch 63/200 | Train Loss: 0.4549 Acc: 0.9900 | Val Loss: 0.5612 Acc: 0.9412 | LR: 8.20e-05
Epoch 64/200 | Train Loss: 0.4562 Acc: 0.9925 | Val Loss: 0.5623 Acc: 0.9412 | LR: 8.14e-05
Epoch 65/200 | Train Loss: 0.4500 Acc: 0.9975 | Val Loss: 0.5629 Acc: 0.9412 | LR: 8.07e-05
Epoch 66/200 | Train Loss: 0.4532 Acc: 0.9900 | Val Loss: 0.5587 Acc: 0.9412 | LR: 8.01e-05
Epoch 67/200 | Train Loss: 0.4518 Acc: 0.9925 | Val Loss: 0.5569 Acc: 0.9176 | LR: 7.94e-05
Epoch 68/200 | Train Loss: 0.4541 Acc: 0.9900 | Val Loss: 0.5615 Acc: 0.9294 | LR: 7.87e-05
Epoch 69/200 | Train Loss: 0.4494 Acc: 0.9975 | Val Loss: 0.5582 Acc: 0.9176 | LR: 7.80e-05
Epoch 70/200 | Train Loss: 0.4496 Acc: 0.9925 | Val Loss: 0.5559 Acc: 0.9294 | LR: 7.73e-05
Epoch 71/200 | Train Loss: 0.4578 Acc: 0.9925 | Val Loss: 0.5550 Acc: 0.9294 | LR: 7.67e-05
Epoch 72/200 | Train Loss: 0.4489 Acc: 0.9950 | Val Loss: 0.5533 Acc: 0.9294 | LR: 7.59e-05
Epoch 73/200 | Train Loss: 0.4575 Acc: 0.9875 | Val Loss: 0.5569 Acc: 0.9176 | LR:

7.52e-05
Epoch 74/200 | Train Loss: 0.4424 Acc: 0.9975 | Val Loss: 0.5555 Acc: 0.9176 | LR: 7.45e-05
Epoch 75/200 | Train Loss: 0.4459 Acc: 0.9975 | Val Loss: 0.5545 Acc: 0.9176 | LR: 7.38e-05
Epoch 76/200 | Train Loss: 0.4493 Acc: 0.9925 | Val Loss: 0.5541 Acc: 0.9176 | LR: 7.31e-05
Epoch 77/200 | Train Loss: 0.4452 Acc: 0.9975 | Val Loss: 0.5527 Acc: 0.9176 | LR: 7.23e-05
Epoch 78/200 | Train Loss: 0.4481 Acc: 0.9975 | Val Loss: 0.5556 Acc: 0.9294 | LR: 7.16e-05
Epoch 79/200 | Train Loss: 0.4429 Acc: 1.0000 | Val Loss: 0.5520 Acc: 0.9176 | LR: 7.08e-05
Epoch 80/200 | Train Loss: 0.4489 Acc: 0.9950 | Val Loss: 0.5554 Acc: 0.9294 | LR: 7.01e-05
Epoch 81/200 | Train Loss: 0.4477 Acc: 0.9975 | Val Loss: 0.5531 Acc: 0.9176 | LR: 6.93e-05
Epoch 82/200 | Train Loss: 0.4455 Acc: 0.9950 | Val Loss: 0.5530 Acc: 0.9176 | LR: 6.86e-05
Epoch 83/200 | Train Loss: 0.4398 Acc: 0.9925 | Val Loss: 0.5530 Acc: 0.9176 | LR: 6.78e-05
Epoch 84/200 | Train Loss: 0.4486 Acc: 0.9975 | Val Loss: 0.5528 Acc: 0.9176 | LR: 6.70e-05
Epoch 85/200 | Train Loss: 0.4527 Acc: 0.9975 | Val Loss: 0.5513 Acc: 0.9412 | LR: 6.62e-05
Epoch 86/200 | Train Loss: 0.4415 Acc: 0.9975 | Val Loss: 0.5499 Acc: 0.9294 | LR: 6.55e-05
Epoch 87/200 | Train Loss: 0.4441 Acc: 0.9950 | Val Loss: 0.5492 Acc: 0.9176 | LR: 6.47e-05
Epoch 88/200 | Train Loss: 0.4451 Acc: 0.9925 | Val Loss: 0.5457 Acc: 0.9294 | LR: 6.39e-05
Epoch 89/200 | Train Loss: 0.4398 Acc: 0.9950 | Val Loss: 0.5460 Acc: 0.9294 | LR: 6.31e-05
Epoch 90/200 | Train Loss: 0.4382 Acc: 0.9975 | Val Loss: 0.5478 Acc: 0.9294 | LR: 6.23e-05
Epoch 91/200 | Train Loss: 0.4370 Acc: 0.9975 | Val Loss: 0.5493 Acc: 0.9294 | LR: 6.15e-05
Epoch 92/200 | Train Loss: 0.4435 Acc: 0.9950 | Val Loss: 0.5488 Acc: 0.9176 | LR: 6.07e-05
Epoch 93/200 | Train Loss: 0.4429 Acc: 0.9975 | Val Loss: 0.5490 Acc: 0.9176 | LR: 5.99e-05
Epoch 94/200 | Train Loss: 0.4434 Acc: 0.9950 | Val Loss: 0.5531 Acc: 0.9176 | LR: 5.90e-05
Epoch 95/200 | Train Loss: 0.4393 Acc: 1.0000 | Val Loss: 0.5554 Acc: 0.9176 | LR: 5.82e-05
Epoch 96/200 | Train Loss: 0.4441 Acc: 0.9950 | Val Loss: 0.5557 Acc: 0.9176 | LR: 5.74e-05
Epoch 97/200 | Train Loss: 0.4353 Acc: 0.9950 | Val Loss: 0.5542 Acc: 0.9294 | LR: 5.66e-05
Epoch 98/200 | Train Loss: 0.4355 Acc: 0.9950 | Val Loss: 0.5515 Acc: 0.9294 | LR: 5.58e-05
Epoch 99/200 | Train Loss: 0.4365 Acc: 1.0000 | Val Loss: 0.5551 Acc: 0.9176 | LR: 5.50e-05
Epoch 100/200 | Train Loss: 0.4392 Acc: 0.9975 | Val Loss: 0.5505 Acc: 0.9176 | LR: 5.41e-05
Epoch 101/200 | Train Loss: 0.4356 Acc: 1.0000 | Val Loss: 0.5489 Acc: 0.9294 | LR:

5.33e-05

Epoch 102/200 | Train Loss: 0.4379 Acc: 0.9975 | Val Loss: 0.5489 Acc: 0.9176 | LR: 5.25e-05

Epoch 103/200 | Train Loss: 0.4396 Acc: 0.9975 | Val Loss: 0.5482 Acc: 0.9294 | LR: 5.17e-05

Epoch 104/200 | Train Loss: 0.4443 Acc: 0.9975 | Val Loss: 0.5518 Acc: 0.9176 | LR: 5.08e-05

Epoch 105/200 | Train Loss: 0.4366 Acc: 0.9975 | Val Loss: 0.5531 Acc: 0.9176 | LR: 5.00e-05

Epoch 106/200 | Train Loss: 0.4329 Acc: 0.9975 | Val Loss: 0.5509 Acc: 0.9176 | LR: 4.92e-05

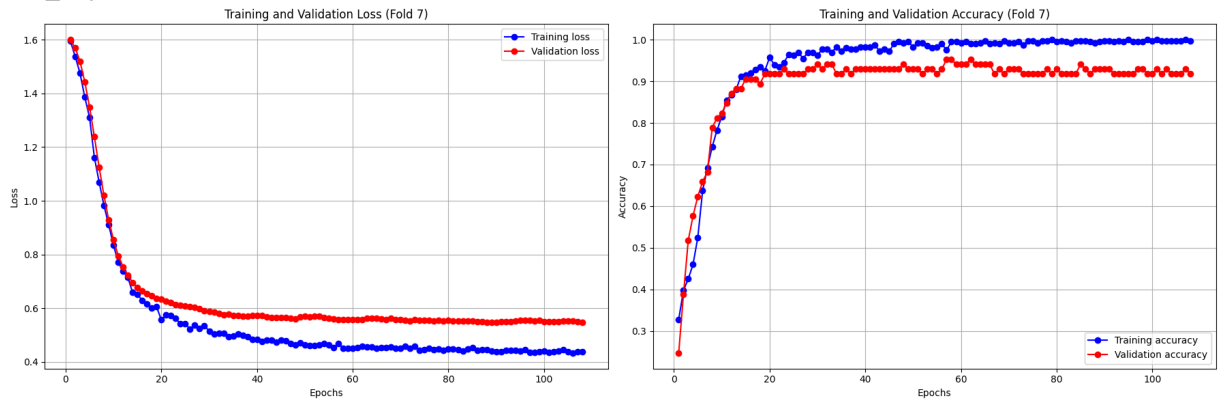
Epoch 107/200 | Train Loss: 0.4374 Acc: 1.0000 | Val Loss: 0.5499 Acc: 0.9294 | LR: 4.83e-05

Epoch 108/200 | Train Loss: 0.4376 Acc: 0.9975 | Val Loss: 0.5465 Acc: 0.9176 | LR: 4.75e-05

Early stopping triggered.

Training complete in 7.4 minutes. Best Val Loss: 0.5457

Best model for fold saved to: new_results_Multimodal_DINO_Concat_KFold\best_model_fold_7.pth



Saved training history plot to: new_results_Multimodal_DINO_Concat_KFold\training_history_fold_7.png

=====

OVERALL PERFORMANCE (FOLD 7)

=====

	precision	recall	f1-score	support
Herpes	1.00	0.83	0.91	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.90	1.00	0.95	18
Sarampion	1.00	0.93	0.96	29
Varicela	0.96	1.00	0.98	25
accuracy			0.96	80
macro avg	0.97	0.95	0.96	80
weighted avg	0.97	0.96	0.96	80

--- Overall Scalar Metrics ---

Accuracy: 0.9625

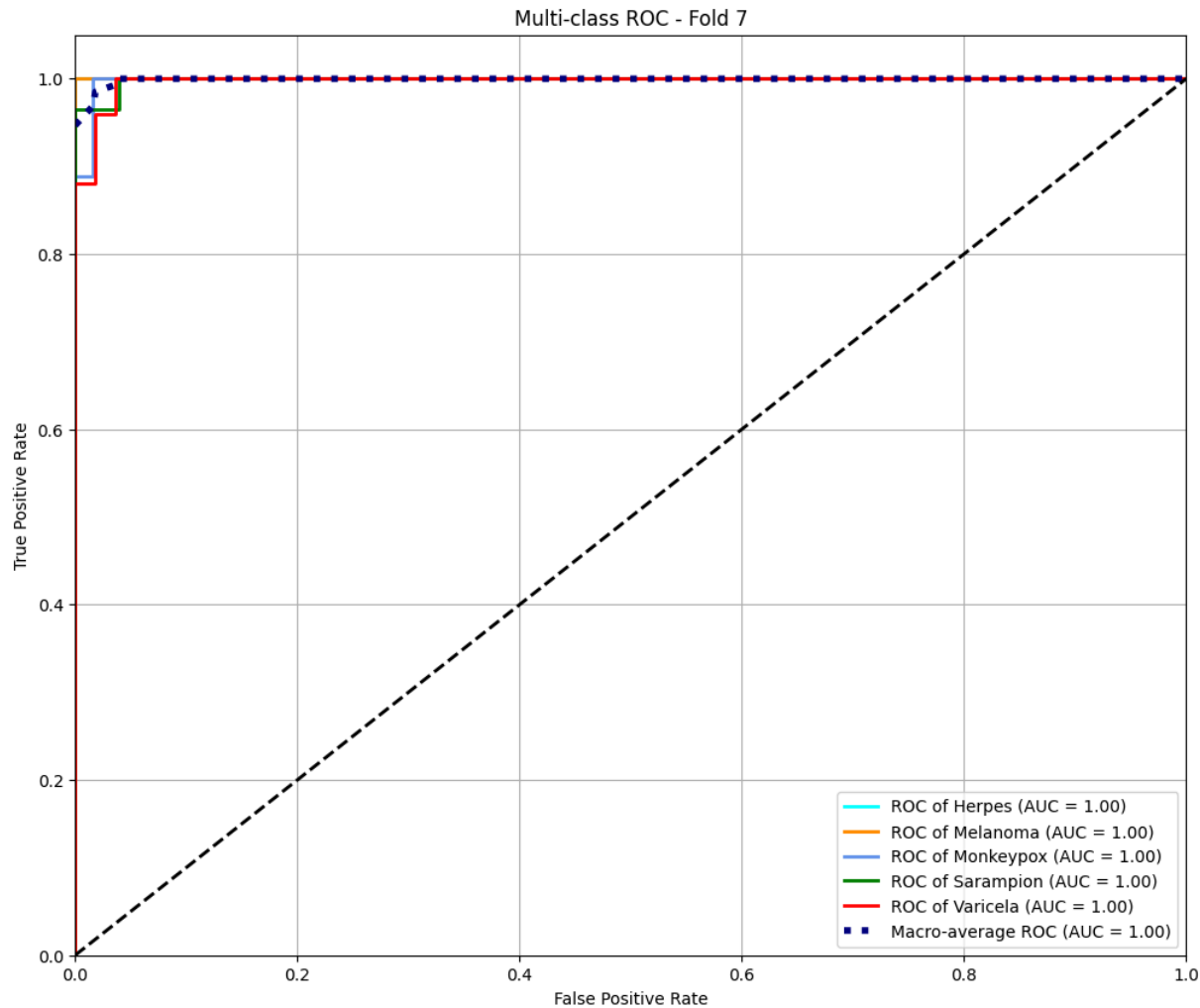
Macro ROC AUC: 0.9988

Cohen's Kappa: 0.9475

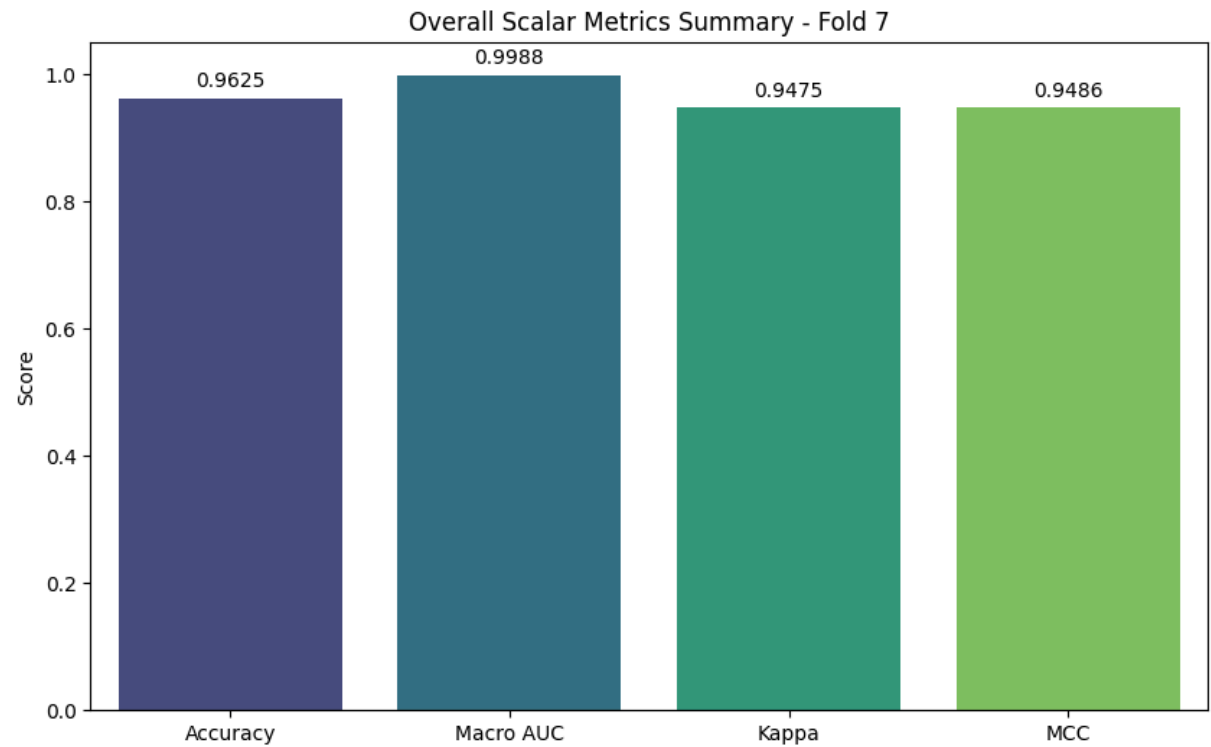
Matthews CorrCoef: 0.9486



Saved normalized confusion matrix to: new_results_Multimodal_DINO_Concat_KFold\confu
sion_matrix_fold_7.png



Saved ROC curves plot to: new_results_Multimodal_DINO_Concat_KFold\roc_curves_fold_7.png



Saved scalar metrics plot to: new_results_Multimodal_DINO_Concat_KFold\scalar_metrics_fold_7.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	15
Varicela	1.00	1.00	1.00	12
accuracy			1.00	30
macro avg	0.60	0.60	0.60	30
weighted avg	1.00	1.00	1.00	30

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	1.00	0.83	0.91	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.88	1.00	0.94	15
Sarampion	1.00	0.86	0.92	14
Varicela	0.93	1.00	0.96	13
accuracy			0.94	50
macro avg	0.96	0.94	0.95	50
weighted avg	0.95	0.94	0.94	50

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.67	1.00	0.80	2
Sarampion	1.00	0.67	0.80	3
Varicela	0.83	1.00	0.91	5
accuracy			0.82	11
macro avg	0.50	0.53	0.50	11
weighted avg	0.77	0.82	0.78	11

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	5
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.94	1.00	0.97	15
Sarampion	1.00	0.96	0.98	26
Varicela	1.00	1.00	1.00	20
accuracy			0.99	68
macro avg	0.99	0.99	0.99	68
weighted avg	0.99	0.99	0.99	68

--- Subgroup: 60+ ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	1.00	1.00	5
Varicela	1.00	1.00	1.00	4
accuracy			1.00	13
macro avg	0.60	0.60	0.60	13
weighted avg	1.00	1.00	1.00	13

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.83	1.00	0.91	5
Sarampion	1.00	0.50	0.67	2
Varicela	1.00	1.00	1.00	1
accuracy			0.89	9

macro avg	0.77	0.70	0.72	9
weighted avg	0.91	0.89	0.88	9

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	5
macro avg	0.40	0.40	0.40	5
weighted avg	1.00	1.00	1.00	5

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	3
Varicela	0.00	0.00	0.00	0
accuracy			1.00	5
macro avg	0.40	0.40	0.40	5
weighted avg	1.00	1.00	1.00	5

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	0.50	0.67	2
Varicela	0.00	0.00	0.00	0
accuracy			0.50	2
macro avg	0.20	0.10	0.13	2
weighted avg	1.00	0.50	0.67	2

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	1.00	0.80	0.89	5
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	16
Varicela	0.95	1.00	0.98	20
accuracy			0.98	46

macro avg	0.99	0.96	0.97	46
weighted avg	0.98	0.98	0.98	46

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.5000 EOD: 0.9000
Worst pair: 18-59|male|middle eastern vs 18-59|male|white
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 1.0000 EOD: 0.6000
Worst pair: 18-59|female|white vs 18-59|male|white
Groups: ['18-59|female|white', '18-59|male|white']
=====
```

```
=====
====
Cross-validated over 7 folds
=====
```

```
====
Accuracy          : 0.938 ± 0.035
Roc Auc Macro     : 0.993 ± 0.007
Kappa             : 0.914 ± 0.049
Mcc               : 0.914 ± 0.049
Wga All           : 0.343 ± 0.336
Eod All           : 0.868 ± 0.069
Wga Filtered      : 0.920 ± 0.066
Eod Filtered      : 0.469 ± 0.237
```

```
--- POOLED OVERALL ACCURACY BY GENDER (n>=15) ---
```

Group	Acc	Count
male	0.938	355
female	0.938	210

```
--- POOLED OVERALL ACCURACY BY AGE (n>=15) ---
```

Group	Acc	Count
18-59	0.946	501
0-17	0.869	61

```
--- POOLED OVERALL ACCURACY BY SKIN TONE (n>=15) ---
```

Group	Acc	Count
white	0.941	338
asian	0.900	70
indian	0.960	50
latino hispanic	0.960	50
black	0.974	38
middle eastern	0.842	19

```
--- POOLED OVERALL ACCURACY BY COMPOSITE (n>=15) ---
```

Group	Acc	Count
18-59 male white	0.934	166
18-59 female white	0.960	126
18-59 male asian	0.897	39
18-59 male indian	1.000	34
18-59 male black	1.000	30
0-17 male white	0.933	30

18-59 female asian	0.962	26
18-59 female latino hispanic	0.920	25
18-59 male latino hispanic	1.000	23
18-59 male middle eastern	0.833	18
0-17 female white	0.867	15

Results, models, and plots saved to 'new_results_Multimodal_DINO_Concat_KFold'

```
In [ ]: # =====
# Attention Map Visualization Script for Multiple Subgroup Comparisons
# - Generates separate plots for different demographic axes (age, skin tone, etc.)
# - Easily configurable via the `plotting_jobs` list in the main section.
# =====

import os
import random
import warnings
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
from PIL import Image
from torchvision import transforms
import cv2

# Conditional import for Hugging Face transformers
try:
    from transformers import AutoModel, AutoConfig
    HAVE_TRANSFORMERS = True
except ImportError:
    HAVE_TRANSFORMERS = False

# Suppress warnings for cleaner output
warnings.filterwarnings('ignore')
torch.backends.cudnn.benchmark = True

# =====
# 1. CONFIGURATION
# =====

class Config:
    DATA_DIR = r"D:\AIUB\Neural\Synthetic_Skin_Disease_Dataset"
    ORIGINAL_METADATA_CSV = r"D:\AIUB\Neural\complete_metadata_m.csv"
    RESULTS_DIR = "new_results_Multimodal_DINO_Concat_KFold"
    LOAD_MODEL_PATH = os.path.join(RESULTS_DIR, "best_model_fold_5.pth")
    MODEL_NAME = "facebook/dinov3-vitb16-pretrain-lvd1689m"
    IMG_SIZE = 224
    DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
    N_SPLITS = 7

config = Config()
print(f"Using device: {config.DEVICE}")

# =====
```

```

# 2. MODEL AND DATA HELPERS (from previous versions)
# =====
def standardize_metadata_columns(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    if 'filepath' in df.columns: df['filepath'] = df['filepath'].astype(str).str.replace(
    if 'filename' not in df.columns:
        if 'filepath' in df.columns: df['filename'] = df['filepath'].apply(lambda p
    if 'label' not in df.columns and 'filepath' in df.columns: df['label'] = df['fi
    df['label'] = df['label'].astype(str)

    # =====> THIS IS THE CORRECTED SECTION <=====
    # Restored the original age group definitions to match the trained model checkp
    if 'age_group' not in df.columns and 'age' in df.columns:
        age_num = pd.to_numeric(df['age'], errors='coerce')
        bins, names = [-1, 17, 29, 44, 64, 200], ['child', 'young_adult', 'adult',
        df['age_group'] = pd.cut(age_num, bins=bins, labels=names).astype(str).fill

    if 'gender' not in df.columns and 'dominant_gender' in df.columns:
        g = df['dominant_gender'].astype(str).str.strip().str.lower()
        df['gender'] = g.map({'man': 'male', 'male': 'male', 'woman': 'female', 'female':
    if 'skin_tone' not in df.columns and 'dominant_race' in df.columns:
        df['skin_tone'] = df['dominant_race'].astype(str).str.strip().str.lower()
    return df

class MetaVocab:
    def __init__(self, df_train, cols=('age_group', 'gender', 'skin_tone')):
        self.cols, self.unk_token = cols, "<UNK>"
        self.vocabs = {c: {v: i for i, v in enumerate(sorted([self.unk_token] + df_
    def encode_row(self, row): return {c: self.vocabs[c].get(str(row[c]), self.voca

class MetadataEncoder(nn.Module):
    def __init__(self, meta_vocab, hidden=128, out_dim=256, dropout=0.1):
        super().__init__()
        sizes = {c: len(v) for c, v in meta_vocab.vocabs.items()}
        emb_dims = {'age_group': min(16, max(4, sizes['age_group'])), 'gender': min
        self.emb_age, self.emb_gen, self.emb_tone = nn.Embedding(sizes['age_group']
        self.mlp = nn.Sequential(nn.Linear(sum(emb_dims.values()), hidden), nn.GELU
    def forward(self, meta_idx): return self.mlp(torch.cat([self.emb_age(meta_idx['

class DINOv3MultimodalConcat(nn.Module):
    def __init__(self, model_name, num_classes, meta_vocab, meta_hidden_dim=128, me
        super().__init__()
        model_config = AutoConfig.from_pretrained(model_name)
        model_config.output_attentions = True
        self.backbone = AutoModel.from_pretrained(model_name, config=model_config)
        image_feature_dim = self.backbone.config.hidden_size
        self.meta_encoder = MetadataEncoder(meta_vocab, hidden=meta_hidden_dim, out
        self.classifier = nn.Sequential(nn.Linear(image_feature_dim + meta_out_dim,
    def forward(self, pixel_values, meta_idx, output_attentions=False):
        outputs = self.backbone(pixel_values=pixel_values, output_attentions=output
        img_features = outputs.last_hidden_state[:, 0]
        meta_features = self.meta_encoder(meta_idx)
        logits = self.classifier(torch.cat((img_features, meta_features), dim=1))
        return (logits, outputs.attentions) if output_attentions else logits

from sklearn.model_selection import StratifiedKFold

```

```

def build_kfold_splits(df_std, n_splits, seed=42):
    df, y = df_std.copy().reset_index(drop=True), df_std['label'].astype(str).value
    skf, folds = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed
    for _, test_idx in skf.split(np.arange(len(df)), y):
        dfi = df.copy(); dfi['split'] = 'Train'; dfi.loc[test_idx, 'split'] = 'Test'
    return folds

# =====
# 3. ATTENTION MAP VISUALIZATION
# =====
def get_model_and_data(config):
    df_std = standardize_metadata_columns(pd.read_csv(config.ORIGINAL_METADATA_CSV))
    fold_1_df = build_kfold_splits(df_std, config.N_SPLITS)[0]
    df_train, df_test = fold_1_df[fold_1_df['split'] == 'Train'], fold_1_df[fold_1_
    classes = sorted(df_train['label'].astype(str).unique())
    class_to_idx = {c: i for i, c in enumerate(classes)}
    idx_to_class = {i: c for c, i in class_to_idx.items()}
    meta_vocab, num_classes = MetaVocab(df_train), len(classes)
    model = DINOv3MultimodalConcat(model_name=config.MODEL_NAME, num_classes=num_cl
    model.load_state_dict(torch.load(config.LOAD_MODEL_PATH, map_location=config.DE
    print(f"Model and data loaded successfully.")
    return model, df_test, meta_vocab, idx_to_class

def generate_attention_map(model, image_tensor, meta_idx):
    input_tensor = image_tensor.unsqueeze(0).to(config.DEVICE)
    meta_idx_tensor = {k: torch.tensor([v], device=config.DEVICE) for k, v in meta_
    with torch.no_grad():
        logits, attentions = model(input_tensor, meta_idx_tensor, output_attentions
        prediction_idx = logits.argmax(1).item()
        avg_attention = torch.mean(attentions[-1].squeeze(0), dim=0)
        num_patches, num_special_tokens = 196, avg_attention.shape[0] - 196
        image_patch_attention = avg_attention[0, num_special_tokens:].cpu().numpy()
        h = w = int(num_patches**0.5)
        attention_map = cv2.normalize(image_patch_attention.reshape(h, w), None, 0, 255
        attention_map = cv2.resize(attention_map, (config.IMG_SIZE, config.IMG_SIZE)).a
        img_np = np.clip((input_tensor.squeeze().cpu().permute(1, 2, 0).numpy() * 0.225
        heatmap = cv2.applyColorMap(attention_map, cv2.COLORMAP_JET)
        overlay = cv2.addWeighted((img_np * 255).astype(np.uint8), 0.6, heatmap, 0.4, 0
    return overlay, prediction_idx

# =====
# 4. MAIN EXECUTION
# =====
def main():
    if not HAVE_TRANSFORMERS:
        print("Missing libraries. Please run: pip install transformers opencv-pytho

    model, df_test, meta_vocab, idx_to_class = get_model_and_data(config)
    eval_transform = transforms.Compose([transforms.Resize((config.IMG_SIZE, config

    # =====
    # =====> EDIT THIS LIST TO DEFINE YOUR COMPARISONS <=====
    # =====
    plotting_jobs = [
        {
            "title": "Comparison by Skin Tone (Male, Adult)",

```

```

        "save_filename": "attention_comparison_skin_tone.png",
        "plot_grid": (2, 3), # Rows, Columns
        "image_criteria": [
            {"gender": "male", "age_group": "adult", "skin_tone": "white"},
            {"gender": "male", "age_group": "adult", "skin_tone": "black"},
            {"gender": "male", "age_group": "adult", "skin_tone": "asian"},
        ]
    },
    {
        "title": "Comparison by Age (Female, White)",
        "save_filename": "attention_comparison_age.png",
        "plot_grid": (2, 3),
        "image_criteria": [
            {"gender": "female", "age_group": "child", "skin_tone": "white"},
            {"gender": "female", "age_group": "adult", "skin_tone": "white"},
            {"gender": "female", "age_group": "senior", "skin_tone": "white"},
        ]
    },
    {
        "title": "Comparison by Gender (Adult, Black)",
        "save_filename": "attention_comparison_gender.png",
        "plot_grid": (2, 2),
        "image_criteria": [
            {"gender": "male", "age_group": "adult", "skin_tone": "black"},
            {"gender": "female", "age_group": "adult", "skin_tone": "black"},
        ]
    },
]

# --- Plotting Loop ---
for job in plotting_jobs:
    print(f"\n--- Generating plot: {job['title']} ---")
    num_images = len(job['image_criteria'])
    rows, cols = job['plot_grid']
    fig, axes = plt.subplots(rows, cols, figsize=(5 * cols, 5 * rows))
    fig.suptitle(job['title'], fontsize=18, fontweight='bold')
    axes = axes.flatten()

    for i, criteria in enumerate(job['image_criteria']):
        # Find a matching image
        subset = df_test
        for key, value in criteria.items():
            subset = subset[subset[key] == value]

        if subset.empty:
            print(f"Warning: No image found for criteria: {criteria}. Skipping.")
            axes[i].text(0.5, 0.5, f"Image not found\n{criteria}", ha='center',
                        axes[i].axis('off')
            axes[i + cols].axis('off')
            continue

        sample = subset.sample(1).iloc[0]

        # Generate attention map
        img_path = os.path.join(config.DATA_DIR, sample['filepath'])
        img_pil = Image.open(img_path).convert("RGB")

```

```

img_tensor, meta_idx = eval_transform(img_pil), meta_vocab.encode_row(s
overlay, pred_idx = generate_attention_map(model, img_tensor, meta_idx)

# Get info for titles
pred_label, true_label = idx_to_class[pred_idx], sample['label']
subtitle = f"True: {true_label}, Pred: {pred_label}\n" + ", ".join([f"{

# Plot original and overlay
axes[i].imshow(img_pil)
axes[i].set_title(subtitle)
axes[i].axis('off')

axes[i + cols].imshow(overlay)
axes[i + cols].axis('off')

# Remove any unused subplots
for i in range(num_images, cols):
    axes[i].axis('off')
    axes[i + cols].axis('off')

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(job['save_filename'])
plt.show()
print(f"Plot saved as {job['save_filename']}")

if __name__ == '__main__':
    main()

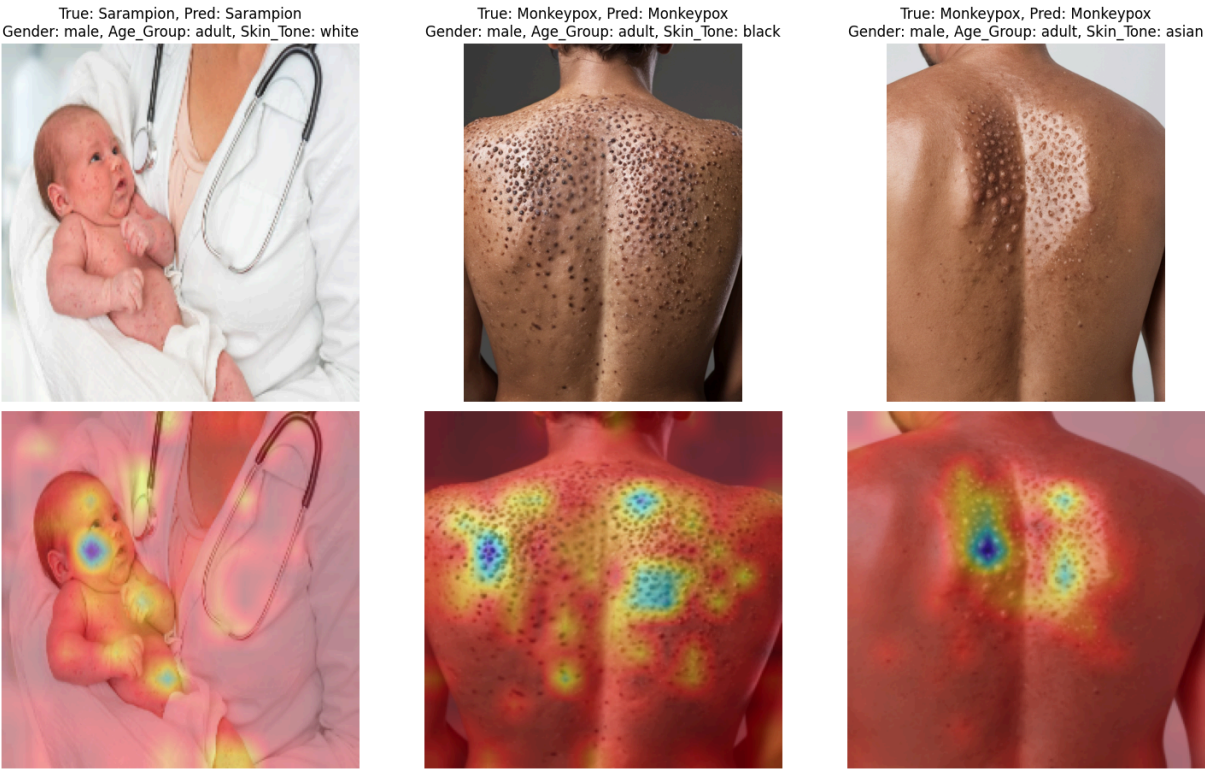
```

Using device: cuda

Model and data loaded successfully.

--- Generating plot: Comparison by Skin Tone (Male, Adult) ---

Comparison by Skin Tone (Male, Adult)



Plot saved as attention_comparison_skin_tone.png

--- Generating plot: Comparison by Age (Female, White) ---
Warning: No image found for criteria: {'gender': 'female', 'age_group': 'senior', 's
kin_tone': 'white'}. Skipping.

Comparison by Age (Female, White)



Plot saved as attention_comparison_age.png

--- Generating plot: Comparison by Gender (Adult, Black) ---

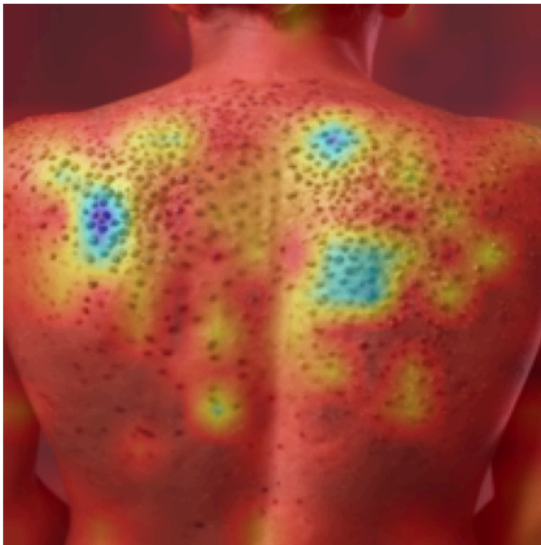
Warning: No image found for criteria: {'gender': 'female', 'age_group': 'adult', 'skin_tone': 'black'}. Skipping.

Comparison by Gender (Adult, Black)

True: Monkeypox, Pred: Monkeypox
Gender: male, Age_Group: adult, Skin_Tone: black



Image not found
{'gender': 'female', 'age_group': 'adult', 'skin_tone': 'black'}



Plot saved as attention_comparison_gender.png

```
In [ ]: # =====
# Attention Map Visualization Script for Multiple Subgroup Comparisons
# - Generates separate plots for different demographic axes (age, skin tone, etc.)
# - Easily configurable via the `plotting_jobs` list in the main section.
# =====

import os
import random
import warnings
import numpy as np
import pandas as pd
```

```

import torch
import torch.nn as nn
import matplotlib.pyplot as plt
from PIL import Image
from torchvision import transforms
import cv2

# Conditional import for Hugging Face transformers
try:
    from transformers import AutoModel, AutoConfig
    HAVE_TRANSFORMERS = True
except ImportError:
    HAVE_TRANSFORMERS = False

# Suppress warnings for cleaner output
warnings.filterwarnings('ignore')
torch.backends.cudnn.benchmark = True

# =====
# 1. CONFIGURATION
# =====
class Config:
    DATA_DIR = r"D:\AIUB\Neural\Synthetic_Skin_Disease_Dataset"
    ORIGINAL_METADATA_CSV = r"D:\AIUB\Neural\complete_metadata_m.csv"
    RESULTS_DIR = "new_results_Multimodal_DINO_Concat_KFold"
    LOAD_MODEL_PATH = os.path.join(RESULTS_DIR, "best_model_fold_5.pth")
    MODEL_NAME = "facebook/dinov3-vitb16-pretrain-lvd1689m"
    IMG_SIZE = 224
    DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
    N_SPLITS = 7

config = Config()
print(f"Using device: {config.DEVICE}")

# =====
# 2. MODEL AND DATA HELPERS (from previous versions)
# =====
def standardize_metadata_columns(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    if 'filepath' in df.columns: df['filepath'] = df['filepath'].astype(str).str.replace('\\', '/')
    if 'filename' not in df.columns:
        if 'filepath' in df.columns: df['filename'] = df['filepath'].apply(lambda p: os.path.basename(p))
    if 'label' not in df.columns and 'filepath' in df.columns: df['label'] = df['filepath'].apply(lambda p: os.path.splitext(os.path.basename(p))[0])
    df['label'] = df['label'].astype(str)

    # =====> THIS IS THE CORRECTED SECTION <=====
    # Restored the original age group definitions to match the trained model checkpoints
    if 'age_group' not in df.columns and 'age' in df.columns:
        age_num = pd.to_numeric(df['age'], errors='coerce')
        bins, names = [-1, 17, 29, 44, 64, 200], ['child', 'young_adult', 'adult', 'elderly']
        df['age_group'] = pd.cut(age_num, bins=bins, labels=names).astype(str).fillna('')

    if 'gender' not in df.columns and 'dominant_gender' in df.columns:
        g = df['dominant_gender'].astype(str).str.strip().str.lower()

```

```

        df['gender'] = g.map({'man':'male','male':'male','woman':'female','female':
if 'skin_tone' not in df.columns and 'dominant_race' in df.columns:
        df['skin_tone'] = df['dominant_race'].astype(str).str.strip().str.lower()
    return df

class MetaVocab:
    def __init__(self, df_train, cols=('age_group', 'gender', 'skin_tone')):
        self.cols, self.unk_token = cols, "<UNK>"
        self.vocabs = {c: {v: i for i, v in enumerate(sorted([self.unk_token] + df_
    def encode_row(self, row): return {c: self.vocabs[c].get(str(row[c]), self.voca

class MetadataEncoder(nn.Module):
    def __init__(self, meta_vocab, hidden=128, out_dim=256, dropout=0.1):
        super().__init__()
        sizes = {c: len(v) for c, v in meta_vocab.vocabs.items()}
        emb_dims = {'age_group': min(16, max(4, sizes['age_group'])), 'gender': min
        self.emb_age, self.emb_gen, self.emb_tone = nn.Embedding(sizes['age_group']
        self.mlp = nn.Sequential(nn.Linear(sum(emb_dims.values()), hidden), nn.GELU
    def forward(self, meta_idx): return self.mlp(torch.cat([self.emb_age(meta_idx['

class DINOv3MultimodalConcat(nn.Module):
    def __init__(self, model_name, num_classes, meta_vocab, meta_hidden_dim=128, me
        super().__init__()
        model_config = AutoConfig.from_pretrained(model_name)
        model_config.output_attentions = True
        self.backbone = AutoModel.from_pretrained(model_name, config=model_config)
        image_feature_dim = self.backbone.config.hidden_size
        self.meta_encoder = MetadataEncoder(meta_vocab, hidden=meta_hidden_dim, out
        self.classifier = nn.Sequential(nn.Linear(image_feature_dim + meta_out_dim,
    def forward(self, pixel_values, meta_idx, output_attentions=False):
        outputs = self.backbone(pixel_values=pixel_values, output_attentions=output
        img_features = outputs.last_hidden_state[:, 0]
        meta_features = self.meta_encoder(meta_idx)
        logits = self.classifier(torch.cat((img_features, meta_features), dim=1))
        return (logits, outputs.attentions) if output_attentions else logits

from sklearn.model_selection import StratifiedKFold
def build_kfold_splits(df_std, n_splits, seed=42):
    df, y = df_std.copy().reset_index(drop=True), df_std['label'].astype(str).value
    skf, folds = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed
    for _, test_idx in skf.split(np.arange(len(df)), y):
        dfi = df.copy(); dfi['split'] = 'Train'; dfi.loc[test_idx, 'split'] = 'Test
    return folds

# =====
# 3. ATTENTION MAP VISUALIZATION
# =====

def get_model_and_data(config):
    df_std = standardize_metadata_columns(pd.read_csv(config.ORIGINAL_METADATA_CSV)
    fold_1_df = build_kfold_splits(df_std, config.N_SPLITS)[0]
    df_train, df_test = fold_1_df[fold_1_df['split'] == 'Train'], fold_1_df[fold_1_
    classes = sorted(df_train['label'].astype(str).unique())
    class_to_idx = {c: i for i, c in enumerate(classes)}
    idx_to_class = {i: c for c, i in class_to_idx.items()}
    meta_vocab, num_classes = MetaVocab(df_train), len(classes)
    model = DINOv3MultimodalConcat(model_name=config.MODEL_NAME, num_classes=num_cl

```

```

model.load_state_dict(torch.load(config.LOAD_MODEL_PATH, map_location=config.DEVICE))
print(f"Model and data loaded successfully.")
return model, df_test, meta_vocab, idx_to_class

def generate_attention_map(model, image_tensor, meta_idx):
    input_tensor = image_tensor.unsqueeze(0).to(config.DEVICE)
    meta_idx_tensor = {k: torch.tensor([v], device=config.DEVICE) for k, v in meta_vocab.items()}
    with torch.no_grad():
        logits, attentions = model(input_tensor, meta_idx_tensor, output_attentions=True)
        prediction_idx = logits.argmax(1).item()
        avg_attention = torch.mean(attentions[-1].squeeze(0), dim=0)
        num_patches, num_special_tokens = 196, avg_attention.shape[0] - 196
        image_patch_attention = avg_attention[0, num_special_tokens:].cpu().numpy()
        h = w = int(num_patches**0.5)
        attention_map = cv2.normalize(image_patch_attention.reshape(h, w), None, 0, 255, cv2.CV_8UC1)
        attention_map = cv2.resize(attention_map, (config.IMG_SIZE, config.IMG_SIZE)).astype(np.float32)
        img_np = np.clip((input_tensor.squeeze().cpu().permute(1, 2, 0).numpy() * 0.225).astype(np.uint8), 0, 255)
        heatmap = cv2.applyColorMap(attention_map, cv2.COLORMAP_JET)
        overlay = cv2.addWeighted((img_np * 255).astype(np.uint8), 0.6, heatmap, 0.4, 0)
    return overlay, prediction_idx

# =====
# 4. MAIN EXECUTION
# =====
def main():
    if not HAVE_TRANSFORMERS:
        print("Missing libraries. Please run: pip install transformers opencv-python")

    model, df_test, meta_vocab, idx_to_class = get_model_and_data(config)
    eval_transform = transforms.Compose([transforms.Resize((config.IMG_SIZE, config.IMG_SIZE))])

    # =====
    # =====> EDIT THIS LIST TO DEFINE YOUR COMPARISONS <=====
    # =====
    plotting_jobs = [
        {
            "title": "Comparison by Gender (Adult, White)",
            "save_filename": "attention_comparison_gender.png",
            "plot_grid": (2, 2), # (Rows, Columns) -> 2 rows (original, heatmap), 2 columns (original, heatmap)
            "image_criteria": [
                {"gender": "male", "age_group": "adult", "skin_tone": "white"},
                {"gender": "female", "age_group": "adult", "skin_tone": "white"}
            ]
        },
        {
            "title": "Comparison by Age Group (Male, White)",
            "save_filename": "attention_comparison_age.png",
            "plot_grid": (2, 2),
            "image_criteria": [
                {"age_group": "child", "gender": "male", "skin_tone": "white"},
                {"age_group": "adult", "gender": "male", "skin_tone": "white"}
            ]
        },
        {
            "title": "Comparison: Black vs. White Skin (Male, Adult)",
            "save_filename": "attention_comparison_black_vs_white.png",

```

```

        "plot_grid": (2, 2),
        "image_criteria": [
            {"skin_tone": "black", "gender": "male", "age_group": "adult"},
            {"skin_tone": "white", "gender": "male", "age_group": "adult"},
        ]
    },
    {
        "title": "Comparison: White vs. Asian Skin (Male, Adult)",
        "save_filename": "attention_comparison_white_vs_asian.png",
        "plot_grid": (2, 2),
        "image_criteria": [
            {"skin_tone": "white", "gender": "male", "age_group": "adult"},
            {"skin_tone": "asian", "gender": "male", "age_group": "adult"},
        ]
    },
    {
        "title": "Comparison: White vs. Indian Skin (Male, Adult)",
        "save_filename": "attention_comparison_white_vs_indian.png",
        "plot_grid": (2, 2),
        "image_criteria": [
            {"skin_tone": "white", "gender": "male", "age_group": "adult"},
            {"skin_tone": "indian", "gender": "male", "age_group": "adult"},
        ]
    },
]

# --- Plotting Loop ---
for job in plotting_jobs:
    print(f"\n--- Generating plot: {job['title']} ---")
    num_images = len(job['image_criteria'])
    rows, cols = job['plot_grid']
    fig, axes = plt.subplots(rows, cols, figsize=(5 * cols, 5 * rows))
    fig.suptitle(job['title'], fontsize=18, fontweight='bold')
    axes = axes.flatten()

    for i, criteria in enumerate(job['image_criteria']):
        # Find a matching image
        subset = df_test
        for key, value in criteria.items():
            subset = subset[subset[key] == value]

        if subset.empty:
            print(f"Warning: No image found for criteria: {criteria}. Skipping.")
            axes[i].text(0.5, 0.5, f"Image not found\n{criteria}", ha='center',
            axes[i].axis('off')
            axes[i + cols].axis('off') # Also turn off the corresponding heatmap
            continue

        sample = subset.sample(1).iloc[0]

        # Generate attention map
        img_path = os.path.join(config.DATA_DIR, sample['filepath'])
        img_pil = Image.open(img_path).convert("RGB")
        img_tensor, meta_idx = eval_transform(img_pil), meta_vocab.encode_row(s
        overlay, pred_idx = generate_attention_map(model, img_tensor, meta_idx)

```

```

    # Get info for titles
    pred_label, true_label = idx_to_class[pred_idx], sample['label']
    subtitle = f"True: {true_label}, Pred: {pred_label}\n" + ", ".join([f"{
    # Plot original and overlay
    # The script plots originals on the first row and heatmaps on the second
    axes[i].imshow(img_pil)
    axes[i].set_title(subtitle)
    axes[i].axis('off')

    axes[i + cols].imshow(overlay)
    axes[i + cols].set_title(f"Attention Map for Image {i+1}")
    axes[i + cols].axis('off')

    # Remove any unused subplots
    for i in range(num_images, cols):
        axes[i].axis('off')
        axes[i + cols].axis('off')

    plt.tight_layout(rect=[0, 0, 1, 0.96])
    plt.savefig(job['save_filename'])
    plt.show()
    print(f"Plot saved as {job['save_filename']}")

if __name__ == '__main__':
    main()

```

Using device: cuda

Model and data loaded successfully.

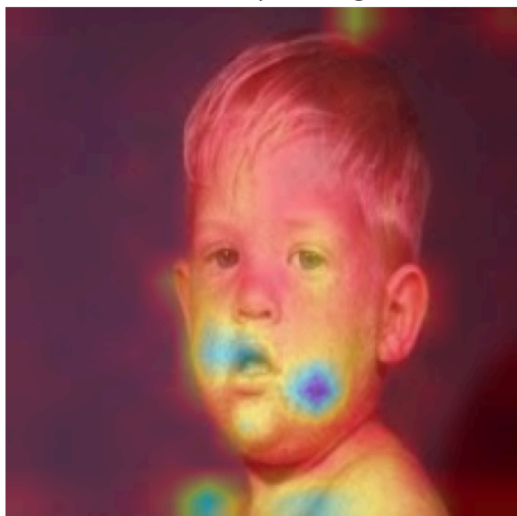
--- Generating plot: Comparison by Gender (Adult, White) ---

Comparison by Gender (Adult, White)

True: Sarampion, Pred: Sarampion
Gender: male, Age Group: adult, Skin Tone: white



Attention Map for Image 1



True: Sarampion, Pred: Sarampion
Gender: female, Age Group: adult, Skin Tone: white



Attention Map for Image 2



Plot saved as attention_comparison_gender.png

--- Generating plot: Comparison by Age Group (Male, White) ---

Comparison by Age Group (Male, White)

True: Sarampion, Pred: Sarampion
Age Group: child, Gender: male, Skin Tone: white



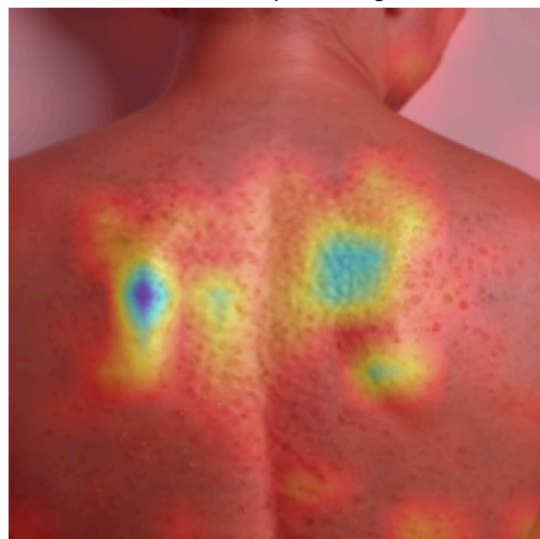
Attention Map for Image 1



True: Monkeypox, Pred: Monkeypox
Age Group: adult, Gender: male, Skin Tone: white



Attention Map for Image 2

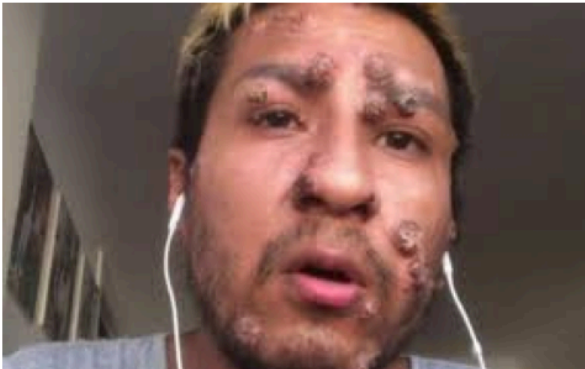


Plot saved as attention_comparison_age.png

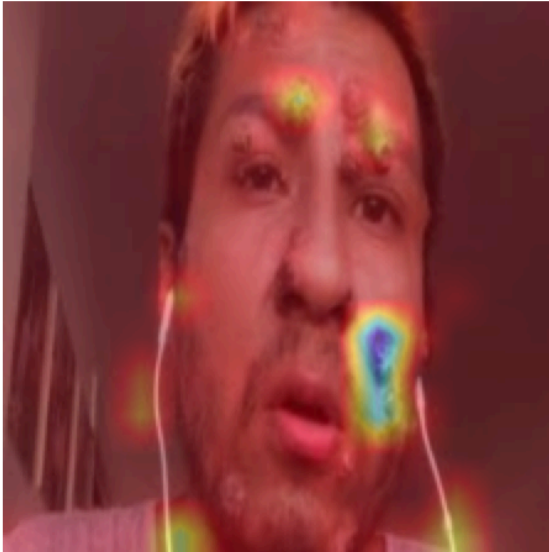
--- Generating plot: Comparison: Black vs. White Skin (Male, Adult) ---

Comparison: Black vs. White Skin (Male, Adult)

True: Monkeypox, Pred: Monkeypox
Skin Tone: black, Gender: male, Age Group: adult



Attention Map for Image 1



True: Sarampion, Pred: Sarampion
Skin Tone: white, Gender: male, Age Group: adult



Attention Map for Image 2



Plot saved as attention_comparison_black_vs_white.png

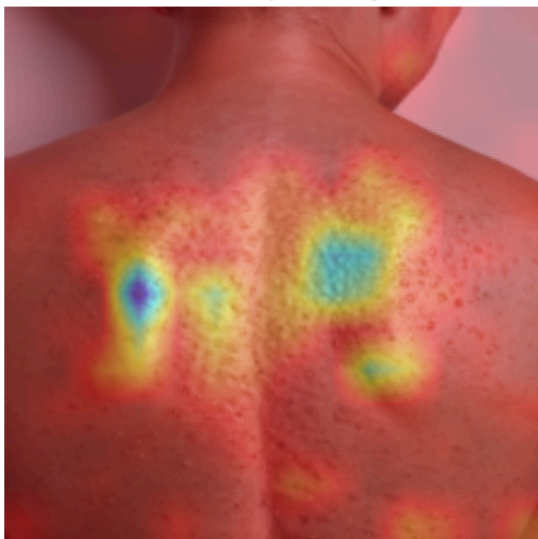
--- Generating plot: Comparison: White vs. Asian Skin (Male, Adult) ---

Comparison: White vs. Asian Skin (Male, Adult)

True: Monkeypox, Pred: Monkeypox
Skin Tone: white, Gender: male, Age Group: adult



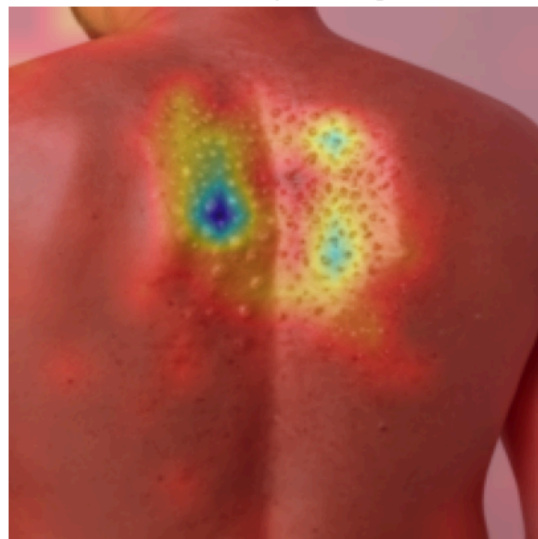
Attention Map for Image 1



True: Monkeypox, Pred: Monkeypox
Skin Tone: asian, Gender: male, Age Group: adult



Attention Map for Image 2



Plot saved as attention_comparison_white_vs_asian.png

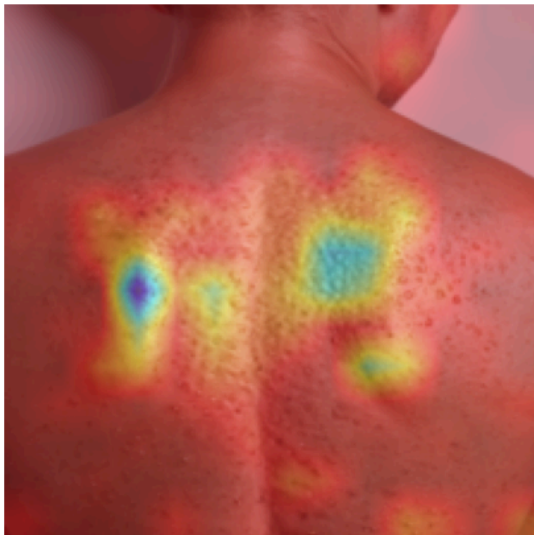
--- Generating plot: Comparison: White vs. Indian Skin (Male, Adult) ---

Comparison: White vs. Indian Skin (Male, Adult)

True: Monkeypox, Pred: Monkeypox
Skin Tone: white, Gender: male, Age Group: adult



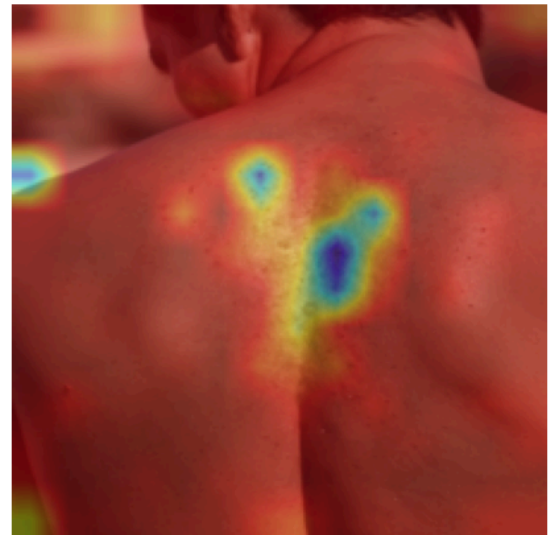
Attention Map for Image 1



True: Sarampion, Pred: Sarampion
Skin Tone: indian, Gender: male, Age Group: adult



Attention Map for Image 2



Plot saved as attention_comparison_white_vs_indian.png

In []:

In []: